API Infrastructure

NHS Pathways Transformation

5 November, 2026

Table of Contents

1 DRAFT Patient Triage API – Red Lines Backup & Restore Test Plan (Pre-Prod & Prod Data) 13
1.1 1. Overview ... 13
1.2 2. Red Lines Scenario Summary (Adapted) ... 13
1.3 3. Architecture Considerations for Backup & Restore ... 14
1.4 4. Epic 1 – Immutable Backup & Restore Design ... 14
1.4.1 4.1 Key Tasks ... 15
1.5 5. Epic 2 – Scenario 1: Restore Pre-Prod into a New Account ... 15
1.5.1 5.1 Account & Base Platform ... 15
1.5.2 5.2 Restore Triage Database ... 16
1.5.3 5.3 Rebuild API & Pathways Uploader ... 16
1.5.4 5.4 Logging, Monitoring & Security Feeds ... 16
1.5.5 5.5 DNS / Routing and External Connectivity ... 16
1.5.6 5.6 Behavioural & Comparative Testing ... 17
1.5.7 5.7 RTO/RPO Measurement ... 17
1.6 6. Epic 3 – Scenario 2: Production Data Restore & Validation ... 17
1.6.1 6.1 Isolated Restore Environment ... 17
1.6.2 6.2 Restore Production Backup ... 17
1.6.3 6.3 Data Integrity Checks ... 18
1.6.4 6.4 Access Control & Data Destruction ... 18
1.6.5 6.5 Restore Time Reporting ... 18
1.7 7. Epic 4 – Runbooks, Evidence & Reporting ... 18
1.7.1 7.1 DR Runbooks ... 18
1.7.2 7.2 Evidence Capture ... 18
1.7.3 7.3 Post-Exercise Review ... 19
1.8 8. Open Questions / Decisions ... 19
2 Green-Blue Deployment - Patient Triage Api - (Proposed) ... 20
2.1 Overview ... 20
2.2 AWS Components ... 20
2.3 API Green/Blue Deployment ... 21

2.3.1 Proposed method 21
2.3.2 Other methods rule out 23
3 Patient Triage Api - Alerting Strategy 25
3.1 Patient Triage API - KPI Alert Levels 25
3.1.1 Purpose 25
3.1.2 Alert Level Definitions 25
3.1.3 Alert Response Summary 27
3.1.4 Alert Implementation Notes 28
3.1.5 Known Gaps Affecting Alert Coverage 28
3.1.6 Related Pages 28
3.2 Patient Triage API KPI and alerting strategy 29
3.2.1 KPI summary 29
3.2.2 KPI coverage 29
3.2.3 Alert coverage 30
3.2.4 Why these metrics meet the strategy 31
3.2.5 Intentional non-alerted metrics 32
3.2.6 Operational conclusion 32
4 Patient Triage Api - Artefact Management Strategy 33
4.1 Purpose 33
4.2 Storage Overview 33
4.3 Path & Directory Structure 34
4.4 Promotion Process 34
4.4.1 1. Build in Development 34
4.4.2 2. Rebuild on Merge to Main 35
4.4.3 3. Promote to Staging 35
4.4.4 4. Build Release 35
4.4.5 5. Promote Release 35
4.4.6 6. Deploy Release 36
4.4.7 Purpose: Ensure production uses verified, immutable builds. Rollback Strategy 37
4.5 Artefact Lifecycle 38
4.6 Artefact Metadata 38
4.6.1 1. build-info.json 38

5 Patient Triage Api - Backup Strategy 39
5.1 DynamoDB Backup Strategy 39
5.1.1 Point-in-Time Recovery (PITR) 39
5.1.2 On-Demand Backups 39
5.1.3 AWS Backup 39
5.1.4 Export to S3 39
5.2 Lambda Backup Strategy 39
5.2.1 Source Code Version Control 39
5.2.2 Infrastructure as Code 39
5.2.3 Lambda Versioning 40
5.2.4 Automated Deployment Pipeline 40
5.3 Recommended Combined Approach 40
6 Patient Triage Api - Clinical Ingestion Steps 41
6.1 Overview 41
6.2 Preconditions 41
6.3 Validation and Extraction Workflow 41
6.4 Detailed Rules 41
6.5 Example Logic (Python) 42
6.6 Failure Handling 42
6.7 Success Criteria 43
7 Patient Triage Api - Dependabot Update Policy 44
7.1 Purpose 44
7.2 Objectives 44
7.3 Update Scope 44
7.4 Schedule 44
7.5 Pull Request Limits 44
7.6 Grouping Rules 45
7.6.1 Patch and Minor Grouping 45
7.6.2 Security Grouping 45
7.7 Labels and Commit Prefix 45
7.8 Governance and Review 45
7.9 Validation Expectations 46

7.10 Known Constraints 46
7.11 Exclusions 46
7.12 Ownership 46
7.13 Revision History 46
8 Patient Triage Api - Disaster Recovery Strategy 47
8.1 Table of Contents 47
8.2 Purpose 47
8.3 Scope 47
8.4 Objectives 48
8.4.1 Data Protection Objectives 48
8.4.2 Service Continuity Objectives 48
8.4.3 Compliance and Operational Objectives 49
8.5 Critical Components & Dependencies 49
8.6 Recovery Objectives 49
8.6.1 Recovery Point Objective (RPO) 49
8.6.2 Recovery Time Objective (RTO) 50
8.7 Key Capabilities 50
8.8 KMS Recovery Strategy 51
8.8.1 KMS Failure Scenarios 51
8.8.2 KMS Recovery Limitations 51
8.8.3 DNS & Traffic Recovery Strategy 51
8.9 Backup & Immutability Strategy 52
8.9.1 Centralised AWS Backup 52
8.10 Disaster Scenarios 53
8.11 Disaster Recovery Test Strategy 54
8.12 Roles & Responsibilities During Incidents 54
8.13 Patient Triage Api Annual Disaster Recovery Tests 56
8.13.1 DR Testing Plans and Results 56
8.13.2 Patient Triage Api Disaster Recovery Plan 2026 56
8.13.3 Patient Triage Api Disaster Recovery Report 2026 58
9 Patient Triage Api - DNS 72
10 Patient Triage Api - Encryption Strategy 73

10.1 Introduction ... 73
10.2 Encryption at Rest ... 73
10.2.1 Centralised Or Decentralised model for managing KMS Keys ... 73
10.2.2 Key Types: Customer Managed, AWS Managed or AWS Owned KMS Keys ... 76
10.2.3 Service-Level Recommendations ... 77
10.2.4 Service-Level Recommendations CMK Key Aliases Adopted ... 92
10.3 Encryption in Transit ... 94
10.4 KMS Compliance with AWS Config ... 95
10.5 Logging ... 96
10.6 Monitoring and Alerting ... 96
10.7 Audits ... 96
10.8 Future Considerations ... 96
10.9 References ... 97
11 Patient Triage Api - Hotfix Strategy (Fix Forward) ... 98
11.1 Purpose ... 98
11.2 Principles ... 98
11.3 Entry Criteria ... 98
11.4 Fix Forward Workflow ... 98
11.5 Step-by-Step Process ... 98
11.5.1 1. Initiation ... 98
11.5.2 2. Implement and Verify ... 99
11.5.3 3. Promote to Reference ... 99
11.5.4 4. Production Deployment ... 99
11.5.5 5. Post-Deployment Actions ... 99
11.6 Ref Validation Gate (Mandatory) ... 99
11.7 Data and Schema Considerations ... 99
11.8 Exit Criteria ... 100
12 Patient Triage Api - Infrastructure Diagrams ... 101
13 Patient Triage Api - Known issues ... 103
13.1 Long APIM Delays ... 103
13.1.1 Description ... 103
13.1.2 Steps take so far ... 104

14 Patient Triage Api - Logging and Monitoring Strategy 105
14.1 Overview 105
14.2 Logging Standards 105
14.2.1 Log Group Naming Convention 105
14.2.2 CloudWatch Retention Policies 106
14.2.3 ODIN Retention Policies (TBC) 106
14.2.4 AWS Service Logging 106
14.2.5 S3 Bucket Logging Matrix 106
14.3 KPI and Alerting Strategy Alignment 107
14.3.1 Alert State Definitions 107
14.3.2 Alert Response Summary 109
14.3.3 Alert Implementation Notes 110
14.3.4 Known Gaps Affecting Alert Coverage 110
14.4 Metrics and Alerts 110
14.4.1 KPI Summary 110
14.4.2 KPI Coverage 110
14.4.3 Alert Coverage 112
14.4.4 Intentional Non-Alerted Metrics 113
14.4.5 Why these metrics meet the strategy 113
14.5 Dashboards 114
14.6 References 114
15 Patient Triage Api - Performance Testing Strategy 115
15.1 Summary 115
15.2 Document Control 115
15.3 1. Test Objectives 115
15.3.1 Primary Objectives 115
15.3.2 Secondary Objectives 116
15.4 2. Performance KPIs and Targets 116
15.4.1 Response Time 116
15.4.2 Throughput and Capacity 116
15.4.3 Reliability Under Load 117
15.5 3. Test Scope and Scale 117

15.5.1 In Scope ... 117
15.5.2 Out of Scope ... 117
15.6 4. Performance Test Types and Approach ... 118
15.6.1 4.1 Baseline Performance Test ... 118
15.6.2 4.2 Load Test - Current Peak Demand ... 118
15.6.3 4.3 Stress Test - Future State Capacity ... 118
15.6.4 4.4 Endurance Test (Soak Test) ... 119
15.6.5 4.5 Spike Test ... 119
15.6.6 4.6 Capacity Test - 2x Normal Traffic ... 119
15.6.7 4.7 Error Scenario Testing ... 120
15.6.8 4.8 Data Layer Performance Testing ... 120
15.7 5. Test Execution Plan and Timeline ... 120
15.7.1 Phase 1: Test Environment Setup and Baseline ... 120
15.7.2 Phase 2: Functional Load Testing ... 120
15.7.3 Phase 3: Stress and Scalability Testing ... 121
15.7.4 Phase 4: Endurance and Soak Testing ... 121
15.7.5 Phase 5: Reports and Recommendations ... 121
15.8 6. Test Environment Requirements ... 121
15.8.1 Infrastructure ... 121
15.8.2 Tools and Technologies ... 121
15.8.3 Test Data Requirements ... 122
15.9 7. Success Criteria and Acceptance ... 122
15.9.1 Critical Pass Criteria (Must Meet) ... 122
15.9.2 Conditional Pass Criteria (Should Meet) ... 122
15.9.3 Fail Criteria (Automatic Failure) ... 123
15.10 8. Monitoring and Observability During Testing ... 123
15.10.1 Metrics to Collect ... 123
15.10.2 Alerting Thresholds During Tests ... 123
15.10.3 Real-Time Dashboards ... 124
15.11 9. Reporting and Analysis ... 124
15.11.1 Test Report Contents ... 124
15.11.2 Comparative Analysis ... 124

15.12 10. Risk and Mitigation 125
15.12.1 Test Risks 125
15.12.2 Performance Risks 125
15.13 11. Assumptions and Dependencies 126
15.13.1 Assumptions 126
15.13.2 Dependencies 126
15.14 12. Sign-Off and Approval 126
15.14.1 Review and Approval 126
15.15 13. References 127
15.16 14. Appendices 127
15.16.1 Appendix A. Test Scenario Reference Guide 127
15.16.2 Appendix B. JMeter Configuration Templates 127
15.16.3 Appendix C. CloudWatch and X-Ray Configuration Checklist 127
15.17 How to create performance test TPS graphs 128
15.17.1 Sign in to AWS console 128
15.17.2 Sign in to AWS CLI 128
15.17.3 Sign in to the DLT solution 129
15.17.4 Go to the run's details 129
15.17.5 Open the bucket 130
15.17.6 Get the S3 URI 130
15.17.7 Synchronise the bucket 131
15.17.8 Extract timestamps and TPS 131
15.17.9 Import into excel 132
15.17.10 Fill in missing seconds 132
15.17.11 Add the transaction counts 133
15.17.12 Add smoothed transaction counts 134
15.17.13 Add target tps 134
15.17.14 Create graph 135
15.18 JMeter Performance Testing Platform — Solution Overview 136
15.18.1 Purpose 136
15.18.2 Background and Decision 137
15.18.3 Architecture Overview 137

15.18.4 Infrastructure Components 138
15.18.5 S3 Folder Structure 140
15.18.6 Test Scenarios 140
15.18.7 How the Automated Workflow Runs 141
15.18.8 How to Run a Test 142
15.18.9 Understanding Results 142
15.18.10 Access Control 143
16 Patient Triage Api - Versioning Strategy 144
16.1 Overview 144
16.2 Semantic Versioning 144
16.2.1 Additional Labels 144
16.3 Initial Version 144
16.4 Automation in GitHub Actions 145
16.5 Version Calculation 145
16.5.1 Commit Message Format 145
16.5.2 Examples 146
16.6 Implementation 146
16.6.1 Formal Releases 146
16.6.2 Pre-releases 147
16.6.3 Release Candidates 147
16.6.4 QA Validation 147
16.7 Environment Promotion Flow 147
16.8 Version Lifecycle 148
16.8.1 Development Phase 148
16.8.2 Production Phase 148
16.9 Best Practices 148
16.10 Rollback Procedures 148
16.10.1 Rollback a Deployment 148
16.11 Clinical Versioning 149
16.11.1 Specifying Clinical Logic 149
16.11.2 Version Consistency and State Management 149
16.11.3 Parallel Versioning 149

16.11.4 Version Retirement Policy 150
16.11.5 Clinical Versioning Best Practices 150
16.11.6 Safety Considerations 151
16.12 References 151

- DRAFT Patient Triage API – Red Lines Backup & Restore Test Plan (Pre-Prod & Prod Data) (see page 13)
- Green-Blue Deployment - Patient Triage Api - (Proposed) (see page 20)
- Patient Triage Api - Alerting Strategy (see page 25)
- Patient Triage Api - Artefact Management Strategy (see page 33)
- Patient Triage Api - Backup Strategy (see page 39)
- Patient Triage Api - Clinical Ingestion Steps (see page 41)
- Patient Triage Api - Dependabot Update Policy (see page 44)
- Patient Triage Api - Disaster Recovery Strategy (see page 47)
- Patient Triage Api - DNS (see page 72)
- Patient Triage Api - Encryption Strategy (see page 73)
- Patient Triage Api - Hotfix Strategy (Fix Forward) (see page 98)
- Patient Triage Api - Infrastructure Diagrams (see page 101)
- Patient Triage Api - Known issues (see page 103)
- Patient Triage Api - Logging and Monitoring Strategy (see page 105)
- Patient Triage Api - Performance Testing Strategy (see page 115)
- Patient Triage Api - Versioning Strategy (see page 144)

1 DRAFT Patient Triage API – Red Lines Backup & Restore Test Plan (Pre-Prod & Prod Data)

1.1 1. Overview

This page summarises how the Patient Triage API will meet the Red Lines Backup Scenario requirements, given its current architecture:

- Stateless API using AWS Lambda for compute:
  - Triage Endpoint (Blue/Green)
  - Triage Engine (Blue/Green)
  - Pathways Uploader / Triage Transformer
- Data:
  - Single Triage Database holding clinical release data only (no live patient or session data).
  - Clinical data is updated ad-hoc via the Pathways Uploader.
  - Backup strategy: automated immutable backup triggered whenever a new clinical release is introduced into the database.
- Supporting services:
  - API Gateway / WAF and upstream API Management proxy (Apigee).
  - KMS and Secrets Manager.
  - CloudWatch, CloudTrail and Kinesis Firehose feeding into ODIN Observability Platform and CSOC.

The purpose of this plan is to define the epics and key tasks required to demonstrate that the Patient Triage API can be recovered from a catastrophic event (e.g. ransomware / malicious insider) in line with Red Lines guidance.

1.2 2. Red Lines Scenario Summary (Adapted)

Red Lines assumes:

- A catastrophic compromise of the target AWS account (or Azure subscription) including:
  - Low confidence in in-account data.
  - CI/CD pipelines are assumed compromised.
- Teams must demonstrate they can:
  a. Scenario 1 – Restore pre-production service into a new account using immutable backups, including:
     - Rebuild of the essential service.
     - Full connectivity to required external systems in non-prod.
     - Comparison of restored pre-prod vs live environment.

b. Scenario 2 – Restore immutable production data into a secure, isolated container and validate data integrity, without impacting live production.

The Patient Triage API has no dynamic/live data, but clinical release data is still safety-critical and must be restorable and verifiable.

1.3 3. Architecture Considerations for Backup & Restore

Key components relevant to backup, restore and DR testing:

- Compute
  - Triage Endpoint (Lambda, Blue/Green)
  - Triage Engine (Lambda, Blue/Green)
  - Triage Transformer (Lambda)
- Data
  - Triage Database containing clinical release content.
  - Pathways Release File Upload (S3 or equivalent) feeding the Transformer.
- Security & Secrets
  - KMS keys.
  - Secrets Manager entries (DB creds, API keys, etc.).
- Ingress / Edge
  - API Management proxy.
  - API Gateway / Triage API.
  - WAF.
- Logging & Monitoring
  - CloudWatch logs.
  - CloudTrail.
  - Kinesis Firehose integrations to ODIN and CSOC.

Because the API is stateless, DR focuses on:

1. Recreating infrastructure and Lambda artefacts from immutable deployment artefacts.
2. Restoring the Triage DB clinical release data from immutable backups.
3. Re-establishing connectivity, logging and security posture in a new account.

1.4 4. Epic 1 – Immutable Backup & Restore Design

Goal: Ensure all stateful elements (clinical data + deployment artefacts + IaC) can be restored independently of the compromised account and CI/CD.

1.4.1 4.1 Key Tasks

- Define backup scope:
  - Identify Triage DB tables/collections containing clinical release data.
  - Define a backup policy triggered per clinical release (plus any baseline if required).
- Implement immutable backups:
  - Use AWS Backup / DB snapshots / S3 Object Lock (or equivalent) to store immutable copies.
  - Store backups in a logically or physically separate backup account/region or immutability vault.
- Immutable deployment artefacts:
  - Store IaC templates for:
    - VPC, security, networking.
    - API Gateway / WAF.
    - Lambdas (Triage Endpoint, Triage Engine, Triage Transformer).
    - Triage DB, KMS, Secrets, logging.
  - Store Lambda packages/container images in a hardened, versioned repository outside the target runtime account.
- Automate backup trigger on release:
  - After Pathways upload + transform + DB write, trigger backup job.
  - Tag each backup with:
    - Clinical release version.
    - Environment (pre-prod / prod).
    - Timestamp.

1.5 5. Epic 2 – Scenario 1: Restore Pre-Prod into a New Account

Goal: Rebuild the full pre-production triage service in a new account, using immutable backups, and verify it behaves equivalently to the existing live system.

1.5.1 5.1 Account & Base Platform

- Provision a new “DR pre-prod” account:
  - Apply standard CCOE/Cloud guardrails.
  - Create VPC, subnets, routing, security groups, IAM structure.
- Document connections to external non-prod systems:
  - API Management proxy.
  - ODIN / CSOC test integrations.
  - Any other dependencies or shared services.

- Record any risks where full non-prod equivalents do not exist.

1.5.2 5.2 Restore Triage Database

- Select a known good immutable backup of the pre-prod Triage DB.
- Restore clinical release data into the new account DB.
- Validate:
  - Record counts and basic referential checks.
  - Correct clinical release version present.
  - No unintended connectivity to production environments.

1.5.3 5.3 Rebuild API & Pathways Uploader

- Deploy via IaC + immutable artefacts:
  - API Gateway / Triage API.
  - WAF (same rules as current pre-prod, or documented differences).
  - Blue/Green Triage Endpoint Lambdas.
  - Blue/Green Triage Engine Lambdas.
  - Pathways Release File Upload component + Triage Transformer Lambda.
- Configure:
  - Lambda env vars, IAM roles, DB connectivity.
  - KMS keys and Secrets Manager entries created fresh in the new account.

1.5.4 5.4 Logging, Monitoring & Security Feeds

- Configure:
  - CloudWatch logs and metrics.
  - CloudTrail for the new account.
  - Kinesis Firehose streams to ODIN / CSOC (or non-prod equivalents).
- Verify:
  - Logs and events are visible in monitoring tools.
  - There is no impact on the production estate.

1.5.5 5.5 DNS / Routing and External Connectivity

- Update DNS/API routing so that:
  - Existing pre-prod endpoints now point to the new environment.
- Ensure that from the perspective of external test consumers:
  - The endpoint behaves identically; the underlying account has changed, not the interface.
- Confirm the environment does not send notifications or updates to production systems.

1.5.6 5.6 Behavioural & Comparative Testing

- Execute regression tests against the new pre-prod:
  - API functionality and triage outcomes.
  - Error handling, authentication, authorisation.
- Test Pathways Uploader in non-prod:
  - Upload a test clinical release file.
  - Confirm transform and DB write succeeds.
- Compare new pre-prod vs live:
  - Infra (IaC diffs, resource inventory).
  - Lambda versions and artefact hashes.
  - Config and IAM policies.
- Document and triage any drift (fix vs accept risk).

1.5.7 5.7 RTO/RPO Measurement

- Capture start time: when the simulated ransomware event is declared.
- Capture end time: when new pre-prod is live and passing agreed test pack.
- Derive:
  - RTO for Scenario 1.
  - RPO: time between last backup and restored state.

1.6 6. Epic 3 – Scenario 2: Production Data Restore & Validation

Goal: Prove we can restore immutable production clinical data into a secure container and validate it, without impacting live production.

1.6.1 6.1 Isolated Restore Environment

- Provision an isolated VPC/account:
  - No connections to production systems.
  - Production-level access controls (RBAC, MFA, approvals).

1.6.2 6.2 Restore Production Backup

- Select an immutable production backup of the Triage DB.
- Restore into the secure environment.
- Record timings (start/end) and dataset size.

1.6.3 6.3 Data Integrity Checks

- Execute automated checks:
  - Diff vs production data where permitted, ignoring any changes after the backup timestamp.
  - Record counts, min/max date ranges for clinical releases.
  - Sampled record-level comparisons.
- Define criteria for pass/fail of the restore.

1.6.4 6.4 Access Control & Data Destruction

- Ensure access is limited to named individuals with production-level approval.
- Once validation is complete:
  - Securely delete the restored environment and Triage DB.
  - Verify deletion in console/API and via audit logs.

1.6.5 6.5 Restore Time Reporting

- Report how long a full production data restore takes under realistic conditions.
- Note bottlenecks (IO, size, tooling) and potential optimisations.

1.7 7. Epic 4 – Runbooks, Evidence & Reporting

Goal: Provide clear, repeatable documentation and evidence for the Red Lines exercise.

1.7.1 7.1 DR Runbooks

- Create runbooks for:
  - Scenario 1 – Pre-prod rebuild in a new account.
  - Scenario 2 – Production data restore & validation.
- Include:
  - Preconditions (Cloud-6 compliance, immutable artefact availability).
  - Roles and responsibilities.
  - Escalation paths and failure handling.

1.7.2 7.2 Evidence Capture

- Define and collect evidence:
  - Screenshots/logs of backup and restore actions.
  - IaC deployment logs.
  - Test reports (API, performance, data validation).

    - RTO/RPO calculations.
- Store evidence in a central location and link from this page.

1.7.3 7.3 Post-Exercise Review

- Hold a short retrospective after each scenario.
- Capture:
    - What worked well.
    - Issues and delays.
    - Improvements to automation, documentation or observability.
- Track follow-up actions as Jira epics/stories.

1.8 8. Open Questions / Decisions

- Target RTO and RPO values for Patient Triage API (to be agreed with service owner).
- Final choice of backup technology for the Triage DB (e.g. AWS Backup vs native snapshots vs others).
- Exact scope of external systems to include in “fully connected” pre-prod (list and status).

2 Green-Blue Deployment - Patient Triage Api - (Proposed)

2.1 Overview

Green/blue deployments provide releases with near zero-downtime and rollback capabilities. The idea behind green/blue deployment is to shift traffic between two identical environments that are running different versions of the application.

Here we refer to green environment as the current active live leg version of Patient Triage API. Once the non-live inactive blue environment leg has been tested and confirmed as functional and stable, traffic will be shifted to it.

The green environment , running with previous version, remains available to roll the deployment back to if an issue arises i.e. with near zero-downtime.

2.2 AWS Components

The below shows the breakdown of AWS components managed separately by each green and blue environment leg as well as AWS components shared by both legs

AWS components managed separately by both green and blue environment stack include:

- AWS Gateway (Rest APIs, Deployment, Response, Resource, Method, Integration, Stage, Custom Domain, Routing Rule)
- AWS CloudWatch (Log Group)
- AWS S3 (mTLS truststore pem object file)
- AWS Route53 (API record)
- AWS IAM (Lambda Invoke Policy)
- AWS Lambda (TriageLambda, StatusAggregatorLambda)

Shared AWS Components include:

- AWS S3 (mTLS truststore bucket, Clinical Ingestion bucket/zip files)
- AWS DynamoDB
- AWS Lambda (IngressLambda, CWLogTransformerLambda, CWMetricsTransformerLambda, CWLogITOCTransformerLambda, CWLogFanoutLambda)
- AWS WAF
- ODIN Splunk forwarder (Log Subscriptions, Metric Stream, Kinesis Firehose)

2.3 API Green/Blue Deployment

Components out of scope:

- Apigee API Proxy - Blue green deployments have been explored by APIM - D038 - Blue-Green deploy process for API Proxies[1] - requires an additional 'router' proxy to direct traffic to a blue/green version of the proxy. Additional complexity and possibly latency. Changes to the API Proxy are infrequent, changes are usually related to authorisation, or allowed paths. These changes can and should be well tested, easily, before the deployment reaches Production. Action: revisit current tests to ensure they cover API Proxy changes well. By not having a blue green deployment of the API Proxy in case of an issue with the new version of the proxy, downtime may be associated with roll back

Any method implemented for a green/blue deployment of the API also needs to ensure we have a consistent API Gateway URL (e.g. api.patient-triage.service.nhs.uk) as in future we may have users connecting directly to our AWS API and not connecting via the Apigee API Proxy.

2.3.1 Proposed method

2.3.1.1 Use Custom Domain Name Routing Rules Targeted at API Gateway ID

This method involves the use of the below key components

- Two (2) API Gateway-Lambda stacks i.e. green and blue environment legs
- Three (3) API Gateway Custom Domain Names i.e. main, green and blue

Smoke tests would be executed over the inactive non-live environment to ensure it is functional before any switch is initiated

Once confirmed, traffic can be routed to the tested non-live environment stack by switching the main custom domain name (api.patient-triage.service.nhs.uk) to the API gateway of the non-live leg deployed with latest version of the code.

While smoke testing the blue/green stack, API Gateway also needs to have custom domain name. This is because ONLY custom domain names can be secured by MTLS.

Pros:

- The green and blue custom domains remain independently available for smoke testing via API Gateway custom domain endpoint before switching over. The smoke tests can make use of most of the current end to end tests to test full stack is functional before switch.
- DNS Caching should not be an issue with this method as we are updating the AWS Custom Domain Name mapping (something internal to AWS) and not making any changes to the route 53 records that a client would use.

Cons:

1. https://nhsd-confluence.digital.nhs.uk/spaces/APM/pages/171844371/D038+-+Blue-Green+deploy+process+for+API+Proxies

- Terraform state dependency i.e. switching over process and pipeline is tied to terraform apply which can take a couple of minutes to complete plan and apply cycle.
  Also, if another pipeline is running which makes use of the same terraform state, the switchover or rollback process can be blocked until the terraform state lock is released.
- API Gateway routing rule propagation is not truly instantaneous. Although it is faster than DNS, the API Gateway routing rule updates still have a small internal propagation delay (couple of seconds) across AWS edge nodes.
  In flight requests, at the moment of switching over, may be handled by either deployment.
- All or nothing switch i.e. we are not able to split traffic by sending a small percentage % of traffic to newly tested stack for validation before full switchover.
- Operational overhead in managing the three custom domains (main, green and blue) and ACM certificate associated with them
- APIM status or smoke tests cannot be carried on non-live leg i.e. before switching over. Status or smoke tests can only be carried on the green and blue API gateway custom domain names.

How to switch the custom domain name to point to different stacks

- The main API Gateway custom domain ( ) is decoupled from the green and blue API gateway deployments via routing rules.
- The green and blue API gateway deployments each has its own dedicated custom domain which remains static throughout the deployment lifecycly
- Before switching, the new deployment is validated and tested independently via its dedicated custom domain e.g. for blue leg blue-<env>.patient-triage.service.nhs.uk[2], without any impact to the live traffic still flowing through the main domain.
- Once the new deployment is confirmed healthy, the switchover is performed by amending the routing rules on the main custom domain to point to the newly tested API gateway deployment and stage.
- Since the switch is an internal AWS API Gateway control plane operation, there is no DNS record change and therefore no DNS propagation delay or client-side cache impact.
  The Route 53 record for api.patient-triage.service.nhs.uk[3]remains unchanged throughout.
- The previously active deployment (green) remains fully operational on its dedicated custom domain, serving as an immediate rollback target, should issues be detected after switching.

Key considerations during implementation

- It was observed that the same S3 file for the truststore certificate cannot be used for multiple custom domains.
  The workaround is to create multiple mTLS truststore files, each containing the same CA information until AWS allow same file be used for multiple custom domains.

2. http://api.patient-triage.service.nhs.uk
3. http://api.patient-triage.service.nhs.uk

2.3.2 Other methods rule out

2.3.2.1 Use Apigee deployment to switch

Traffic could be switched to the non-live stack by redeploying the API Proxy, with the API Proxy now forwarding traffic to the blue API Gateway custom domain.
Here we would be required to modify the url for the API Proxy target server in Apigee. Both blue and green stack API Gateways would require their own custom domain names.

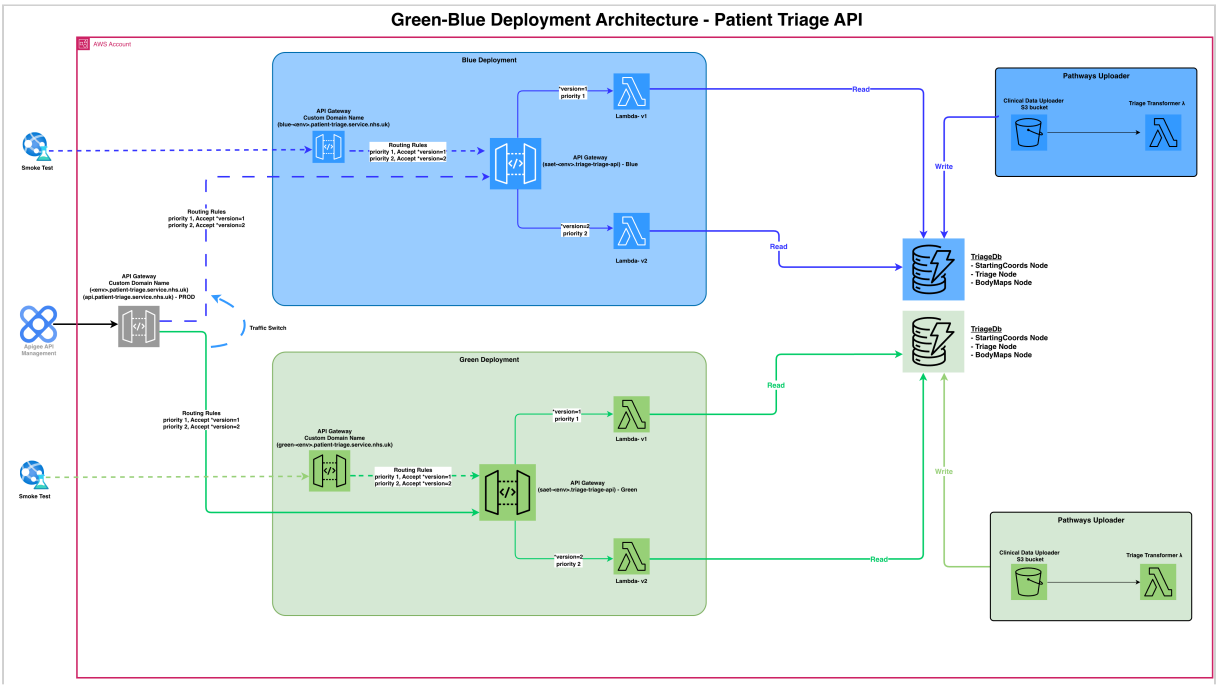
This method was ruled out due to the requirement to have a consistent API Gateway URL and also avoiding the need to involve APIM with our green-blue deployments.

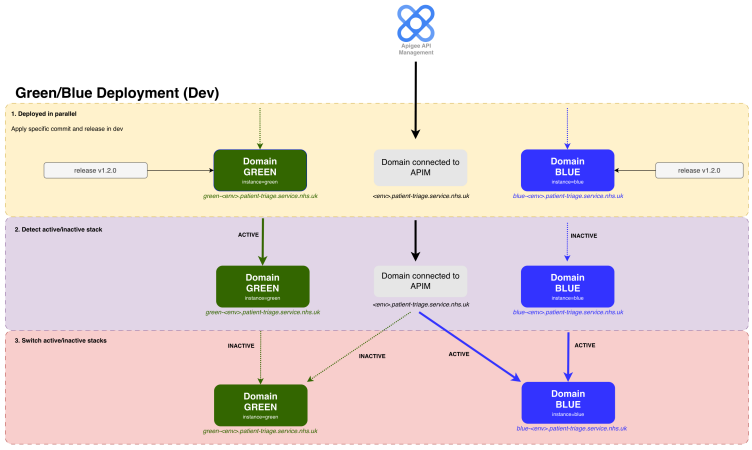
Method:

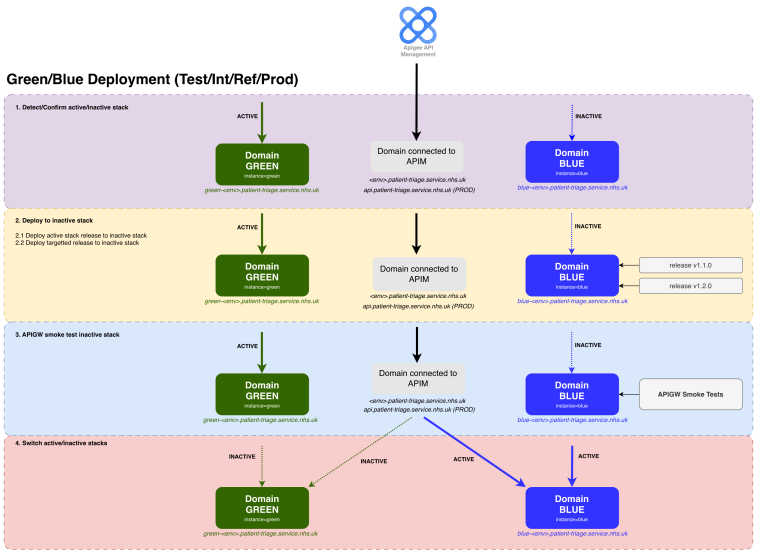
1. Green stack is active
2. Deploy release to non-live Blue stack
3. Run smoke tests against Blue.
4. Deploy Apigee changes. Specify Blue as the active stack
5. Smoke test
6. To roll back, redeploy apigee, specifying Green as the active stack

2.3.2.2 Use API Gateway stages to switch

Api gateway stages are already used to manage application versions so this was ruled out for use in managing green-blue deployments.




Apigee API Management
Green/Blue Deployment (Dev)
1. Deployed in parallel
Apply specific commit and release in dev
release v1.2.0
Domain GREEN
instance=green
green-<env>.patient-triage.service.nhs.uk
Domain connected to APIM
<env>.patient-triage.service.nhs.uk
Domain BLUE
instance=blue
blue-<env>.patient-triage.service.nhs.uk
release v1.2.0
2. Detect active/inactive stack
ACTIVE
INACTIVE
3. Switch active/inactive stacks
INACTIVE
ACTIVE


Apigee API Management
Green/Blue Deployment (Test/Int/Ref/Prod)
1. Detect/Confirm active/inactive stack
ACTIVE
INACTIVE
Domain GREEN
instance=green
green-<env>.patient-triage.service.nhs.uk
Domain connected to APIM
<env>.patient-triage.service.nhs.uk
api.patient-triage.service.nhs.uk (PROD)
Domain BLUE
instance=blue
blue-<env>.patient-triage.service.nhs.uk
2. Deploy to inactive stack
2.1 Deploy active stack release to inactive stack
2.2 Deploy targetted release to inactive stack
release v1.1.0
release v1.2.0
3. APIGW smoke test inactive stack
APIGW Smoke Tests
4. Switch active/inactive stacks

3 Patient Triage Api - Alerting Strategy

3.1 Patient Triage API - KPI Alert Levels

3.1.1 Purpose

This page defines the alert thresholds for each KPI established in the Patient Triage API - Key Performance Indicators (see page 25) page. Each KPI is assigned three alert states:

- ✅ **GREEN** — Service operating within target. No action required.
- ⚠️ **AMBER** — Service degraded or approaching breach. Investigation required.
- ❌ **RED** — SLA breach or imminent breach. Immediate escalation required.

Alert thresholds are derived from the NFR-defined targets. AMBER thresholds act as early-warning triggers to allow corrective action before a RED state is reached. All production alerts on this page are implemented and notified via ODIN / Grafana.

3.1.2 Alert Level Definitions

3.1.2.1 Availability

KPI	GREEN	AMBER	RED	Alert mechanism	Source NFR
Service availability	>= 99.90% in the rolling 30-day window	99.50% – 99.89%	< 99.50%	ODIN / Grafana availability alert rule. Alert fires when 30-day rolling availability drops below 99.90% (CloudWatch metric used as telemetry source only).	NFR: Availability[4]

3.1.2.2 Recovery

Recovery objectives are time-tracked during declared major incidents. Alert states represent progress against RTO and RPO commitments rather than continuous metric thresholds.

4. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208308/NFR+Availability

KPI	GREEN	AMBER	RED	Alert mechanism	Source NFR
Recovery Time Objective (RTO) — 2 hours	Service restored within 90 minutes of incident declaration	Service not restored within 90 minutes of incident declaration	Service not restored within 2 hours of incident declaration	ODIN / Grafana incident-timer alert integrated with the incident management tool. AMBER Teams message to lead Developer at T+90 min. RED escalated to service owner at T+2 hours.	NFR: Availability[5]

3.1.2.3 Performance

KPI	GREEN	AMBER	RED	Alert mechanism	Source NFR
P95 response time — Triage POST endpoint	< 300 ms	300 ms – 500 ms	> 500 ms	ODIN / Grafana P95 latency alert rule. AMBER fires when P95 crosses 300 ms (the NFR-defined service target). RED fires at 500 ms, which is the load-test ceiling defined in the scalability NFR. This will be evaluated over a 5-minute rolling window. AWS X-Ray used for root-cause analysis.	NFR: Performance[6]
Overall error rate during peak load	< 0.1 %	0.1% – 0.5%	> 0.5 %	ODIN / Grafana error-rate alert rule evaluated over a 5-minute rolling window during peak traffic hours.	NFR: Performance[7]
5xx error rate under normal load	<= 0.01%	0.01% – 0.05%	> 0.05%	ODIN / Grafana 5xx alert rule evaluated over a 5-minute rolling window under normal (non-peak) operating conditions. Separate alert from peak-load error rate to allow independent tuning.	NFR: Performance[8]

5. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208308/NFR+Availability
6. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208291/NFR+Performance
7. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208291/NFR+Performance
8. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208291/NFR+Performance

3.1.2.4 Capacity and Throughput

KPI	GREEN	AMBER	RED	Alert mechanism	Source NFR
Observed throughput against peak demand baseline (6.56 TPS)	< 9 TPS (below 1.4x baseline; autoscaling not yet required)	9 TPS – 13 TPS (1.4x – 2x baseline; autoscaling in progress, confirm scaling response)	> 13 TPS sustained without confirmed autoscale completion (2x baseline exceeded before scaling is confirmed healthy)	ODIN / Grafana TPS alert rule on the Triage POST endpoint. AMBER used to confirm autoscaling is responding before the traffic resilience target is breached.	NFR: Scalability[9]
Observed throughput against capacity ceiling (120 TPS)	< 80 TPS (< 67% of design capacity)	80 TPS – 100 TPS (67% – 83% of design capacity)	> 100 TPS (> 83% of design capacity; approaching the load-test ceiling with limited headroom)	ODIN / Grafana TPS alert rule. AMBER provides an early warning that demand is closing on the validated load-test ceiling, triggering a capacity review.	NFR: Scalability[10]
Traffic resilience — sustained load vs normal volume	Sustained traffic <= 1.5x normal volume, service healthy	Sustained traffic 1.5x – 2x normal volume, service health under review	Sustained traffic >= 2x normal volume with degraded health indicators (latency or error rate entering AMBER or RED)	ODIN / Grafana composite alert combining TPS metric and health indicators. Triggered when both conditions are met simultaneously.	NFR: Scalability[11]

3.1.3 Alert Response Summary

Alert state	Expected response	Response time target
GREEN	No action required. Monitor via ODIN / Grafana dashboard on normal operational cadence.	N/A
AMBER	Engineer investigates and determines whether corrective action is needed. Incident opened if the AMBER state persists for more than 15 minutes or is trending toward RED.	Acknowledge within 15 minutes
RED	Incident declared. Engineer engages immediately. Service owner notified. Escalation path followed per the Disaster Recovery Runbook.	Acknowledge within 5 minutes

9. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208298/NFR+Scalability
10. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208298/NFR+Scalability
11. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208298/NFR+Scalability

3.1.4 Alert Implementation Notes

- All continuous metric alerts (latency, error rate, throughput) should be configured in ODIN / Grafana with evaluation periods of at least 5 minutes to reduce noise from transient spikes.
- Availability and maintenance thresholds are calculated over a rolling 30-day window unless stated otherwise.
- CloudWatch remains an approved telemetry source for infrastructure and application metrics, but alerting ownership and notification routing sits in ODIN / Grafana.
- The AMBER threshold for P95 latency (300 ms) is deliberately aligned with the NFR-defined service target so that any breach of the target immediately generates an operational warning.
- The RED threshold for P95 latency (500 ms) is aligned with the scalability NFR load-test ceiling, representing the upper boundary of validated service behaviour.
- ODIN dashboards should reflect the GREEN / AMBER / RED boundaries defined on this page so that operators have consistent visual alignment between the dashboard and the alert states.

3.1.5 Known Gaps Affecting Alert Coverage

Gap	Impact on alerting	Action required
Autoscaling response time target is TBC	No specific AMBER or RED threshold can be set for scaling speed. Current alert covers the downstream effect (throughput and latency) rather than scaling speed directly.	Confirm target and add a dedicated autoscaling-speed alert once the value is agreed.
Cost / budget threshold is undefined	No financial alert threshold can be published. AWS Cost Anomaly Detection is in place as a best-effort control.	Agree a budget threshold and add a cost alert to this page.

3.1.6 Related Pages

- Disaster Recovery Runbook
- NFR: Observability, Monitoring, Metrics, Dashboards and Alerting[12]
- NFR: Performance[13]
- NFR: Scalability[14]
- NFR: Availability[15]

12. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208069/NFR+Observability+Monitoring+Metrics+Dashboards+and+Alerting
13. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208291/NFR+Performance
14. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208298/NFR+Scalability
15. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208308/NFR+Availability

3.2 Patient Triage API KPI and alerting strategy

This page explains how the Patient Triage API dashboard and alert rules work together to meet the application's KPIs and support the alerting strategy.

3.2.1 KPI summary

The application KPIs are centered on service health, error rate, traffic, and latency. The dashboard gives visibility into those KPIs, and the alerts page when the KPI moves outside the acceptable range. In practice, that means:

- Availability alerts protect the user-facing success rate over time.
- Error-rate alerts catch sudden degradation before it becomes a wider incident.
- Traffic alerts identify overload, no-traffic, or anomalous load conditions.
- Latency and response-class panels provide supporting evidence for investigation, even where they are not directly alerted.

The intent is to keep the alert set focused on actionable KPI breaches rather than every metric fluctuation.

3.2.2 KPI coverage

The dashboard at **Patient Triage API - Service KPI Overview** is the KPI view used to interpret the alerts.

KPI	Dashboard coverage	Alert coverage	Why this meets the KPI
Availability	Service Availability Over 30 Days	Service Availability 99.9% and Service Availability 99.5%	Keeps the service within the expected availability target and raises an alert when the 30-day outcome drops below the acceptable threshold.
Error rate	5xx Rate, Peak Error Rate, Response Class Split	Peak Error Rate > 0.1%, Peak Error Rate > 0.5%, 5xx Rate	Detects when failure rates rise above the acceptable operating band and gives enough signal to distinguish early warning from serious degradation.
Traffic / throughput	Current TPS (5min), Current TPS vs Baseline, Throughput Trend	Current TPS > 9 TPS, Current TPS > 13, Current TPS = 0	Detects unusual traffic movement, overload, or complete traffic loss so the team can respond before the KPI turns into a service issue.
Latency	Latency Signals (Avg/ Sum), P50 response time, P95 response time, P99 response time, Triage Latency Avg (Proxy)	None directly; used for triage	Keeps latency visible for operational review and incident diagnosis, even though latency is not currently a paging KPI.

Dashboard panel	Purpose	How it supports the alerting strategy
Service Availability Over 30 Days	Long-window availability view	Matches the availability alerts and shows whether the service is meeting its SLO target over time.
4xx Rate	Client-error trend	Useful for diagnosis and release validation, but not a paging signal in this strategy.
5xx Rate	Server-error trend	Mirrors the critical error condition behind the 5xx alert.
Current TPS (5min)	Current request volume	Confirms whether traffic is healthy, overloaded, or unexpectedly absent.
Current TPS vs Baseline	Current traffic versus expected baseline	Helps distinguish a real traffic drop from normal variation.
Peak Error Rate	Error burst indicator	Supports the higher-severity error-rate alerts.
Latency Signals (Avg/Sum)	Latency trend context	Provides investigation context for performance issues, even though latency is not currently an alert trigger.
P95 response time	High-latency view	Shows tail latency for triage and root-cause analysis.
P50 response time	Median latency view	Helps separate broad degradation from tail-only issues.
P99 response time	Extreme tail-latency view	Supports investigation of user-visible spikes.
Triage Latency Avg (Proxy)	Additional latency proxy	Gives another signal for confirming latency-related incidents.
Throughput Trend	Traffic history	Shows traffic shape and supports post-incident analysis.
Response Class Split	Response mix over time	Helps explain whether a degradation is dominated by 2xx, 4xx, or 5xx outcomes.

3.2.3 Alert coverage

The alert rules in **Alerts.json** are built from the same Prometheus metrics used in the dashboard, which keeps the KPI view and the paging rules aligned.

Alert	Trigger	Severity intent	Strategy fit
Service Availability 99.9%	Rolling 30-day availability drops below 99.9%	Highest importance availability breach	Directly enforces the availability KPI.
Service Availability 99.5%	Rolling 30-day availability drops below 99.5%	Lower availability threshold for extended degradation	Provides a second availability boundary so the team can distinguish warning-level from critical KPI drift.
Peak Error Rate > 0.1%	Peak 5xx + 4xx error rate exceeds 0.1% on the ref workload slice	Early warning	Detects KPI deterioration before it becomes broad user impact.
Peak Error Rate > 0.5%	Peak 5xx + 4xx error rate exceeds 0.5% on the ref environment	Higher-severity error condition	Escalates when the error KPI is clearly outside the acceptable operating range.
5xx Rate	5xx rate exceeds 0.01% over the last 5 minutes	Immediate server-error signal	Protects the error KPI from server-side failures that users would feel first.
Current TPS > 9 TPS	Request rate rises above 9 TPS	Load anomaly	Flags a traffic KPI shift that may require capacity or dependency checks.
Current TPS > 13	Request rate rises above 13 TPS	Higher load anomaly	Escalates sustained or bursty load above the lower traffic KPI band.
Current TPS = 0	Request rate falls to zero	Availability or intake failure signal	Detects complete traffic loss, which can indicate an outage, upstream routing problem, or broken integration path.

3.2.4 Why these metrics meet the strategy

The dashboard and alerts are aligned around the main application KPI questions.

1. Is the availability KPI healthy? The 30-day availability panel and availability alerts answer this.
2. Is the error KPI healthy? The 5xx and peak error panels plus the error-rate alerts answer this.
3. Is the traffic KPI healthy? The TPS panels and traffic alerts answer this.

That combination is effective because it covers both symptom detection and investigation context:

- Availability alerts cover the SLO outcome that backs the availability KPI.
- Error-rate alerts catch direct user impact and burst failures that move the error KPI.
- TPS alerts catch traffic anomalies that often explain or precede KPI movement.

- Latency and response-class panels help determine whether the KPI issue is latency, saturation, dependency failure, or a response-mix shift.

3.2.5 Intentional non-alerted metrics

Some dashboard metrics are intentionally not used as alerts.

- 4xx rate is visible for troubleshooting, but it is usually more useful as a diagnostic trend than a paging condition.
- P50, P95, P99, and latency proxy panels are retained for triage, but latency alone is not currently part of the paging KPI strategy in these rules.
- Response class split and throughput trend provide supporting context rather than independent alert conditions.

This keeps the alert set focused and reduces noise while preserving the evidence needed to understand incidents quickly.

3.2.6 Operational conclusion

Taken together, the dashboard and alert rules meet the application KPIs by:

- monitoring the availability, error, traffic, and latency KPIs in one place,
- alerting on the highest-signal KPI breaches,
- detecting traffic anomalies that often explain incidents, and
- keeping enough dashboard context to support fast triage without paging on every symptom.

4 Patient Triage Api - Artefact Management Strategy

Version	1.1
Last Updated	March 2026
Owner	Platform Team (Infrastructure engineers across all teams)

4.1 Purpose

This document defines how build artefacts are **stored, versioned, and promoted** in AWS S3 for deployments across multiple environments. The strategy ensures **traceability, reproducibility**, and **efficient lifecycle management** of artefacts, supporting development, QA, and production workflows.

4.2 Storage Overview

```
s3://patient-triage-api-mgmt-artefacts-bucket/
 ├─ development/dev/
 |   ├─ <Workspace>/
 |   |   ├─ read-only-viewer-server.zip
 |   |   ├─ read-only-viewer-assets.zip
 |   |   ├─ lambda_function.zip
 |   |   └─ build-info.json ← minimal metadata pointing to commit hash
 |   └─ latest/
 |       ├─ read-only-viewer-server.zip
 |       ├─ read-only-viewer-assets.zip
 |       ├─ lambda_function.zip
 |       └─ build-info.json
 ├─ staging/test/
 |   ├─ ${prerelease_version}/
 |   |   ├─ read-only-viewer-server.zip
 |   |   ├─ read-only-viewer-assets.zip
 |   |   ├─ lambda_function.zip
 |   |   └─ build-info.json
 ├─ release-candidates/int/ & release-candidates/ref/
 |   ├─ v<semver>-rc.n/
 |   |   ├─ read-only-viewer-server.zip
 |   |   ├─ read-only-viewer-assets.zip
 |   |   ├─ lambda_function.zip
 |   |   └─ build-info.json ← full metadata for the release
 ├─ releases/prod/
 └──── v<semver>/
       ├─ read-only-viewer-server.zip
       ├─ read-only-viewer-assets.zip
```

```
        ├─ lambda_function.zip
        └─ build-info.json ← full metadata for the release
```

Storage Service	AWS S3 (central management AWS account)
Bucket Name	patient-triage-api-mgmt-artefacts-bucket
Access Pattern	Cross-account read-only for deployment accounts

4.3 Path & Directory Structure

Key Principles:

- **Development builds:**
- **Main branch rebuilds:** Merging to main triggers a rebuild; this build is the source for staging promotion.
- **Staging builds:** Track commit hashes for QA/testing; optionally maintain a `latest/` pointer.
- **Release builds:** Use semantic version tags (`vX.Y.Z`).
- **Promotion:** Copy artefacts between directories (`development → staging → releases`) rather than rebuild.
- **Release Candidates (RCs):** Optional intermediate releases for production-like validation (vX.Y.Z-rc.n).

4.4 Promotion Process

4.4.1 1. Build in Development

Trigger: Push to feature/task branch
Action:

- Conduct appropriate tests, scans, checks, SonarCloud analysis etc
- Build artefact
- Integration test (LocalStack)

Purpose: Enables early testing before merging to main.

4.4.2 2. Rebuild on Merge to Main

Trigger: Merge PR to main following successful test, scans, SonarCloud analysis
Action:

- CI/CD pipeline rebuilds artefacts for the merged commit
- Generate build-info.json containing commit SHA, build timestamp etc
- Upload to development/dev/<commit_sha>/
- Upload to development/dev/latest/ (overwrite)
- Auto deploy to development environment

Purpose: Ensures reproducible, tested artefacts are derived solely from the main branch.

4.4.3 3. Promote to Staging

Trigger: Manual "Successful rebuild and deployment on `main` + Artefacts and build-info verified"
Action:

- Copy artefacts from `development/dev/latest/` → `staging/test/<prerelease_version>/`
- Manual deploy to test environment
- QA/Functional Test

Purpose: QA validation of the exact main branch build prior to release.

4.4.4 4. Build Release

Trigger: Manual “QA sign-off, Functional test pass, **Build Release Workflow**”
Action:

- Copy artefacts from `staging/test/<prerelease_version>/` → `release-candidates /int/vX.Y.Z-rc.n/`
- Copy artefacts from `staging/test/<prerelease_version>/` → `release-candidates /ref/vX.Y.Z-rc.n/`
- Manual deployment to Int and Ref environments
- Partner Integration and reference testing

Purpose: Provides immutable, versioned artefacts for production deployment.

4.4.5 5. Promote Release

Trigger: Manual “Partner integration and reference test passed, **Promote Release Workflow**”
Action:

- Copy artefacts from latest RC `release-candidates /ref/ vX.Y.Z-rc.n/` → `releases/prod/vX.Y.Z/`

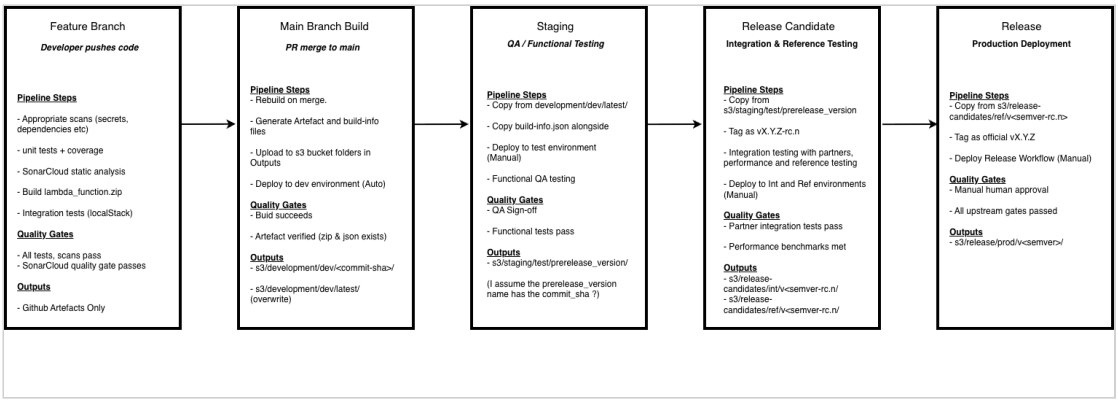
Purpose: Produce the final immutable artefacts for production.

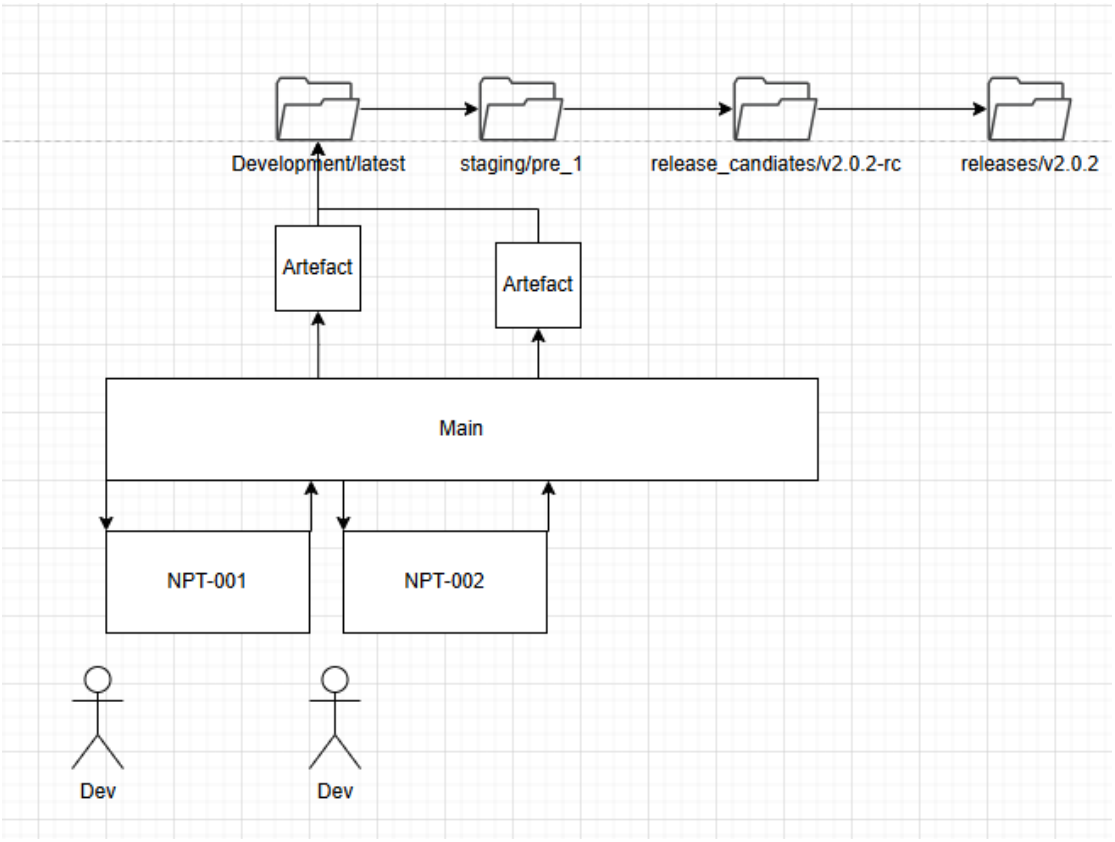
4.4.6 6. Deploy Release

Trigger: Manual “Necessary approval in place, **Deploy Release Workflow**”
Action:

- Uses artefacts from `releases/prod/vX.Y.Z/`
- Manual deploy to Production

4.4.7 Purpose: Ensure production uses verified, immutable builds.





Rollback Strategy

If rollback is required:

1. Identify previous version in `releases/prod/v<semver>/`.
2. Redeploy directly from that artefact.
3. No rebuild or re-promotion required.
   → Ensures production always deploys verified, immutable builds.

4.5 Artefact Lifecycle

`development/`	30 days (excluding /latest)	Automatically expired via S3 lifecycle.
`staging/`	90 days (excluding last 5)	For QA and pre-production testing.
`releases/`	Keep all releases indefinitely	All production releases are immutable and must remain permanently available to support traceability, compliance, and reliable rollback.

4.6 Artefact Metadata

4.6.1 1. `build-info.json`

- **Scope:** Single build / commit
- **Purpose:** Tracks commit SHA, build timestamp, semantic version (if applicable)
- **Location:** In each artefact directory (e.g., `development/<commit_sha>/build-info.json`)

Example:

```
{
  "git_commit": "a1b2c3d4",
  "build_timestamp": "2025-10-28T12:34:56Z",
  "semantic_version": null
}
```

Similar ADR: ADR-053: Use a central S3 bucket for artefacts[16]

16. https://nhsd-confluence.digital.nhs.uk/spaces/CSP/pages/564757441/ADR-053+Use+a+central+S3+bucket+for+artefacts

5 Patient Triage Api - Backup Strategy

5.1 DynamoDB Backup Strategy

5.1.1 Point-in-Time Recovery (PITR)

- Turned on for DynamoDB to aid faster recovery times.

5.1.2 On-Demand Backups

- Create manual backups before major changes
- Retain indefinitely until manually deleted

5.1.3 AWS Backup

- Automated backups will be created as part of the deployment pipeline.
- A backup must be taken before the release can proceed.

5.1.4 Export to S3

- Backups will be exported to S3 as part of the deployment process.
- Periodic exports for long-term archival and analytics
- Cost-effective for compliance requirements

5.2 Lambda Backup Strategy

5.2.1 Source Code Version Control

- Store all Lambda code in GitHub
- Tag releases corresponding to deployed versions
- This is the primary backup
- The backup process of Github repositories is owned and operated by the Engineering authority. The current schedule is daily at 4pm with a 30 days retention period. Github repository restore process[17]

5.2.2 Infrastructure as Code

- Use Terraform
- Version control your IaC templates

17. https://nhsd-confluence.digital.nhs.uk/spaces/CIS/pages/1113384241/Github+repository+restore+process

- Ensures complete function configuration is recoverable (environment variables, IAM roles, triggers)

5.2.3 Lambda Versioning

- Publish Lambda versions for each deployment
- Use aliases for environment management (dev, staging, prod)
- Versions are immutable snapshots

5.2.4 Automated Deployment Pipeline

- CI/CD pipeline can quickly redeploy from source
- Store deployment scripts in version control

5.3 Recommended Combined Approach

Backup Schedule
1. **At Deployment**: DynamoDB backed up

6 Patient Triage Api - Clinical Ingestion Steps

6.1 Overview

This document describes the ingestion checks required before extracting an incoming ZIP file.

6.2 Preconditions

- The ZIP file has been received and is accessible.
- The expected number of files in the ZIP is known in advance.
- The maximum allowed file size is 10 MB per file.
- The extracted content includes an XML hash manifest that defines hash type and checksums per file.

6.3 Validation and Extraction Workflow

1. Open the ZIP file.
2. Use `filezip.namelist()` to retrieve all file names inside the ZIP.
3. Verify that the number of files returned by `filezip.namelist()` matches the expected number of files.
4. Verify that none of the files in the ZIP are larger than 10 MB.
5. If all validations pass, extract the ZIP contents.
6. Read the XML hash manifest from the extracted files.
7. Get the hash type from the XML (for example, `SHA-256`).
8. For each extracted file listed in the XML, generate a checksum using the same hash type.
9. Compare each generated checksum to the checksum value provided in the XML.
10. If all checksum values match, continue ingestion.
11. If any validation fails, stop processing and raise an ingestion error.

6.4 Detailed Rules

- **File list retrieval**: Use `filezip.namelist()` as the source of truth for entries in the ZIP.
- **File count check**: `len(filezip.namelist())` must equal `expected_file_count`.
- **File size check**: Every file must be less than or equal to 10 MB (10 * 1024 * 1024 bytes).
- **Extraction condition**: Extraction is allowed only when both file count and file size checks pass.
- **Hash manifest format**: After extraction, parse XML in the form:

```
<Hashes>
    <Type>SHA-256</Type>
    <Files>
        <file1>checksum</file1>
        <file2>checksum</file2>
        <file3>checksum</file3>
    </Files>
</Hashes>
```

- **Hash algorithm rule**: All extracted files use the same hash algorithm specified by `<Type>`.
- **Checksum verification**: For each file listed under `<Files>`, compute the checksum using the declared type and compare with XML value.
- **Tamper detection**: If any checksum does not match, fail ingestion and treat file set as potentially tampered.

6.5 Example Logic (Python)

1. Open the ZIP file.
2. Call `filezip.namelist()` to get the list of files in the ZIP.
3. Validate that the file count equals `expected_file_count`.
4. Validate that each file size is less than or equal to 10 MB.
5. If pre-validation passes, extract the ZIP contents to the target location.
6. Read and parse the XML hash manifest from the extracted files.
7. Read the hash type from `<Type>` (for example, `SHA-256`).
8. Read expected checksums from `<Files>` where each filename maps to a checksum.
9. For each extracted file listed in the XML, compute checksum using the declared hash type.
10. Compare computed checksum to expected checksum from XML.
11. If all checksums match, continue ingestion.
12. If any validation or checksum check fails, stop ingestion and raise an error.

6.6 Failure Handling

- Log validation failure details, including ZIP name and reason.
- Do not partially extract files when validation fails.
- Quarantine or remove extracted files when checksum validation fails.
- Mark ingestion run as failed and notify downstream monitoring/alerting.

6.7 Success Criteria

- ZIP contains exactly the expected number of files.
- No file in the ZIP exceeds 10 MB.
- ZIP contents are extracted only after pre-extraction checks pass.
- XML manifest hash type is read successfully.
- Each extracted file checksum matches the checksum defined in the XML manifest.

7 Patient Triage Api - Dependabot Update Policy

7.1 Purpose

This policy defines how dependency updates are managed for the patient-triage-api-develop repository using Dependabot.

7.2 Objectives

- Keep third-party dependencies current and reduce exposure to known vulnerabilities.
- Prioritize safe automation with patch and minor updates.
- Ensure security advisories are handled through dedicated security update grouping.

7.3 Update Scope

Dependabot is configured for the following ecosystems:

- GitHub Actions
- Docker (scripts/docker)
- pip (repository root)
- Terraform (common, environments, local, modules, remote, and stacks)

7.4 Schedule

All update checks run weekly on Monday (Europe/London timezone):

- GitHub Actions: 06:00
- Docker: 06:15
- pip: 06:30
- Terraform: 06:45

7.5 Pull Request Limits

- GitHub Actions: 5 open PRs
- Docker: 5 open PRs
- pip: 5 open PRs
- Terraform: 10 open PRs per configured directory

7.6 Grouping Rules

7.6.1 Patch and Minor Grouping

- GitHub Actions: github-actions-patch-minor
- pip: python-patch-minor
- Terraform: terraform-patch-minor

Each of these groups allows only:

- minor
- patch

7.6.2 Security Grouping

Security updates are explicitly grouped using:

- applies-to: security-updates
- patterns: *

Configured security groups:

- github-actions-security
- docker-security
- python-security
- terraform-security

7.7 Labels and Commit Prefix

- All Dependabot PRs include dependency labels per ecosystem (for example dependencies, terraform, python, docker, ci).
- Commit message prefix is set to `deps`.

7.8 Governance and Review

- Security update PRs should be triaged first.
- Patch updates should normally be fast-tracked when CI passes.
- Minor updates require normal review and validation.
- Major updates are not grouped by this policy and should be handled explicitly.

7.9 Validation Expectations

Before merging Dependabot PRs:

- CI pipeline must pass.
- Any relevant integration and performance checks should remain within agreed thresholds.
- Terraform changes should be reviewed for plan impact.

7.10 Known Constraints

- Dependabot pip support requires supported Python manifests (for example requirements.txt, pyproject.toml, setup.py, or setup.cfg).
- If the repository uses non-standard filenames (such as dependencies.txt), pip updates and related security updates may not be discovered automatically.

7.11 Exclusions

This policy does not:

- Fix application source-code vulnerabilities directly.
- Replace dedicated security testing and scanning processes.
- Guarantee immediate remediation if no patched dependency version is available.

7.12 Ownership

- Repository maintainers own configuration and policy updates.
- Service owners approve risk acceptance when updates cannot be applied immediately.

7.13 Revision History

- 2026-07-15: Added explicit security update grouping across configured ecosystems.

8 Patient Triage Api - Disaster Recovery Strategy

Version	1.0
Last Updated	May 2025
Owner	Platform Team (Infrastructure engineers across all teams)
Status	WIP
Service Level	BRONZE

8.1 Table of Contents

8.2 Purpose

This Disaster Recovery (DR) Strategy defines how the **Patient Triage Api platform** maintains business continuity in the event of data loss, service failure, ransomware, misconfiguration, or full AWS regional outage. The strategy incorporates **AWS Backup immutable vaults**, ensuring tamper-proof backups that meet NHS security requirements.

The DR strategy covers:

- **AWS services:** Lambda, DynamoDB, S3, CloudFront, AWS Secrets Manager
- **Recovery objectives: RPO ≤ 5 minutes**, **RTO ≤ 4 hours** for production
- **Automation:** Infrastructure-as-Code (Terraform) enables rapid rebuild
- **Data protection:** Automated backups, point-in-time recovery, and immutable backups via AWS Backup Vault Lock

8.3 Scope

In Scope:

- All AWS infrastructure deployed via Terraform
- DynamoDB,
- S3 buckets (artefacts, logs)
- AWS Secrets Manager and SSM Parameter Store

- KMS keys, CloudWatch logs, alarms
- GitHub Patient triage api repository - see Github repository restore process[18]

This includes all resources required to rebuild or restore the Patient triage api production environment.

Note: Red Lines - Backup Scenario - Scope Inclusions[19]

Out of Scope:

- Development/Test Environments: On-demand restoration
- AWS global services (IAM, Route 53 hosted zones) - AWS-managed resilience
- GitHub - GitHub's own DR/backup
- Third-Party SaaS: GitHub Actions, external APIs

Note: Red Lines – Backup Scenario - Exclusions[20]

8.4 Objectives

8.4.1 Data Protection Objectives

- **RPO ≤ 5 minutes** for production data
- **Zero data loss** for committed transactions
- Immutability enforced for all critical backups
- Backups protected against tampering or deletion (AWS Backup Vault Lock)

8.4.2 Service Continuity Objectives

- **RTO ≤ 4 hours** for restoring production services
- Rebuild production environment in primary or secondary region using Terraform

18. https://nhsd-confluence.digital.nhs.uk/spaces/CIS/pages/1113384241/Github+repository+restore+process
19. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Red-Lines---Backup-Scenario---Scope-Inclusions.aspx
20. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Red-Lines---Backup-Scenario---Scope-Exclusions.aspx?CT=1750849141263&OR=OWA-NT-Mail&CID=872f70dc-6025-51fd-2564-4c8a8997cd9b&xsdata=MDV8MDJ8YWpheS5tdWRnYWwxQG5ocy5uZXR8NDEzNGRiYmVkZDZkNDU3NTE2MDIwOGRkOTkwZWM0YTJ8MzdjMzU0YjI4NWIwNDdmNWIyMjIwN2I0OGQ3NzRlZTN8MHwwfDYzODgzNTAxMDg5NjY2MTE5MnxVbmtub3dufFRXRnBiR1pzYjNkOGV5SkZiWEIwZVUxaGNHa2lPblJ5ZFdVc0lsWWlPaUl3TGpBdU1EQXdNQ0lzSWxBaU9pSlhhVzR6TWlJc0lrRk9Jam9pVFdGcGJDSXNJbGRVSWpveWZRPT18MHx8fA%3d%3d&sdata=UnRlUVUwWTBGOXdqWTZRU0p4VFBJTHdZZ2I4NFNZVUtDRnNjaU9QMXQ5TT0%3d&clickParams=eyJYLUFwcE5hbWUiOiJNaWNyb3NvZnQgT3V0bG9vayBXZWIgQXBwIiwiWC1BcHBWZXJzaW9uIjoiMjAyNTA2MTMwMTYuMTUiLCJPUyI6Ik1hYyBPUyBYIHVuZGVmaW5lZCJ9&SafelinksUrl=https%3a%2f%2fnhs.sharepoint.com%2fsites%2fX26_EngineeringCOE%2fSitePages%2fRed-Lines---Backup-Scenario---Scope-Exclusions.aspx

8.4.3 Compliance and Operational Objectives

- Meet NHS data security standards during recovery
- Minimise manual intervention through automation (IaC)
- Optimise costs while maintaining DR readiness

8.5 Critical Components & Dependencies

DynamoDB	All services	PITR + AWS Backup
S3 Buckets	Artefacts, UI, logs	Versioning + immutable backups
CI/CD Pipelines	Source repo, secrets	Manual/delayed deployment if repo compromised
Secrets Store	All services using keys/ secrets	Versioned; backed up via AWS Backup; rotation required during compromise
Artefact Repositories	Build and deployment processes	Restore from backup or rebuild from source
KMS	IAM, account boundary	Encryption dependency for all services; multi-region recommended
DNS / Route 53	Registrar access	Required for failover and traffic routing

8.6 Recovery Objectives

8.6.1 Recovery Point Objective (RPO)

Recovery Point Objective is the maximum acceptable amount of time between the current moment and the most recent data recovery point.

DynamoDB	Last release + PITR	AWS Backup	1 version	✅
S3 *Artefacts Bucket* *Log Buckets*	Near-zero	Versioning + AWS Backup	7 days	✅
Secrets Manager	Near-real-time (versioning)	Versioning + AWS Backup	7–35 days	✅

8.6.2 Recovery Time Objective (RTO)

Recovery Time Objective is the maximum acceptable delay between service disruption and service restoration.

1.Incident Detection • Grafana alerts	15 min	T+15 min
2. Incident Assessment • Determine scope • Activate DR plan	30 min	T+45 min
3. Infrastructure Rebuild • Terraform policies deployment • Terraform account deployment • Terraform application deployment	90 min	T+2h 15min
4. Data Restoration • Restore database backup	60 min	T+3h 15min
5. Data Validation • Smoke tests • Data integrity check	30 min	T+3h 45min
6. DNS Failover • Route 53 update • CloudFront cache • User traffic resumed	15 min	T+4h (RTO met)

NOTE: *RTO is to be confirmed via recovery testing.*

8.7 Key Capabilities

Infrastructure Rebuild	✅	Terraform deploys all stacks from versioned state
Immutable Backups	✅	AWS Backup, DynamoDB, S3
DynamoDB Backup	✅	Point-in-Time Recovery + AWS Backup vault lock

Application Artefacts	⚠️ TO DO	S3 versioning + AWS Backup
Secrets Recovery	⚠️ TO DO	Versioning + AWS Backup + rotation
Terraform State Backup	⚠️ TO DO	Versioning applied; bucket protected
KMS Key Recovery	⚠️ Manual	Original keys cannot be restored if lost; new keys are provisioned via Terraform. Data restored from backups is re-encrypted.

8.8 KMS Recovery Strategy

AWS Key Management Service (KMS) underpins the security of **all encrypted data** within the patient triage api platform. Loss, compromise, or inaccessibility of KMS keys directly impacts data availability.

8.8.1 KMS Failure Scenarios

Accidental key deletion	Immediate data inaccessibility	Key deletion protection + IAM controls prevent accidental deletion
Key compromise	Confidentiality risk	Disable key, rotate data, re-encrypt resources
Full AWS account loss	Keys permanently lost	Create new AWS account, provision new KMS keys, restore data from backups, rotate secrets

8.8.2 KMS Recovery Limitations

- Data cannot be decrypted without original keys
- Key deletion is never a remediation step
- Multi-Region KMS keys reduce recovery complexity and time

8.8.3 DNS & Traffic Recovery Strategy

The patient triage api platform uses:

- **Amazon Route 53** for authoritative DNS
- **CloudFront** for edge distribution and TLS termination

- Origin endpoints hosted in regional AWS infrastructure

DNS is the final control point for directing user traffic during disaster recovery.

8.8.3.1 Private Beta Limitation (Controlled DNS Failover)

During the **Private Beta phase**, DNS failover will be applied **only to APIs exposed behind API Management (APIM)**.

This provides:

- Greater control over traffic routing during recovery
- Reduced operational risk while DR processes are validated
- Improved security and monitoring through APIM
- A smaller blast radius in the event of misconfiguration or recovery issues

8.8.3.2 DNS Failure Scenarios

Application outage	Users receive errors	DNS cutover to recovered environment
Regional outage	Origin unreachable	Secondary region rebuilt via Terraform, CloudFront updated, Route 53 records updated, low TTLs used
Account loss	DNS records unavailable	Recreate hosted zones, restore Terraform-defined records, update registrar NS delegation **manually**, recreate CloudFront and TLS certificates

8.9 Backup & Immutability Strategy

8.9.1 Centralised AWS Backup

The centralised immutable backup solution with AWS Backup to ensure that the data is securely backed up and protected against unauthorised access and ransomware. This guide[21] will help you leverage these backups for disaster recovery.

AWS Backup is configured as the **single authoritative backup system** for:

- DynamoDB tables
- S3 buckets (optional: artefacts, config backups)

21. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/TechRadar-CloudBackups.aspx

8.9.1.1 AWS Backup Vault Lock

To meet ransomware and tamper-resistant requirements, **Backup Vault Lock** is enabled:

- Prevents deletion or modification of backups
- Enforces retention policies even by admin roles
- Ensures legal/audit compliance

8.10 Disaster Scenarios

Failure of one AZ	Service continues on remaining AZ; multi-AZ redundancy ensures availability	None. Handled transparently by AWS managed services.
Failure of CI/CD (GitHub)	Delayed or manual deployment depending on the type of change. Teams may either pause or manually redeploy pipelines.	Reduced deployment velocity; no impact to running services.
Failure of Source Code Repository (GitHub)	GitHub source code repository is backed up every 24h. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/GitHub-Repository-Backup.aspx	Loss of recent commits (up to 24h) possible; restore coordinated through platform tooling.
Failure of the whole AWS Region	Manual failover; rebuild environment in secondary region using Terraform + restore backups **Note:** A full AWS Region failure is considered out of current scope for this service as handling a complete regional disaster recovery is subject to an organisation-wide decision.	A low-probability, high-impact event. Requires: • Authority to operate in the secondary region • Coordination with dependent and upstream services • Re-establishment of IAM trust and access • Data integrity and consistency validation
Failure of the whole AWS Account	Create AWS account; restore infrastructure, data, secrets, and artefacts	Governance approval required. All integrations, trust relationships, and credentials must be re-established.
Compromised AWS account	Treat the account as unrecoverable. Create AWS account, restore infrastructure and data from immutable backups taken **prior to compromise**, rotate all secrets and credentials, and validate data integrity before resuming service.	Full security incident response required. Data integrity validation is mandatory before resuming service.

Failure of the Secrets Store	Restore non-compromised secrets; rotate compromised secrets; validate integrations	Temporary service disruption may occur during rotation and validation.
Failure of a System Dependency	Validate graceful degradation and recovery	Service functionality may be reduced depending on the dependency affected.
Failure of a Artefact Repository	Restore build artefacts from backup or rebuild from source code	Redeployment may be delayed while artefacts are rebuilt or restored.

8.11 Disaster Recovery Test Strategy

- Teams perform **full backup-restore tests** to clean environments
- Validate recovery of full application
- Define **test schedule, success criteria, and review**
- Document results for audits

AWS Fault injection to mimic the above failures.

8.12 Roles & Responsibilities During Incidents

Platform Lead	Activate the DR plan; approve failover decisions; coordinate incident response	• Executive authority to invoke DR • Access to incident runbooks • Pre-approved secondary region/ account strategy • Clear escalation paths and on-call contacts
Infrastructure Engineers	Deploy Infrastructure-as-Code (Terraform); restore backups; validate infrastructure integrity; rebuild CI/CD pipelines and restore artefacts	• Access to AWS accounts • Access to GitHub repository • Access to backup vaults

Infrastructure Engineers	Team Rotate compromised secrets and certificates; ensure compliance; validate integration of restored secrets	• Access to Secrets Manager, Parameter Store • Predefined rotation procedures • Inventory of dependent services and integrations
Test Engineers	Validate restored services; run smoke tests and data integrity checks; verify functionality of dependencies	• Test environments available and representative • Automated smoke and regression test suites • Known-good expected results and validation criteria • Access to monitoring and logging systems
Service Owners	Confirm service functionality; validate upstream/ downstream dependencies; ensure SLA targets are met	• Service-level objectives (SLOs/SLA definitions) • Visibility of service health metrics • Knowledge of business-critical workflows • Confirmation from platform and test teams
Product Owner	Provide updates to stakeholders; manage internal and external communications; document incident reports	• Agreed communication channels and templates • Authoritative status updates from technical leads • Stakeholder contact lists
Digital Operations & Service Management (DOSM) Team[22]	Responsible for the reliability and performance of all live services; coordinate incident response and recovery activities in line with NHS operational processes.	• Established incident management processes • Clear hand-off and coordination model with platform and service teams • Access to monitoring, alerting, and service status information

8.13 Patient Triage Api Annual Disaster Recovery Tests

Disaster recovery testing will be performed annually.

8.13.1 DR Testing Plans and Results

2026			
2027			

8.13.2 Patient Triage Api Disaster Recovery Plan 2026

Purpose: Provide engineers with a clear, step-by-step guide to test the Future DoS Disaster Recovery (DR) strategy, validate recovery objectives, and document results.

8.13.2.1 Pre-Test Preparation

Complete all items before beginning any test scenario. Do not proceed until all are confirmed.

1. Confirm **all backups are complete** and immutable (AWS Backup Vault Lock)
2. Verify **Terraform state is current** and versioned
3. Confirm **access** to AWS accounts, Secrets Manager, SSM, GitHub, and backup vaults
4. Notify stakeholders and schedule **maintenance window**
5. Prepare **test environment** (secondary region or isolated account if required)

8.13.2.2 Test Scenarios & Step-by-Step Execution

8.13.2.2.1 Scenario A: Single-Service Failure (DynamoDB)

1. Simulate service failure (e.g., stop DynamoDB table)
2. Restore from **PITR + AWS Backup**
    - DynamoDB: Restore to point-in-time
3. Validate **data integrity**:
    - Record counts
    - Sample data checksums
4. Execute **smoke tests** for all affected services
5. Document RTO/RPO achieved

22. https://nhsd-confluence.digital.nhs.uk/spaces/SM/pages/333293837/Teams+and+Team+Members

8.13.2.2.2 Scenario B: Full Data Store Recovery

1. Take snapshot of production as baseline
2. Restore all critical data stores into test environment
3. Re-run **DMS replication** if applicable
4. Validate:
    - Data consistency
    - Dependent Lambda
5. Log time taken for recovery

8.13.2.2.3 Scenario C: Infrastructure Rebuild

1. In isolated AWS account / secondary region, run Terraform to deploy:
    - Networking
    - Compute / Lambda / ECS services
    - Database
    - S3 buckets and artefacts
2. Apply **account-wide IAM policies**
3. Restore **Terraform state**
4. Execute smoke tests for infrastructure health

8.13.2.2.4 Scenario D: Secrets / KMS Recovery

1. Simulate secrets loss / rotation requirement
2. Restore versioned secrets from AWS Backup
3. Validate:
    - All services can retrieve secrets
    - KMS keys decrypt data correctly
4. Rotate keys if required

8.13.2.2.5 Scenario E: Full End-to-End Recovery

1. Combine scenarios B + C + D
2. Rebuild infrastructure from Terraform
3. Restore all critical data
4. Restore secrets and KMS keys
5. Run smoke tests across all services
6. Document RPO/RTO achieved

8.13.2.3 Test Validation & Success Criteria

- RPO ≤ 5 minutes

- RTO ≤ 4 hours (full environment)
- Smoke tests pass on all services
- Data integrity validated across DynamoDB
- Secrets retrieved correctly
- CI/CD pipelines functional and artefacts deployable

8.13.2.4 Post-Test Activities

1. Compile **DR Test Report**:
    - Time to recovery
    - Issues / gaps identified
    - Lessons learned
2. Update **DR runbooks** and Terraform scripts based on findings
3. Share report with:
    - Platform Lead
    - Service Owners
    - NHSE Cluster Team (if required)

8.13.3 Patient Triage Api Disaster Recovery Report 2026

Date	
Environment	
Test Owner	

8.13.3.1 Objectives

Recovery Point Objective (RPO)	≤ 5 min		☐ / ☐
Recovery Time Objective (RTO)	≤ 4 h		☐ / ☐
Data Integrity	All critical data restored accurately		☐ / ☐
Secrets Recovery	All secrets functional		☐ / ☐

8.13.3.2 Test Scenario & Results

TEST SCENARIOS COVERED IN REF ENVIRONMENT	RPO (≤ 5 min)	RTO (≤ 4 h)	Data Integrity	Secrets Recovery	Comments
Scenario A: Single-Service Failure (DynamoDB) - Remote AWS Backup Restoration					
Scenario B: Full Data Store Recovery (S3 bucket housing truststore certificates) - Remote AWS Backup Restoration					

Scenario C: Secrets (`saet-ref-account-wide`) Recovery
 	
`apikey`
`proxygen_base_url`
`proxygen_client_id`
`proxygen_client_secret`
`proxygen_key_id`
`proxygen_key_pem`
`apim_environment`					

Scenario D: Infrastructure Rebuild					
Scenario D: KMS Recovery					
Scenario E: Full End-to-End Recovery					

8.13.3.3 Lessons Learned / Recommendations

...

8.13.3.4 Approval & Sign-Off

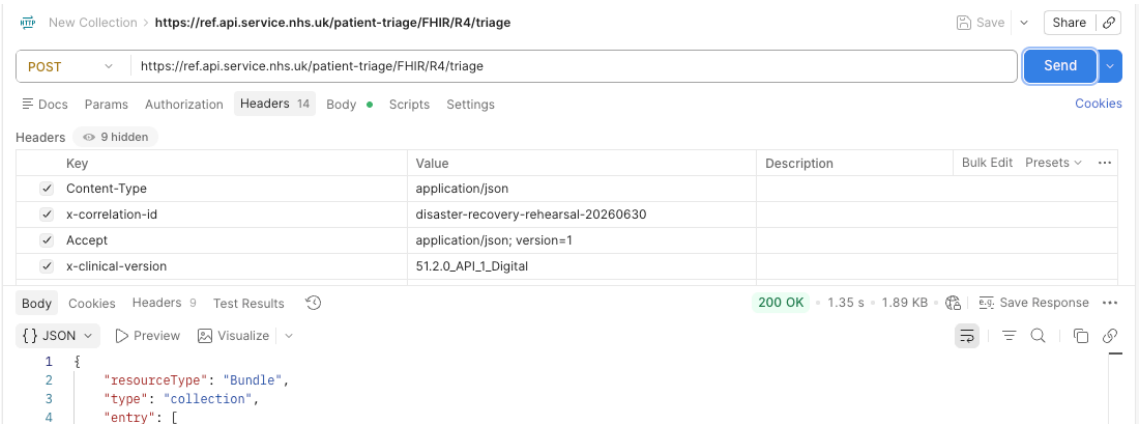
Platform Lead		

Test Engineer(s)		
Service Owner(s)		

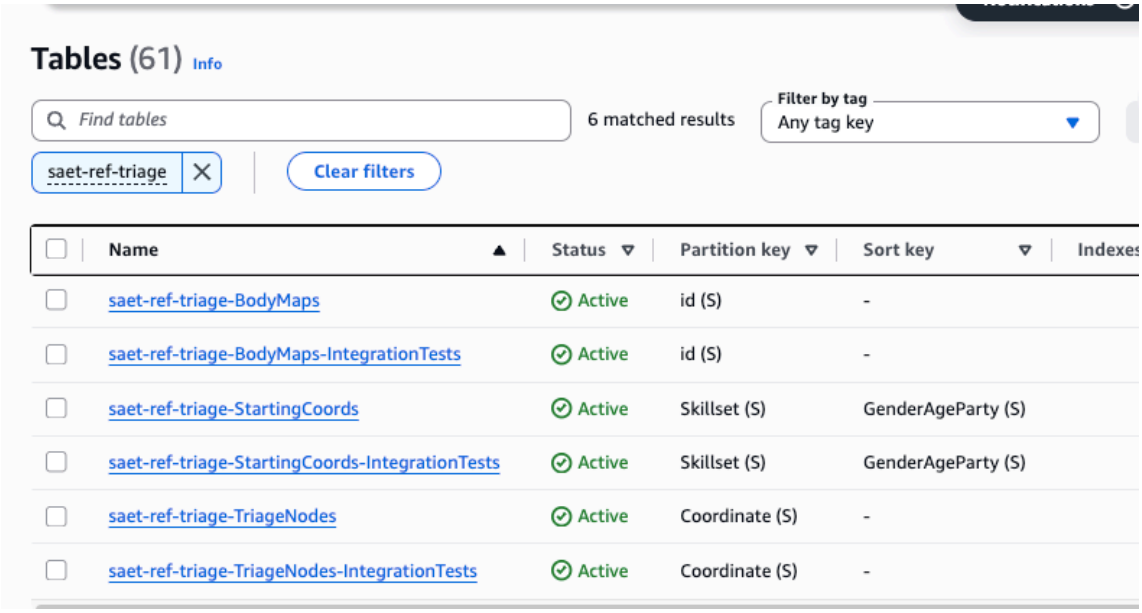
8.13.3.5 Appendices

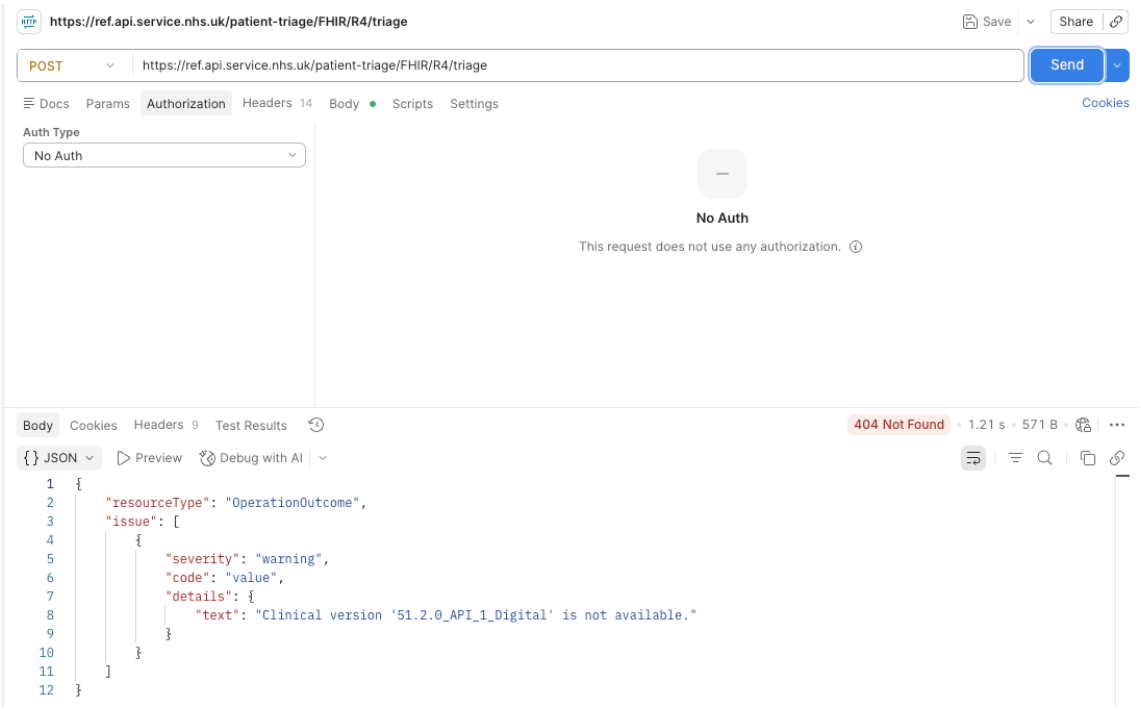
8.13.3.5.1 Scenario A

1. Pre DynamoDB tables deletion

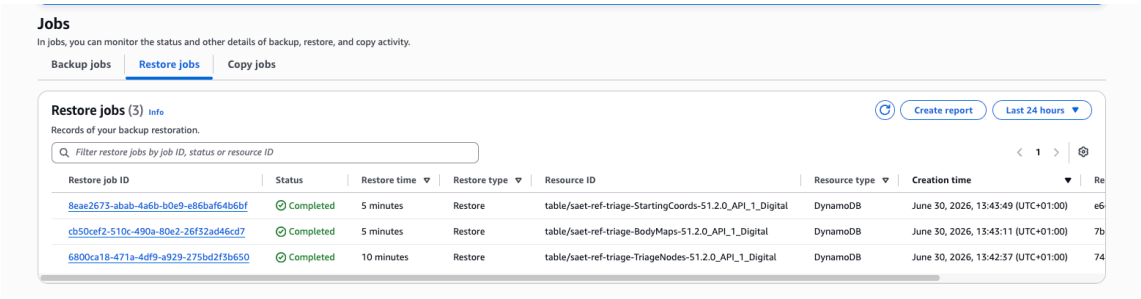


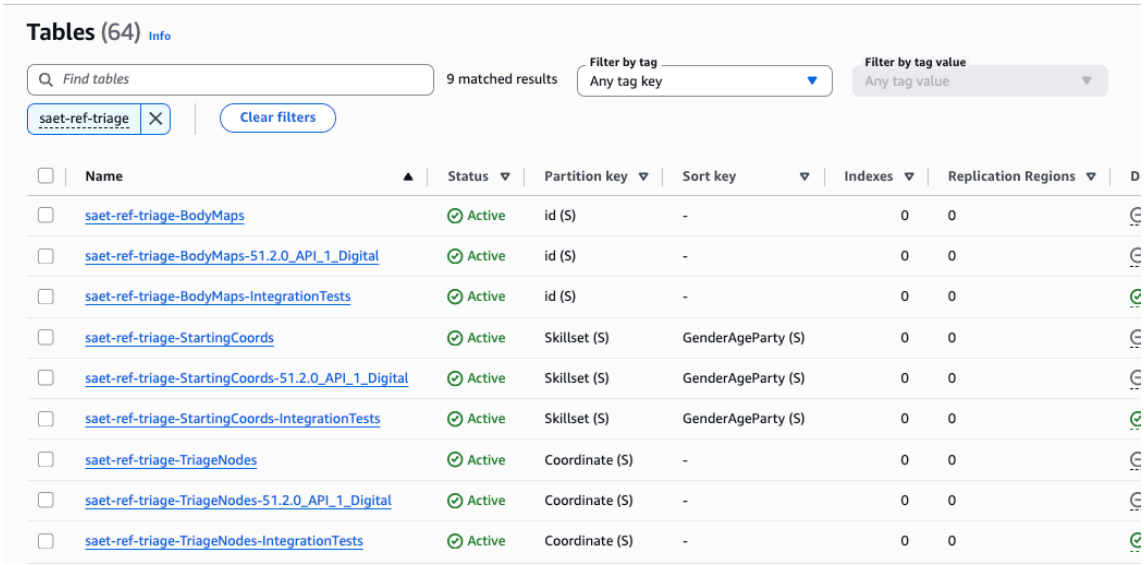
2. Post DynamoDB tables deletion



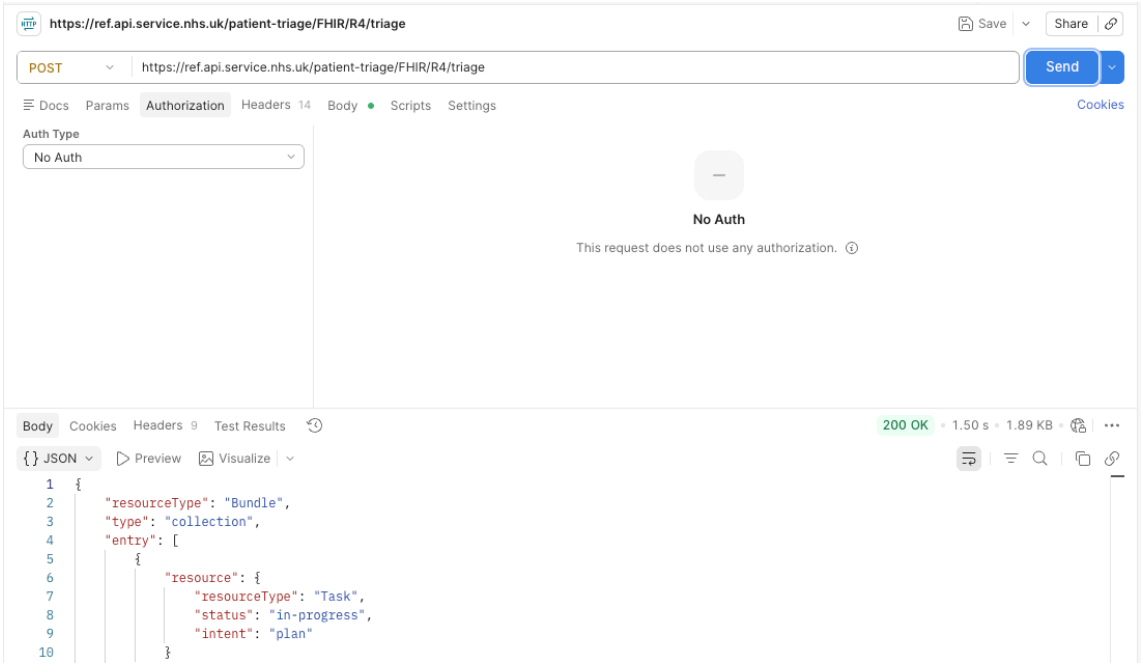


3. AWS Backup Restoration of DynamoDB tables



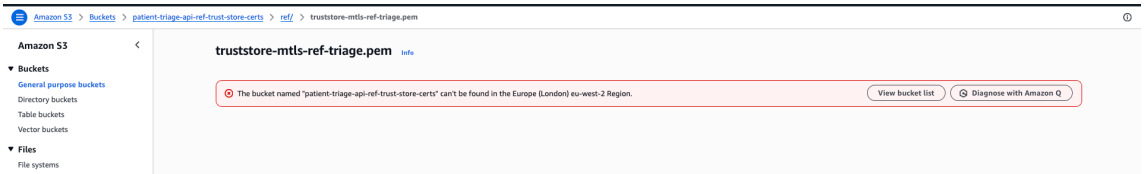


4. Post Restoration Check

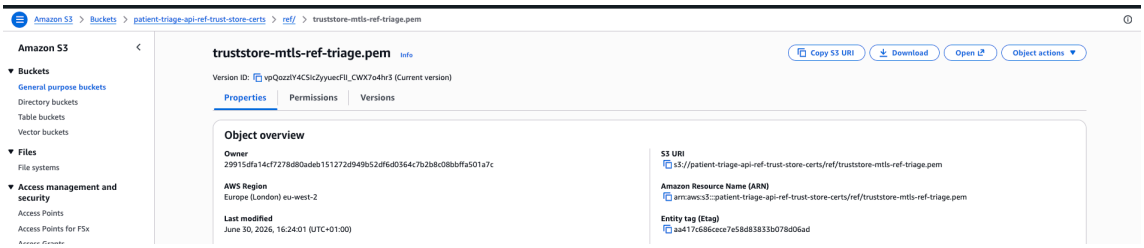


8.13.3.5.2 Scenario B

1. S3 bucket (MTLS Truststore Certificates) deletion

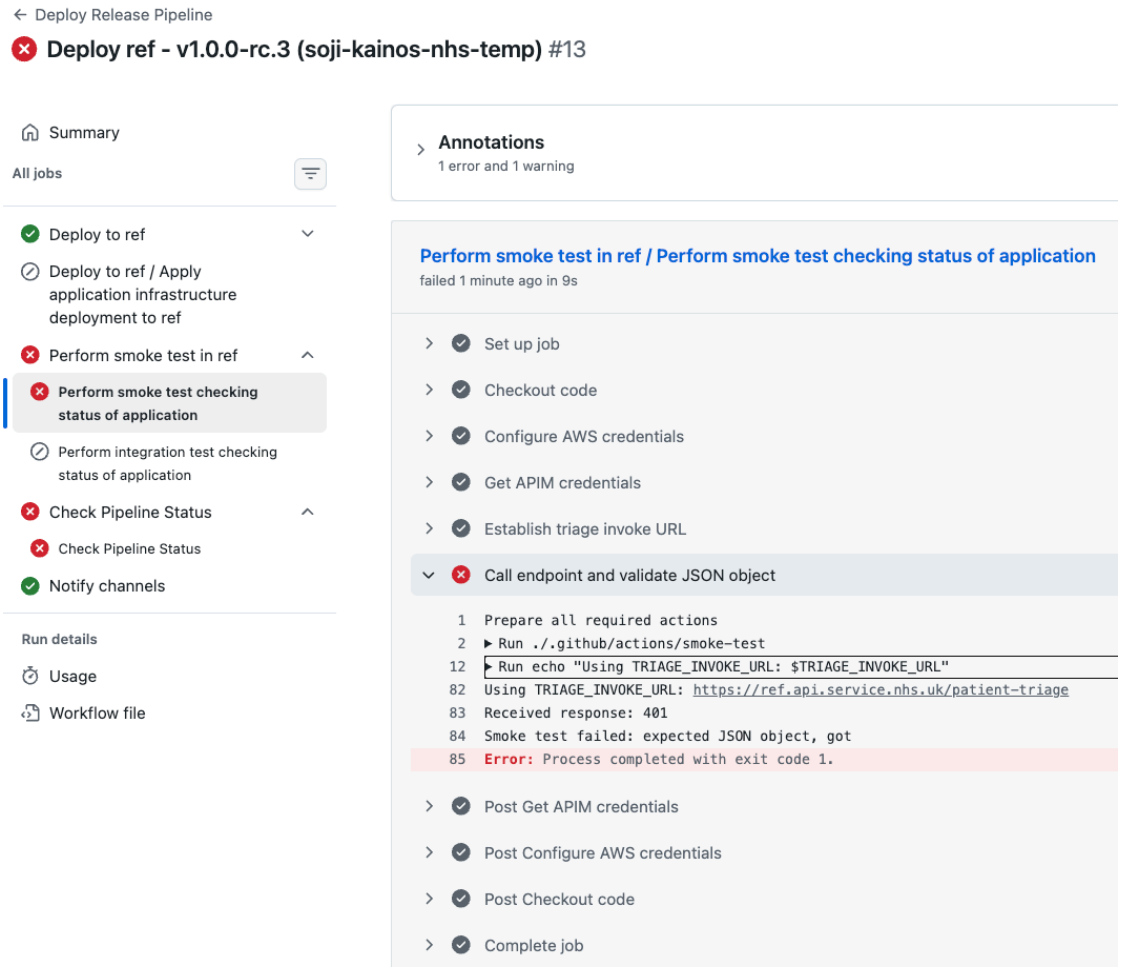


2. AWS Backup Restoration of S3 bucket

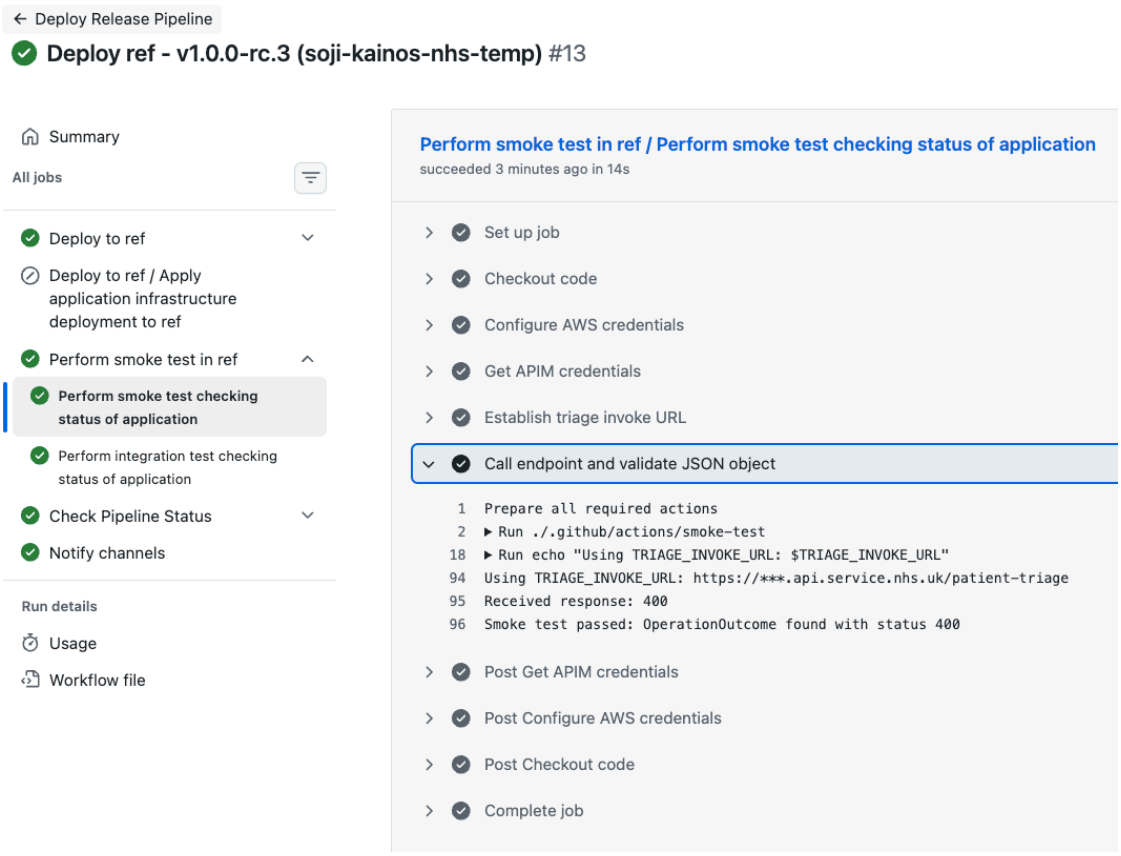


8.13.3.5.3 Scenario C

1. Secret key/values for `saet-ref-account-wide` deleted.
   This results in Deploy Release Pipeline failure during smoke/integration test stage.

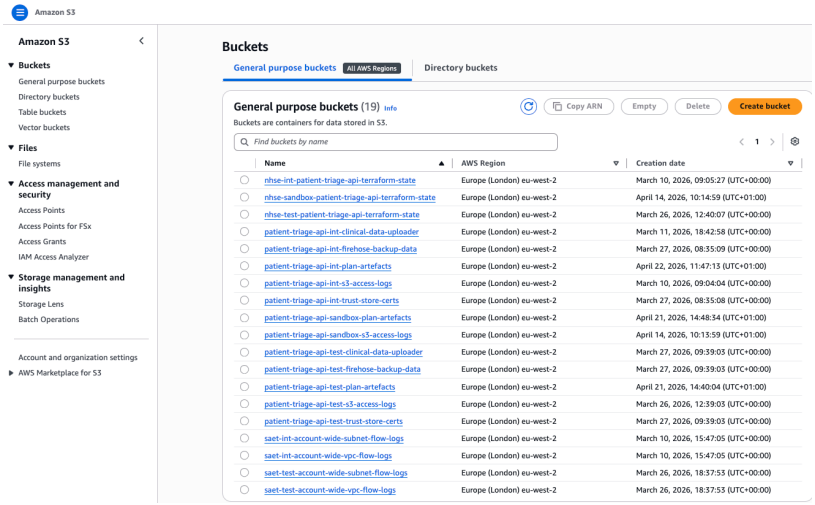


2. Secret key/values for saet-ref-account-wide restored from backup account.
   This results in Deploy Release Pipeline passing

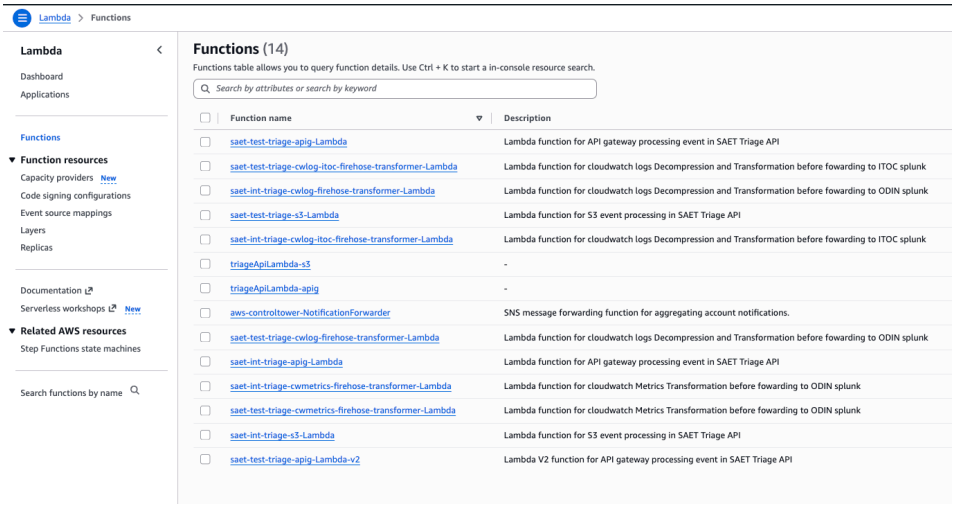


8.13.3.5.4 Scenario D

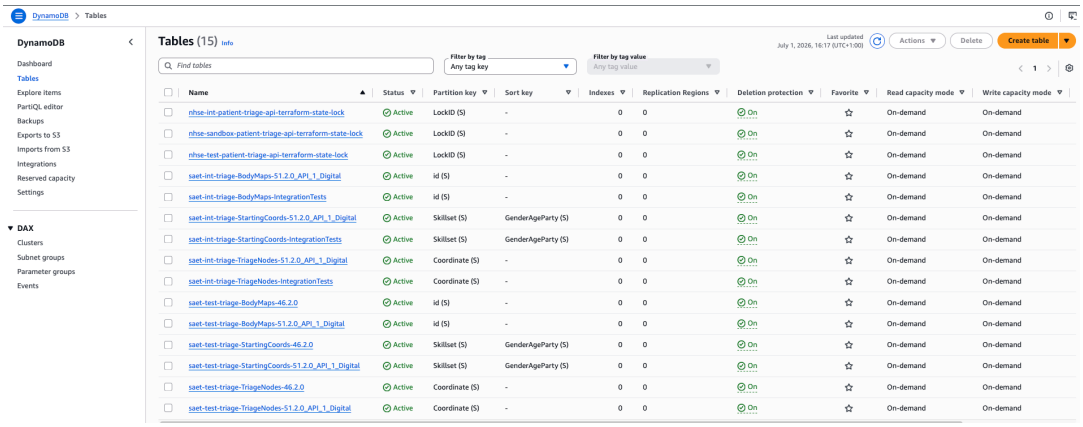
1. All AWS Infrastructure and Triage Application in REF env currently decommissioned
    a. S3



b. Lambda



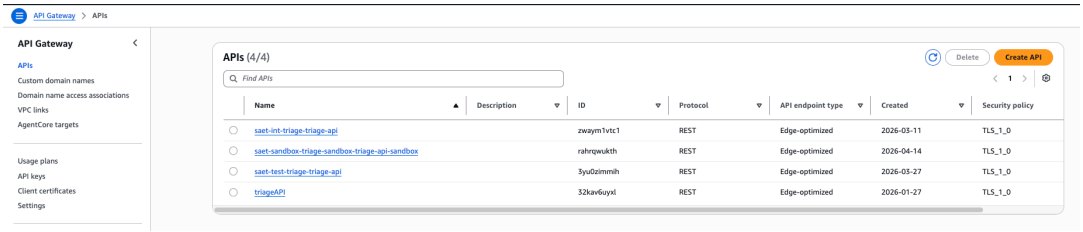
c. DynamoDB



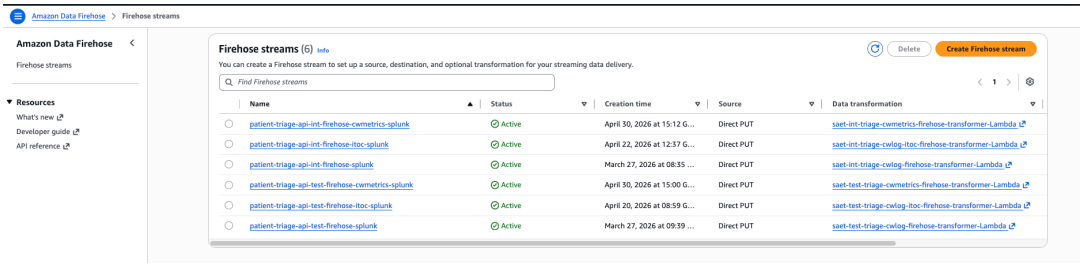
d. API Gateway
e. Amazon Data Firehose
f. AWS Backup
g. KMS
h. VPC
i. Secrets
j. Route53 / Hosted Zone
k. CloudWatch Log Groups
l. CloudWatch Metrics
m. Github Runner Roles

DynamoDB

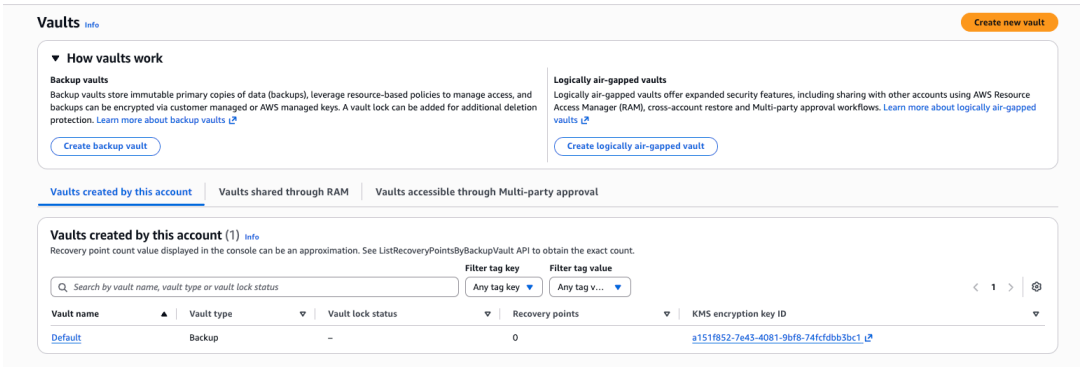
API Gateway



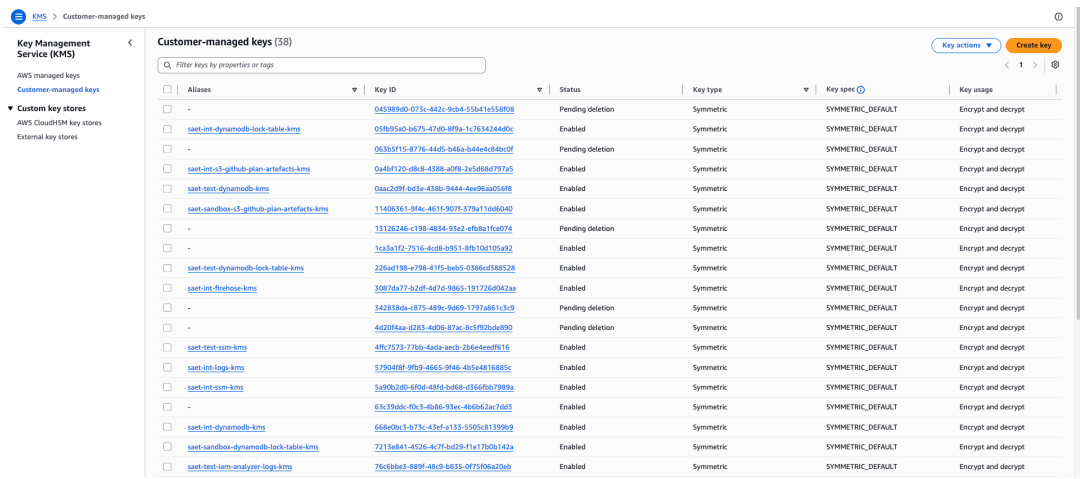
Amazon Data Firehose

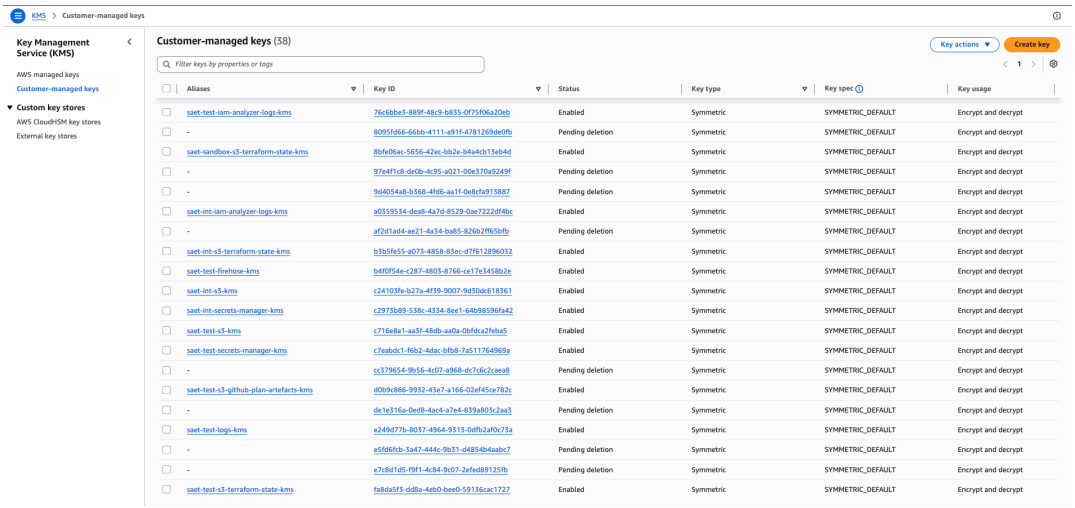


AWS Backup Source

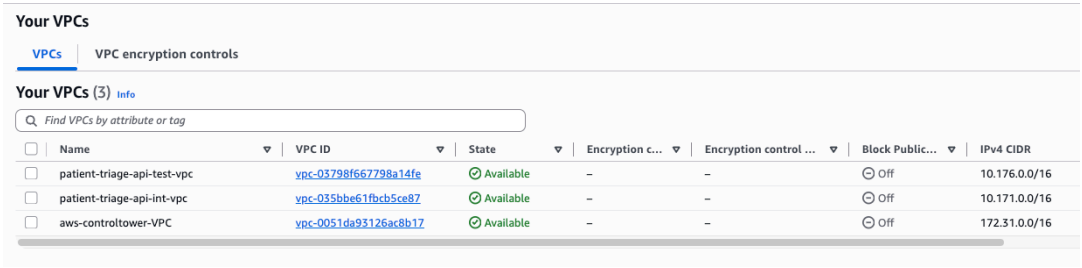


KMS

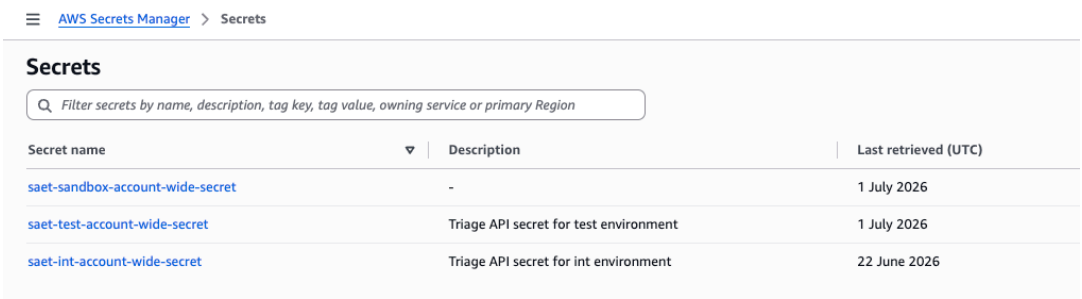




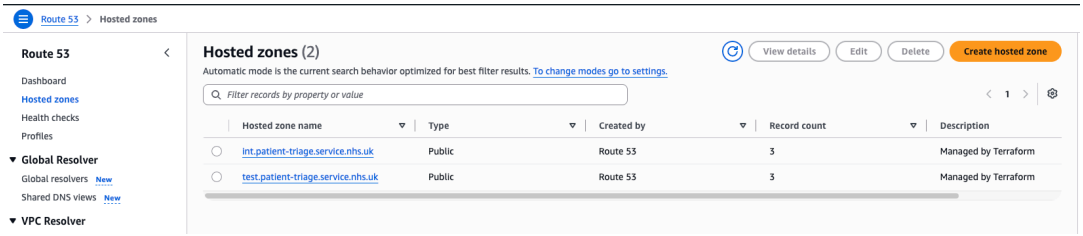
VPC



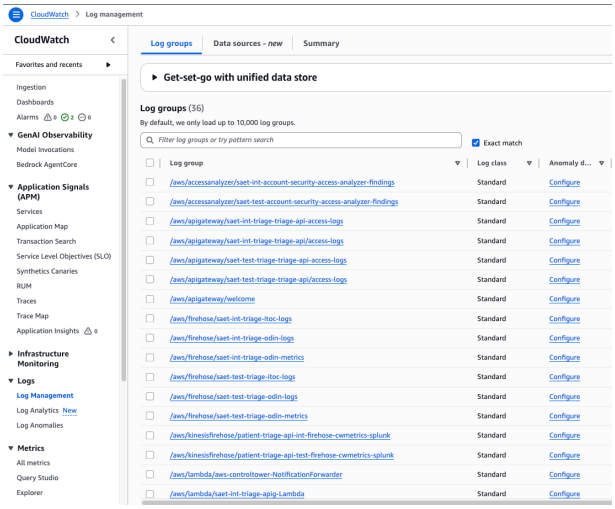
Secrets

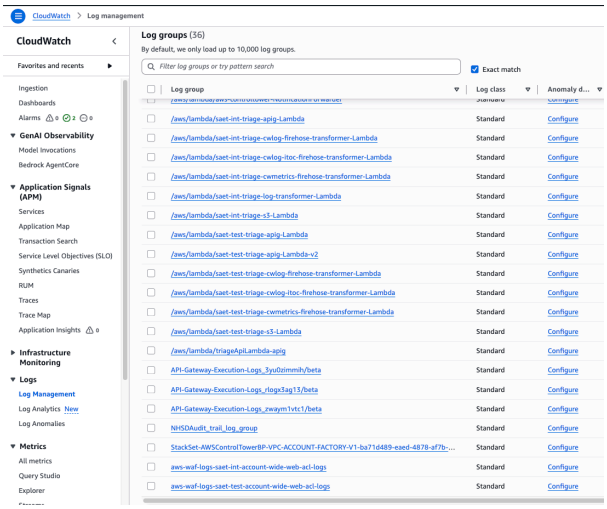


Route 53/ Hosted Zones

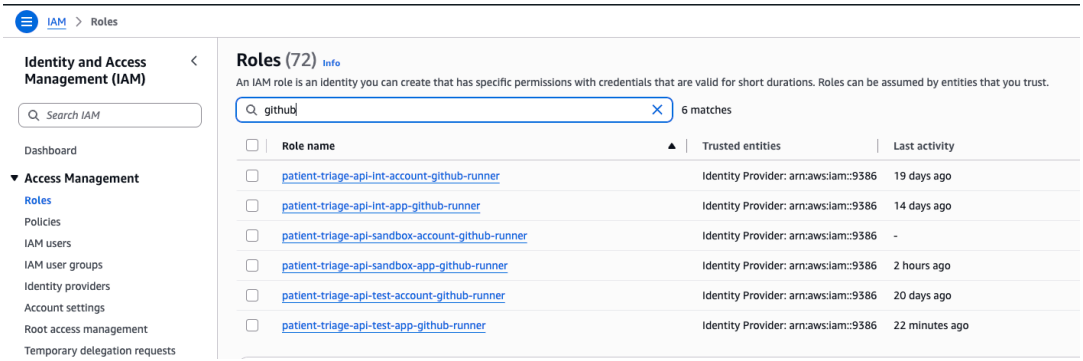


CloudWatch Log Groups

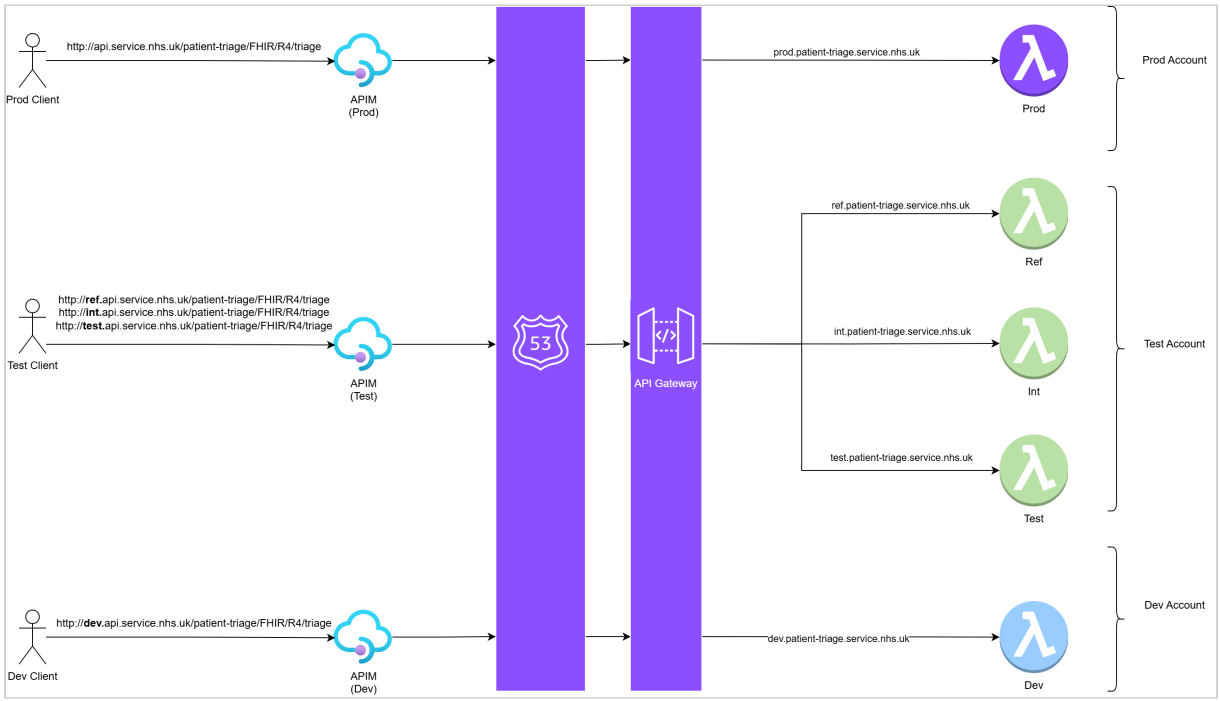




Github Runner Role


IAM > Roles
Identity and Access Management (IAM)
Search IAM
Dashboard
Access Management
Roles
Policies
IAM users
IAM user groups
Identity providers
Account settings
Root access management
Temporary delegation requests
Roles (72) Info
An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.
github
6 matches
Role name
Trusted entities
Last activity
patient-triage-api-int-account-github-runner
Identity Provider: arn:aws:iam::9386
19 days ago
patient-triage-api-int-app-github-runner
Identity Provider: arn:aws:iam::9386
14 days ago
patient-triage-api-sandbox-account-github-runner
Identity Provider: arn:aws:iam::9386
-
patient-triage-api-sandbox-app-github-runner
Identity Provider: arn:aws:iam::9386
2 hours ago
patient-triage-api-test-account-github-runner
Identity Provider: arn:aws:iam::9386
20 days ago
patient-triage-api-test-app-github-runner
Identity Provider: arn:aws:iam::9386
22 minutes ago

9 Patient Triage Api - DNS



10 Patient Triage Api - Encryption Strategy

10.1 Introduction

This document details the **encryption strategy** for applications hosted on AWS, designed to be implemented across multiple AWS accounts. By leveraging **native AWS encryption services**, we aim to achieve strong data protection, regulatory compliance, and operational efficiency while optimising costs.

10.2 Encryption at Rest

Encryption at rest protects data that is stored on disk, database, or backup media. In AWS, this is typically handled by **AWS Key Management Service (KMS)**, which manages encryption keys and integrates with services like S3, EBS, and RDS to secure stored data.

With **AWS KMS** we have to answer two questions:

1. **Centralised Or Decentralised model for managing KMS Keys**
2. **Which type of KMS key to use - Customer Managed, AWS Managed or AWS Owned KMS Keys**

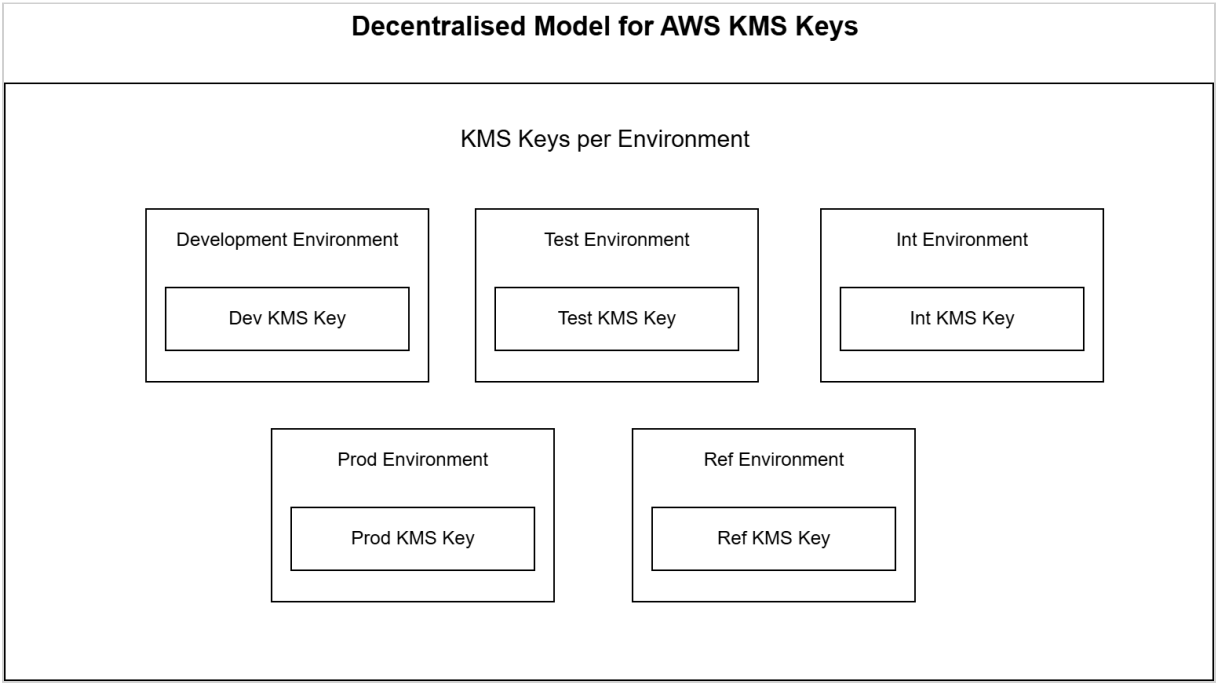
10.2.1 Centralised Or Decentralised model for managing KMS Keys

10.2.1.1 Decentralised Model

In this approach, we create and manage the keys in the AWS accounts that would use them.

A visual representation of how this would be set up in our case is shown below.

Development Environment
Test Environment
Ref/Preprod Environment
Prod Environment
Int Environment
Dev KMS Keys
Test KMS Keys
Int KMS Keys
Ref KMS Keys
Prod KMS Keys
Decentralised Model for AWS KMS Keys
KMS Keys Per Environment



10.2.1.1.1 Pros and Cons

Security through isolation	Control over who creates the keys a little more involved
No Single point of failure	Management of keys has to be done from individual accounts
Simple same account access and management	
Misconfigurations will impact only the specific environment	

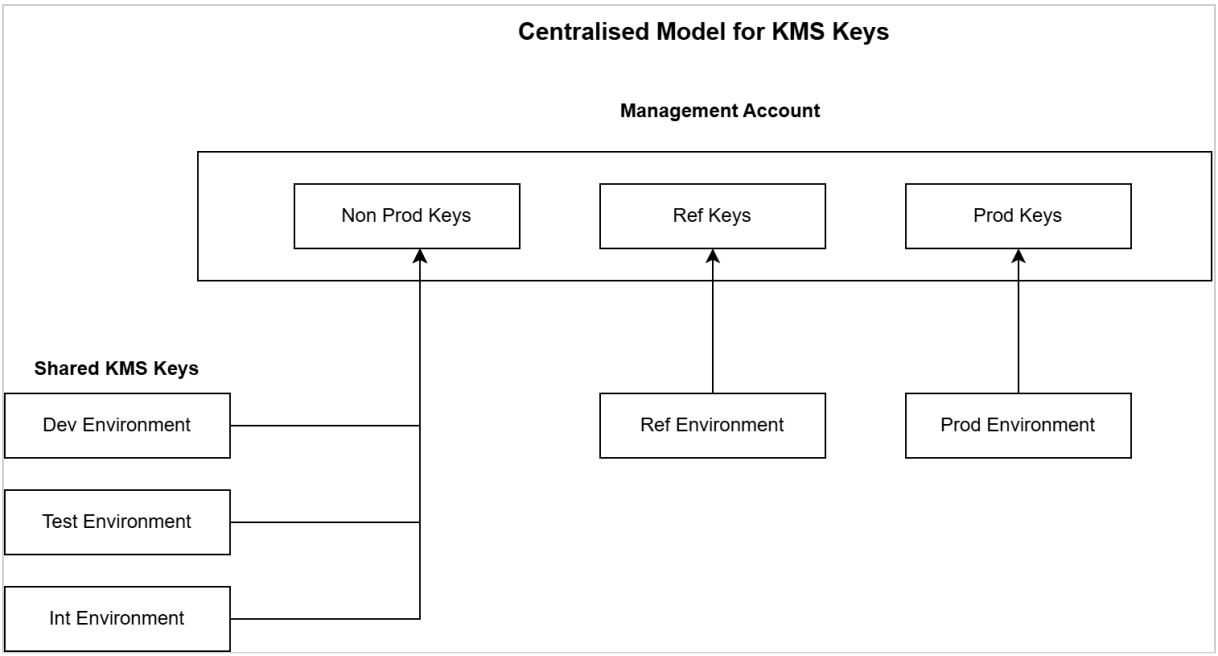
10.2.1.2 Centralised Model

In this approach, we create and manage the keys a single AWS accounts (patient triage api management account) and use cross-account access from our respective environments.

A visual representation of how this would be set up in our case is shown below.

Non-prod Keys
Ref/pre-prod Keys
Prod Keys

Dev environment
Test environment
Int environment
Management Account
Ref environment
Prod environment
Centralised Model for AWS KMS Keys
Shared KMS Keys



10.2.1.2.1 Pros and Cons

Robust control over who creates and manages the keys	Misconfigurations might impact multiple environments
Management of keys from a single account	Complex cross-account access and management
Marginally Cheaper	Security through isolation is not applicable
	Single point of failure

10.2.1.3 Recommendation

Considering *security through isolation, reduced impact of misconfigurations, simple set up and access, a not a huge addition to cost of additional keys* and recommendation from AWS (AWS document in reference section), we would go with a **decentralised** model approach.

10.2.2 Key Types: Customer Managed, AWS Managed or AWS Owned KMS Keys

AWS KMS provides option on which type of key one can create and use with the service. They are

1. **Customer Managed Keys:** The KMS keys that you create and manage for use in your own cryptographic applications are known as *customer managed keys*
2. **AWS Managed Keys:** If you want an AWS service to encrypt your data but don't want the overhead or costs of managing keys, you can use an *AWS managed key*
3. **AWS Owned Keys:** AWS Managed Keys are legacy and newer services use AWS Owned Keys

The image below shows the difference between these key types for certain considerations.

Consideration	Customer managed keys	AWS managed keys	AWS owned keys
Key policy	Exclusively controlled by the customer	Controlled by service; viewable by customer	Exclusively controlled and only viewable by the AWS service that encrypts your data
Logging	AWS CloudTrail customer trail or event data store	CloudTrail customer trail or event data store	Not viewable by the customer
Life cycle management	Customer manages rotation, deletion, and AWS Region	AWS service manages rotation (annual), deletion, and Region	AWS service manages rotation (annual), deletion, and Region
Pricing	Monthly fee for key existence (pro-rated hourly); the caller is charged for API usage	No charge for key existence; the caller is charged for API usage	No charges to customer

10.2.3 Service-Level Recommendations

Listed below are the AWS service, functionality or application within the Patient Triage Api that use the AWS service, the KMS key type currently being used, available options around Key type and recommendation of which key type we would want to use.

AWS Service	Patient Triage Api functionality/ application/ others	Current KMS Key Type	Available options for KMS Key Types	KMS Key Recommendations	Key Rotation options	Key Rotation Recommendation	Comments
DynamoDB	Tables	AWS Managed Keys	AWS Owned Keys Customer Managed Keys	AWS Owned Keys (eg. Terraform lock table) Customer Managed Keys for Patient Triage Api data	**AWS Owned Keys** Not Applicable **AWS Managed Keys** Rotates every 3 years **Customer Managed Keys** • **Automatic Rotation** • Every 365 days	Customer Managed Keys with Automatic Rotation	

AWS Service	Patient Triage Api functionality/ application/ others	Current KMS Key Type	Available options for KMS Key Types	KMS Key Recommendations	Key Rotation options	Key Rotation Recommendation	Comments
					• **Manual Rotation** • Number of days set within the terraform code		

AWS Service	Patient Triage Api functionality/ application/ others	Current KMS Key Type	Available options for KMS Key Types	KMS Key Recommendations	Key Rotation options	Key Rotation Recommendation	Comments
Lambda	Environment Variables	AWS Managed Keys	Customer Managed Keys	AWS Managed Keys	**AWS Managed Keys** Rotates every 3 years **Customer Managed Keys** • **Automatic Rotation** • Every 365 days	NA	

AWS Service	Patient Triage Api functionality/ application/ others	Current KMS Key Type	Available options for KMS Key Types	KMS Key Recommendations	Key Rotation options	Key Rotation Recommendation	Comments
					• **Manual Rotation** • Number of days set within the terraform code		

AWS Service	Patient Triage Api functionality/ application/ others	Current KMS Key Type	Available options for KMS Key Types	KMS Key Recommendations	Key Rotation options	Key Rotation Recommendation	Comments
Cloud Watch	DMS CloudWatch	AWS Owned Keys	Customer Managed Keys	AWS Owned Keys	**AWS Owned Keys** Not Applicable **Customer Managed Keys** • **Automatic Rotation** • Every 365 days	NA	
	VPN						
	API Gateway						
	ReadOnly Viewer WAF						

AWS Service	Patient Triage Api functionality/ application/ others	Current KMS Key Type	Available options for KMS Key Types	KMS Key Recommendations	Key Rotation options	Key Rotation Recommendation	Comments
	CloudTrail Logs				• **Manual Rotation** • Number of days set within the terraform code		

AWS Service	Patient Triage Api functionality/ application/ others	Current KMS Key Type	Available options for KMS Key Types	KMS Key Recommendations	Key Rotation options	Key Rotation Recommendation	Comments
Secrets Manager	All Secrets	AWS Managed Keys	Customer Managed Keys	Customer Managed Keys	**AWS Managed Keys** Rotates every 3 years **Customer Managed Keys** • **Automatic Rotation** • Every 365 days	Customer Managed Keys with Automatic Rotation	

AWS Service	Patient Triage Api functionality/ application/ others	Current KMS Key Type	Available options for KMS Key Types	KMS Key Recommendations	Key Rotation options	Key Rotation Recommendation	Comments
					• **Manual Rotation** • Number of days set within the terraform code		

AWS Service	Patient Triage Api functionality/ application/ others	Current KMS Key Type	Available options for KMS Key Types	KMS Key Recommendations	Key Rotation options	Key Rotation Recommendation	Comments
SSM Parameter Store	API's client secrets	AWS Managed Keys	Customer Managed Keys	Customer Managed Keys	**AWS Managed Keys** Rotates every 3 years **Customer Managed Keys** • **Automatic Rotation** • Every 365 days	Customer Managed Keys with Automatic Rotation	

AWS Service	Patient Triage Api functionality/ application/ others	Current KMS Key Type	Available options for KMS Key Types	KMS Key Recommendations	Key Rotation options	Key Rotation Recommendation	Comments
	Account Ids			AWS Managed Keys	• **Manual Rotation** • Number of days set within the terraform code	NA	

AWS Service	Patient Triage Api functionality/ application/ others	Current KMS Key Type	Available options for KMS Key Types	KMS Key Recommendations	Key Rotation options	Key Rotation Recommendation	Comments
S3	Trust Store	AWS Owned Keys	AWS Owned Keys Customer Managed Keys	Customer Managed Keys	**AWS Owned Keys** Not Applicable **Customer Managed Keys** • **Automatic Rotation** • Every 365 days	Customer Managed Keys with Automatic Rotation	
	VPC Flow Logs			AWS Owned Keys		NA	
	Subnet Flow logs			AWS Owned Keys		NA	

AWS Service	Patient Triage Api functionality/ application/ others	Current KMS Key Type	Available options for KMS Key Types	KMS Key Recommendations	Key Rotation options	Key Rotation Recommendation	Comments
	ReadOnly viewer frontend			AWS Owned Keys	• **Manual Rotation** • Number of days set within the terraform code	NA	

AWS Service	Patient Triage Api functionality/ application/ others	Current KMS Key Type	Available options for KMS Key Types	KMS Key Recommendations	Key Rotation options	Key Rotation Recommendation	Comments
AWS Backup		NA	AWSManaged Keys Customer Managed Keys	Customer Managed Keys	**AWS Managed Keys** Rotates every 3 years **Customer Managed Keys** • **Automatic Rotation** • Every 365 days	Customer Managed Keys with Automatic Rotation	

AWS Service	Patient Triage Api functionality/ application/ others	Current KMS Key Type	Available options for KMS Key Types	KMS Key Recommendations	Key Rotation options	Key Rotation Recommendation	Comments
					• **Manual Rotation** • Number of days set within the terraform code		

The recommendations are based on what we know currently, however, as a project we should move to Customer Managed Keys if we consider our functionality to be critical and need auditing, rotation control or fine grain access control.

10.2.3.1 Key Rotation and Lifecycle

This section aims to point one to relevant AWS documentation which provides clarity on how KMS manages rotation and lifecycle.

KMS Key Hierarchy[23] talks about the HSM Backing Key, Derived Encryption Keys, Customer Data Keys and how KMS manages all of these.

AWS Key Management Service[24] talks about KMS with images that explain the concept.

10.2.4 Service-Level Recommendations CMK Key Aliases Adopted

AWS Service	Resource Name	Environment	Customer Managed Key (CMK) Key Aliases Adopted	Amazon S3 managed Keys (SSE-S3)
S3	nhse-<env>-patient-triage-api-terraform-state		saet-<env>-s3-terraform-state-kms	
	patient-triage-api-<env>-clinical-data-uploader		saet-<env>-s3-kms	
	patient-triage-api-<env>-firehose-backup-data		saet-<env>-s3-kms	
	patient-triage-api-<env>-plan-artefacts		saet-<env>-s3-github-plan-artefacts-kms	
	patient-triage-api-<env>-s3-access-logs			Amazon S3 managed keys (SSE-S3)
	patient-triage-api-<env>-trust-store-certs		saet-<env>-s3-kms	
	saet-<env>-account-wide-subnet-flow-logs			Amazon S3 managed keys (SSE-S3)
	saet-<env>-account-wide-vpc-flow-logs			Amazon S3 managed keys (SSE-S3)

23. https://docs.aws.amazon.com/kms/latest/cryptographic-details/key-hierarchy.html
24. https://docs.aws.amazon.com/kms/latest/developerguide/overview.html

AWS Service	Resource Name	Environment	Customer Managed Key (CMK) Key Aliases Adopted	Amazon S3 managed Keys (SSE-S3)
	saet-<env>-backup-source-backup-reports			Amazon S3 managed keys (SSE-S3)
	patient-triage-api-mgmt-artefacts-bucket	mgmt	saet-mgmt-artefacts-s3-kms	
DynamoDB				
Lambda				
Secrets Manager				
SSM Parameter Store				
AWS Backup				

AWS Service	Resource Name	Environment	Customer Managed Key (CMK) Key Aliases Adopted	Amazon S3 managed Keys (SSE-S3)
CloudWatch				

10.3 Encryption in Transit

Encryption in transit protects data as it moves between systems, preventing interception or tampering. In AWS, this is usually achieved through **TLS (Transport Layer Security)** protocols, such as HTTPS for web traffic or encrypted connections to databases and APIs.

S3	• Allow only encrypted connections over HTTPS (TLS) by using the `aws:SecureTransport` condition on S3 bucket policies. For more information, see What S3 bucket policy should I use to comply with the AWS Config rule s3-bucket-ssl-requests-only?[25] • Use Gateways endpoint for communication with S3	
VPC	• Use VPC endpoints in AWS PrivateLink to privately connect your VPCs to supported AWS services without using an internet gateway. • Configure security group rules[26] that allow traffic only from ports associated with secure protocols, such as HTTPS over TCP/443. Periodically audit security groups and their rules.	

25. https://aws.amazon.com/premiumsupport/knowledge-center/s3-bucket-policy-for-config-rule/

DynamoDB	• Use Gateways endpoint for communication with DynamoDB	
Lambda	• Use IAM DB authentication • Use SSL to communicate with RDS • Ensure the Lambda is within the VPC (private subnet)	
Secrets Manager	• Use VPC Interface endpoints for communication with Secrets Manager	
CloudTrail	• Enable log file validation for trails	

- For TLS we should use the latest version available. If we are using a lower version, we should get necessary approval from the Cyber Secure Team.
- Enforce certificate validation where needed
- Ensure weak ciphers are disallowed

10.4 KMS Compliance with AWS Config

AWS Config records the configuration of KMS keys and evaluates them against compliance rules. To ensure KMS Keys are compliant we should implement the below config rules.

dynamodb-table-encrypted-kms	Evaluation
iam-customer-policy-blocked-kms-actions	Evaluation
iam-inline-policy-blocked-kms-actions	Evaluation
kms-cmk-not-scheduled-for-deletion	Evaluation
kms-key-policy-no-public-access	Evaluation
kms-key-tagged	Evaluation

26. https://docs.aws.amazon.com/vpc/latest/userguide/security-group-rules.html

s3-default-encryption-kms	Evaluation

10.5 Logging

We will use the below AWS services to log API calls or operations or actions on KMS.

- Implement CloudTrail to log all necessary information

10.6 Monitoring and Alerting

We will set up CloudWatch Metrics for KMS. The metrics are published by KMS per CMK. Alerts can be set up on

- Sudden spikes in the usage
- Exceeded usage from the proposed baseline
- alerts you when your request rate reaches a certain percentage of a quota value. More information present here[27].
- The imported key material in a KMS key is nearing its expiration date. More information present here[28].
- A KMS key that is pending deletion is still being used. More information present here[29].
- The key material in a KMS key was automatically rotated.
- A KMS key was deleted.

10.7 Audits

We also need to audit the encryption coverage

Periodic audit of KMS keys across the project and encryption coverage	10th September	In Progress	10th March

10.8 Future Considerations

We should consider the below considering the advancements in quantum computing

- Use of AWS External Key Store (XKS) for advanced key control
- Use hybrid post-quantum TLS with AWS KMS

27. https://aws.amazon.com/blogs/security/manage-your-aws-kms-api-request-rates-using-service-quotas-and-amazon-cloudwatch/
28. https://docs.aws.amazon.com/kms/latest/developerguide/imported-key-material-expiration-alarm.html
29. https://docs.aws.amazon.com/kms/latest/developerguide/deleting-keys-creating-cloudwatch-alarm.html

10.9 References

Enforce SSL communication with S3 buckets and objects	What S3 bucket policy should I use to comply with the AWS Config rule s3-bucket-ssl-requests-only?[30]
Centralised or Decentralised Model for KMS Keys (AWS Document)	https://docs.aws.amazon.com/prescriptive-guidance/latest/aws-kms-best-practices/key-management.html#:~:text=disabling%20KMS%20keys-,Choosing%20a%20centralized%20or%20decentralized%20model,-AWS%20recommends%20that
Customer Managed, AWS Managed or AWS Owned KMS Keys	https://docs.aws.amazon.com/prescriptive-guidance/latest/aws-kms-best-practices/key-management.html#:~:text=KMS%20key%20model.-,Choosing%20customer%20managed%20keys%2C%20AWS%20managed%20keys%2C%20or%20AWS%20owned%20keys,-The%20KMS%20keys

30. https://aws.amazon.com/premiumsupport/knowledge-center/s3-bucket-policy-for-config-rule/

11 Patient Triage Api - Hotfix Strategy (Fix Forward)

11.1 Purpose

Define a hotfix approach that fixes the latest code line, validates in Reference, and only then promotes to Production. Hotfixes move forward rather than rolling back the codebase.

11.2 Principles

- Fix the current head of `main` (latest code line).
- Minimize change scope; focus on the defect and targeted tests.
- Always validate in Ref environment before Production.
- Prefer backward-compatible changes; avoid schema changes unless approved.

11.3 Entry Criteria

- Defect confirmed in Production and reproducible in Ref or Int.
- Hotfix can be isolated to a small change set.
- Risk assessment completed and approved by Tech Lead and Release Manager.
- Incident ticket created and labeled HOTFIX.

11.4 Fix Forward Workflow

```
1. Incident Identified in Production
2. Create Hotfix Branch from main
3. Implement Minimal Fix + Tests
4. CI Pipeline + Peer Review
5. Deploy to Reference and Validate
6. Approve and Deploy to Production
7. Post-Deployment Monitoring and Closure
```

11.5 Step-by-Step Process

11.5.1 1. Initiation

- Create branch: `hotfix/NPT-####-short-description` from `main`.
- Update release notes with scope, risk, and rollback posture.
- Notify on-call and QA of pending hotfix.

11.5.2 2. Implement and Verify

- Apply the smallest code change that resolves the defect.
- Add or update unit/integration tests to prevent regression.
- Run full CI checks (lint, security scan, unit, integration).
- Peer review by at least one senior engineer.

11.5.3 3. Promote to Reference

- Merge to `main` and deploy to Dev automatically.
- QA validates in Test using a targeted regression suite.
- Tag release candidate: `vX.Y.Z-hotfix.1`.
- Deploy to Reference and execute:
  - Smoke tests
  - Targeted regression tests for the defect
  - Clinical scenario validation (if clinical logic impacted)

11.5.4 4. Production Deployment

- Required approvals: Release Manager, Tech Lead, Clinical Safety Officer (if applicable).
- Deploy using standard production workflow (blue-green with canary).
- Heightened monitoring for 60 minutes.

11.5.5 5. Post-Deployment Actions

- Update incident ticket with root cause and fix details.
- Communicate resolution to stakeholders.
- Schedule post-incident review within 5 business days.

11.6 Ref Validation Gate (Mandatory)

- Hotfix must pass Ref validation before Production.
- If Ref validation fails, repeat fix-forward cycle.
- Rollback only if production impact is severe and ongoing.

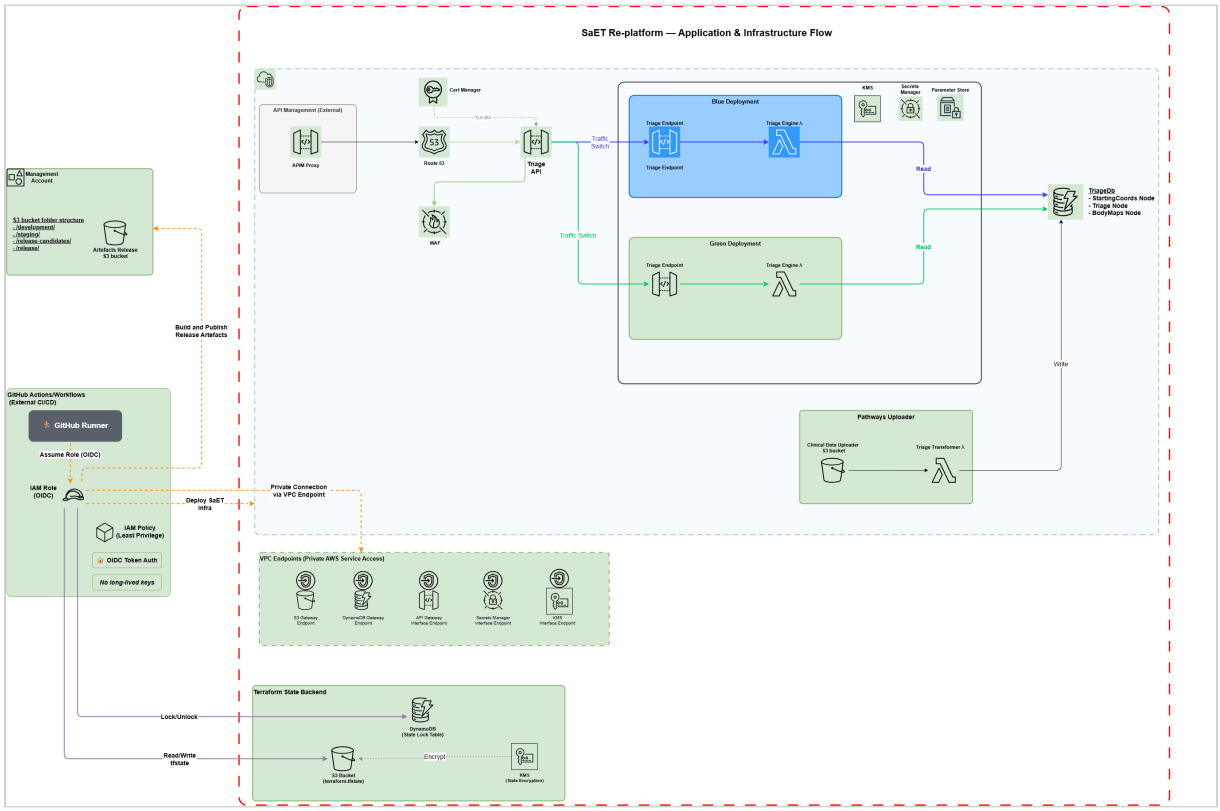
11.7 Data and Schema Considerations

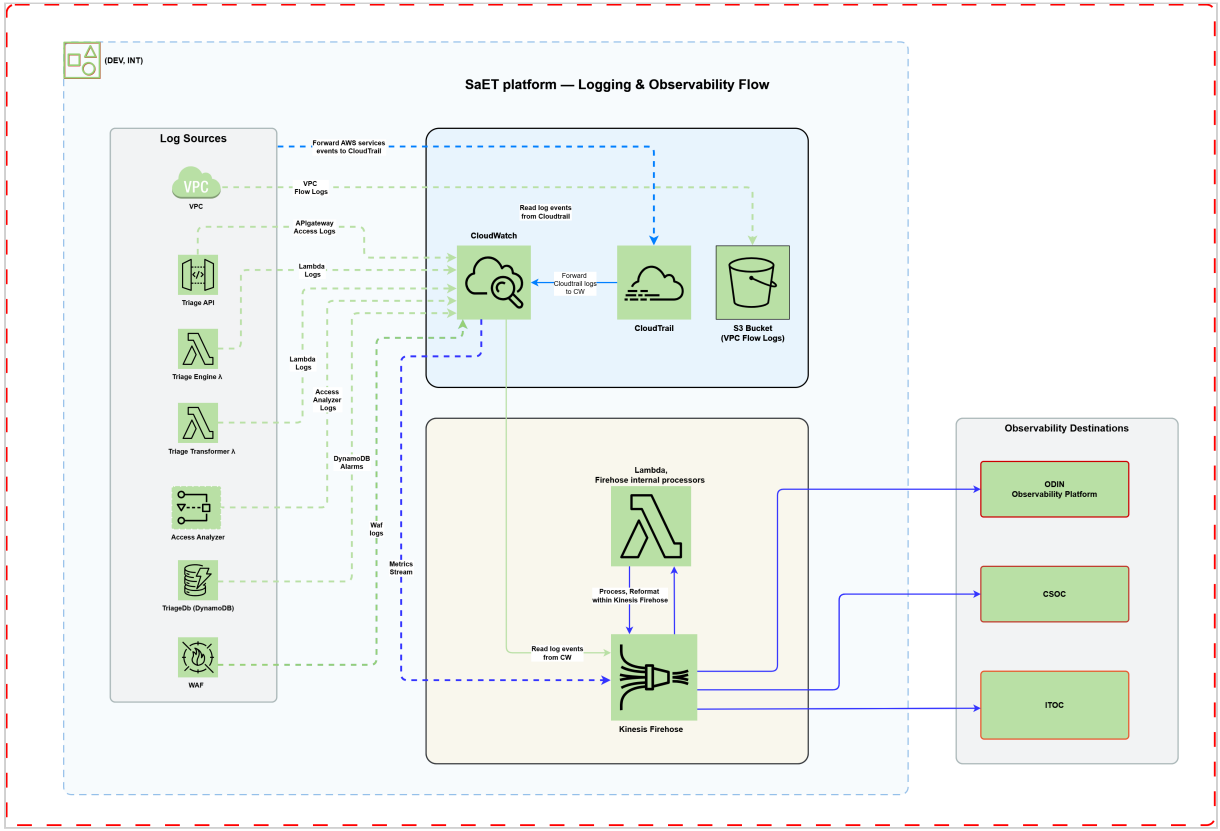
- Prefer backward-compatible changes only.
- If schema changes are unavoidable, require explicit CAB approval and expanded testing window.

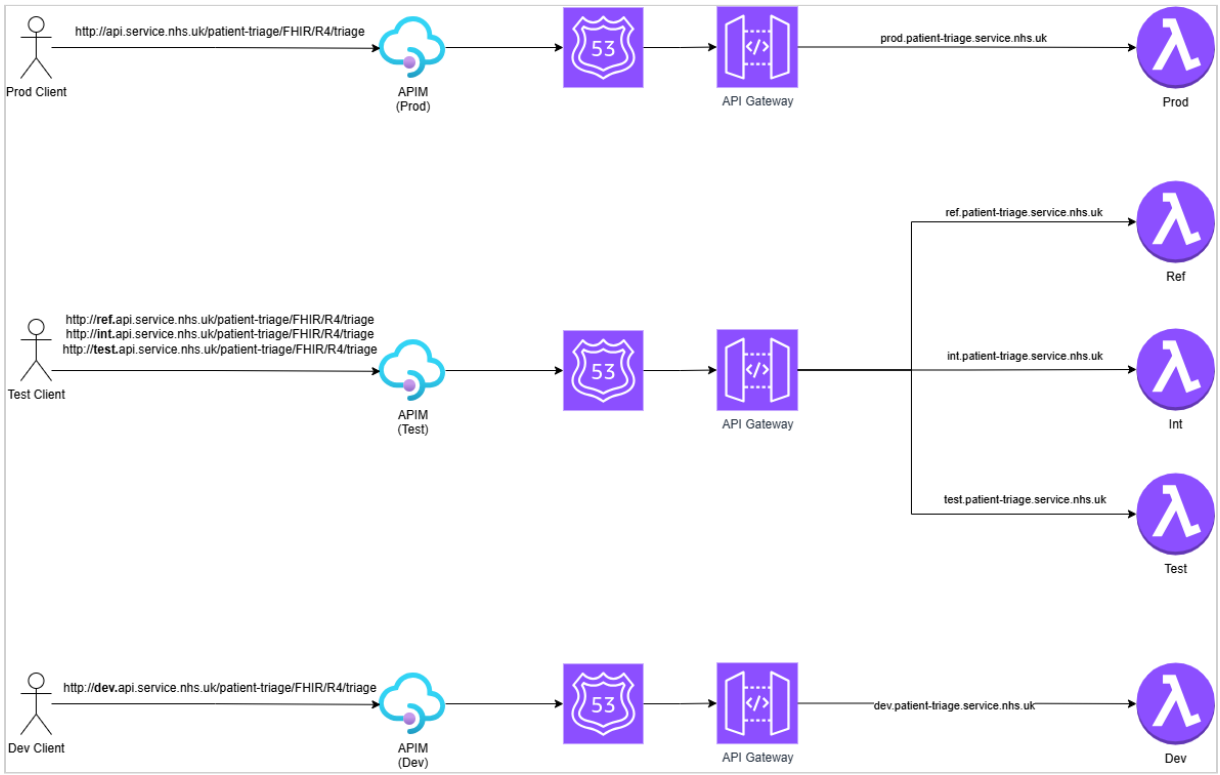
11.8 Exit Criteria

- Defect no longer reproducible in Ref.
- All automated tests green.
- Production metrics stable for 60 minutes.
- Stakeholders notified of resolution.

12 Patient Triage Api - Infrastructure Diagrams




(DEV, INT)
SaET platform — Logging & Observability Flow
Log Sources
VPC
Triage API
Triage Engine λ
Triage Transformer λ
Access Analyzer
TriageDb (DynamoDB)
WAF
Forward AWS services events to CloudTrail
VPC Flow Logs
APIgateway Access Logs
Lambda Logs
Lambda Logs
Access Analyzer Logs
DynamoDB Alarms
Waf logs
Metrics Stream
Read log events from Cloudtrail
CloudWatch
Forward Cloudtrail logs to CW
CloudTrail
S3 Bucket (VPC Flow Logs)
Lambda, Firehose internal processors
Process, Reformat within Kinesis Firehose
Read log events from CW
Kinesis Firehose
Observability Destinations
ODIN Observability Platform
CSOC
ITOC

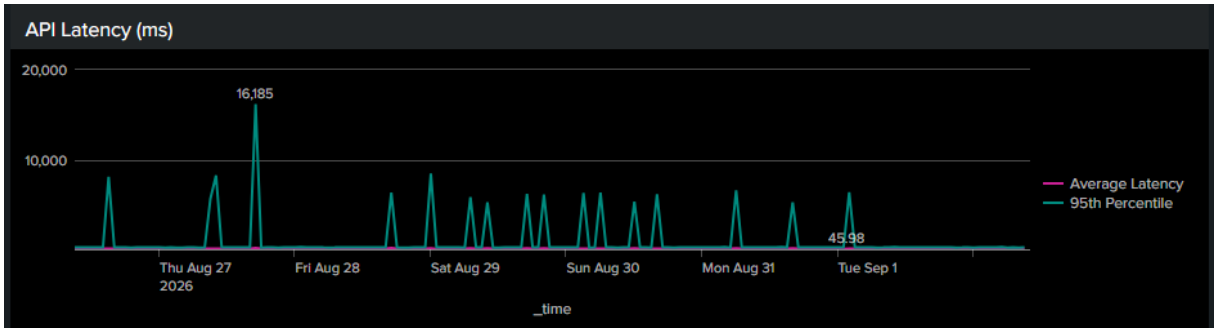

Prod Client
http://api.service.nhs.uk/patient-triage/FHIR/R4/triage
APIM (Prod)
API Gateway
prod.patient-triage.service.nhs.uk
Prod
Test Client
http://ref.api.service.nhs.uk/patient-triage/FHIR/R4/triage
http://int.api.service.nhs.uk/patient-triage/FHIR/R4/triage
http://test.api.service.nhs.uk/patient-triage/FHIR/R4/triage
APIM (Test)
API Gateway
ref.patient-triage.service.nhs.uk
Ref
int.patient-triage.service.nhs.uk
Int
test.patient-triage.service.nhs.uk
Test
Dev Client
http://dev.api.service.nhs.uk/patient-triage/FHIR/R4/triage
APIM (Dev)
API Gateway
dev.patient-triage.service.nhs.uk
Dev

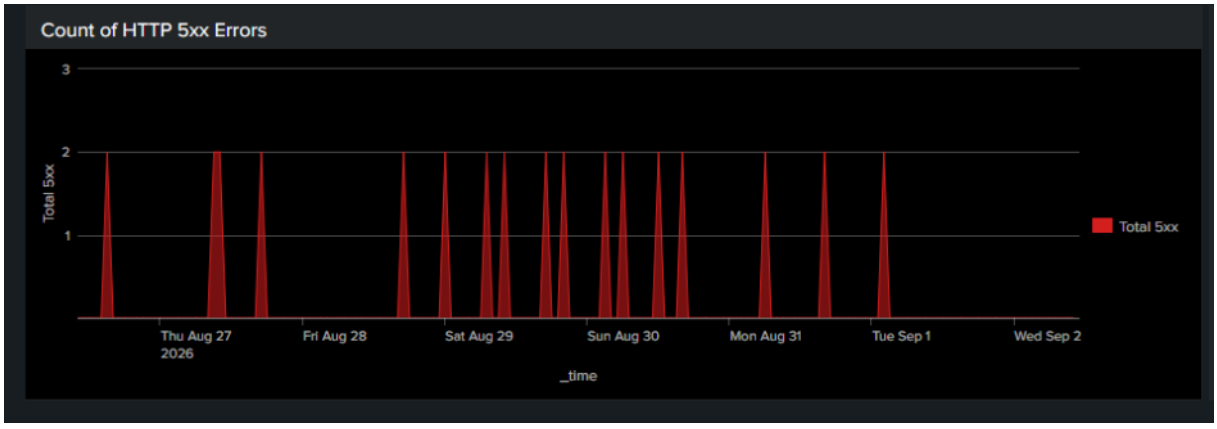
13 Patient Triage Api - Known issues

13.1 Long APIM Delays

13.1.1 Description

We have been experiencing sporadic delays coming from the APIM environment. There will be occasional delays of several seconds before the service returns to normal. These delays appear to coincide with 500 errors that are being thrown in the APIM environment. This can be seen from the below graph from the APIM Splunk dashboard[31].





We can see that there are no 500 errors being generated on the API over that time.

31. https://nhsdigital.splunkcloud.com/en-US/app/nhse_patient_triage_api_all_sh_all_viz/itoc_-_patient_triage_-_apm?form.global_time.earliest=-7d%40h&form.global_time.latest=now&form.span=1d&form.env=prod&form.method_token=*&form.status_filter=proxy.suffix%3D%22*%22



13.1.2 Steps take so far

We created a service now ticket[32] on 14 Jul 2026

- APIM confirmed that this was a known issue on their side and they needed to make a fix to it. *502 is a known issue. We have some documentation available here:* https://nhsd-confluence.digital.nhs.uk/spaces/APM/pages/996023031/APM-5921+Apigee+latency+issue+-Unexpected+EOF+at+target . *Can you please let us know which service you are trying to invoke which is causing 502 ?*
- The outcome of the service now ticket was that they implemented a temporary fix. The fix is to kill a long running calls after 2 seconds so they retry again rather than hang for longer.

  Hi, we had implemented a temporary fix in Int environment and it got overwritten on Jul 24th, 2026 due to new revision. We have applied the fix again. Can this be retested?
  Please note that any release of api in IINT will override the fix.
- This ticket was marked as resolved with the temporary fix as the solution.

We have created a second service now ticket[33] on 02 Sept 2026

- We are asking for a long term fix to the issue not just a temporary fix.

32. https://nhsdigitallive.service-now.com/now/nav/ui/classic/params/target/incident.do%3Fsys_id%3Dc745e502c38e8f100df1f45a050131e0%26sysparm_view%3Dess%26sysparm_record_target%3Dincident%26sysparm_record_row%3D4%26sysparm_record_rows%3D5%26sysparm_record_list%3Dcaller_id%253Djavascript%253Ags.getUserID%2528%2529%255EORDERBYnumber%26sysparm_view%3Dess
33. https://nhsdigitallive.service-now.com/nav_to.do?uri=incident.do?sys_id=4907428247034310179 5de7b716d43ac%26sysparm_view=ess

14 Patient Triage Api - Logging and Monitoring Strategy

14.1 Overview

This page describes the approach for implementing standardised logging and monitoring across the Patient Triage API and connected AWS infrastructure. It combines the operational logging standards with the alerting strategy defined in the companion alerting documents so that the service has a single, consistent view of observability, alert thresholds, and response actions.

The monitoring model supports four outcomes:

- Detecting service degradation before it impacts users.
- Providing enough evidence to diagnose whether the issue is latency, capacity, dependency failure, or a response-mix shift.
- Maintaining compliance and security visibility through structured AWS log retention and access logging.
- Aligning dashboard telemetry and alerts to the same GREEN / AMBER / RED operating model used in the alerting strategy.

14.2 Logging Standards

14.2.1 Log Group Naming Convention

All AWS resources will follow the CloudWatch log group naming standard:

```
/aws/{AWS_RESOURCE}/{PROJECT}-{ENV}-{STACK}-{RESOURCE_NAME}
/aws/{AWS_RESOURCE}/{PROJECT}-{ENV}-{STACK}-{RESOURCE_NAME}-{WORKSPACE}
```

Example:

- /aws/lambda/saet-dev-triage-TBC (see page 105)
- /aws/api-gateway/saet-dev-triage-TBC (see page 105)

Exceptions:

- WAF: aws-waf-logs-saet-dev-read-only-viewer-waf-web-acl-log-group

14.2.2 CloudWatch Retention Policies

Log Type	Dev / Test	Sandpit	Int	Pre-Prod	Prod	Notes / Purpose
Application logs	14 days (default)	14 days (default)	30 days	30 days	90 days	Debugging, feature monitoring
Audit logs	30 days	30 days	30 days	180 days	1-7 years	Security, compliance, tracking user actions
System logs	14 days	14 days	30 days	60 days	90 days	Server health, OS events, troubleshooting

14.2.3 ODIN Retention Policies (TBC)

Log Type	Dev / Test	Sandpit	Int	Pre-Prod	Prod	Notes / Purpose
Application logs			30 Days	30 Days	90 Days	Debugging, feature monitoring
Audit logs			30 Days	30 Days	90 Days	Security, compliance, tracking user actions
System logs			30 Days	30 Days	90 Days	Server health, OS events, troubleshooting

14.2.4 AWS Service Logging

- S3 access logging must be enabled on all buckets holding sensitive or critical data.
- CloudTrail must be enabled across all accounts with log file validation.
- VPC Flow Logs should be enabled for production and integration environments and retained according to compliance requirements.

14.2.5 S3 Bucket Logging Matrix

Bucket Type	Logging Config	Requirement
Terraform State Buckets	TBC (see page 105)	✓ Mandatory
Flow Log Buckets	TBC (see page 105)	✓ Mandatory

Bucket Type	Logging Config	Requirement
CloudFront / Access Log Buckets	TBC (see page 105)	✓ Mandatory
Cert Buckets	TBC (see page 105)	✓ Mandatory
Artefact Buckets	TBC (see page 105)	⚠️ Optional

14.3 KPI and Alerting Strategy Alignment

The alerting strategy defines the service operating model: each KPI has a GREEN state (operating within target), an AMBER state (degraded or approaching breach), and a RED state (SLA breach or imminent breach). The monitoring and dashboard design must align to this model so operators can move quickly from telemetry to action.

Alert thresholds are derived from the NFR-defined targets. AMBER thresholds allow early warning before a RED state is reached. All production continuous metric alerts should be implemented and notified through ODIN / Grafana.

14.3.1 Alert State Definitions

14.3.1.1 Availability

KPI	GREEN	AMBER	RED	Alert mechanism	Source NFR
Service availability	>= 99.90% in the rolling 30-day window	99.50% - 99.89%	< 99.50%	ODIN / Grafana availability alert rule. Alert fires when 30-day rolling availability drops below 99.90% (CloudWatch used as telemetry source only).	NFR: Availability[34]

14.3.1.2 Recovery

Recovery objectives are time-tracked during declared major incidents. Alert states represent progress against RTO and RPO commitments rather than continuous metric thresholds.

34. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208308/NFR+Availability

KPI	GREEN	AMBER	RED	Alert mechanism	Source NFR
Recovery Time Objective (RTO) - 2 hours	Service restored within 90 minutes of incident declaration	Service not restored within 90 minutes of incident declaration	Service not restored within 2 hours of incident declaration	ODIN / Grafana incident-timer alert integrated with the incident management tool. AMBER Teams message to lead Developer at T+90 min. RED escalated to service owner at T+2 hours.	NFR: Availability[35]

14.3.1.3 Performance

KPI	GREEN	AMBER	RED	Alert mechanism	Source NFR
P95 response time - Triage POST endpoint	< 300 ms	300 ms - 500 ms	> 500 ms	ODIN / Grafana P95 latency alert rule. AMBER fires when P95 crosses 300 ms. RED fires at 500 ms, the load-test ceiling defined in the scalability NFR. Evaluated over a 5-minute rolling window. AWS X-Ray used for root-cause analysis.	NFR: Performance[36]
Overall error rate during peak load	< 0.1 %	0.1% - 0.5%	> 0.5 %	ODIN / Grafana error-rate alert rule; evaluated over a 5-minute rolling window during peak traffic hours.	NFR: Performance[37]
5xx error rate under normal load	<= 0.01%	0.01% - 0.05%	> 0.05%	ODIN / Grafana 5xx alert rule evaluated over a 5-minute rolling window under normal operating conditions. Separate alert from peak-load error rate to allow independent tuning.	NFR: Performance[38]

35. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208308/NFR+Availability
36. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208291/NFR+Performance
37. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208291/NFR+Performance
38. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208291/NFR+Performance

14.3.1.4 Capacity and Throughput

KPI	GREEN	AMBER	RED	Alert mechanism	Source NFR
Observed throughput against peak demand baseline (6.56 TPS)	< 9 TPS (below 1.4x baseline; autoscaling not yet required)	9 TPS - 13 TPS (1.4x - 2x baseline; autoscaling in progress, confirm scaling response)	> 13 TPS sustained without confirmed autoscale completion (2x baseline exceeded before scaling is confirmed healthy)	ODIN / Grafana TPS alert rule on the Triage POST endpoint. AMBER is used to confirm autoscaling is responding before the traffic resilience target is breached.	NFR: Scalability[39]
Observed throughput against capacity ceiling (120 TPS)	< 80 TPS (< 67% of design capacity)	80 TPS - 100 TPS (67% - 83% of design capacity)	> 100 TPS (> 83% of design capacity; approaching the load-test ceiling with limited headroom)	ODIN / Grafana TPS alert rule. AMBER provides early warning that demand is closing on the validated load-test ceiling and triggers a capacity review.	NFR: Scalability[40]
Traffic resilience - sustained load vs normal volume	Sustained traffic <= 1.5x normal volume, service healthy	Sustained traffic 1.5x - 2x normal volume, service health under review	Sustained traffic >= 2x normal volume with degraded health indicators (latency or error rate entering AMBER or RED)	ODIN / Grafana composite alert combining TPS metric and health indicators. Triggered when both conditions are met simultaneously.	NFR: Scalability[41]

14.3.2 Alert Response Summary

Operational response should follow the service alerting model defined by the alerting strategy:

- GREEN service operating within target; no action required.
- AMBER service degraded or approaching breach; investigation required.
- RED SLA breach or imminent breach; immediate escalation required.
- Continuous metric alerts (latency, error rate, throughput) should be configured in ODIN / Grafana with evaluation periods of at least 5 minutes to reduce noise from transient spikes.
- Availability and maintenance thresholds are calculated over a rolling 30-day window unless stated otherwise.
- The AMBER threshold for P95 latency (300 ms) is aligned with the NFR-defined service target so any breach of the target immediately generates an operational warning.
- The RED threshold for P95 latency (500 ms) is aligned with the scalability NFR load-test ceiling, representing the upper boundary of validated service behaviour.

39. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208298/NFR+Scalability
40. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208298/NFR+Scalability
41. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208298/NFR+Scalability

14.3.3 Alert Implementation Notes

- All continuous metric alerts should be implemented in ODIN / Grafana and aligned to the same signal sources used by the dashboard.
- CloudWatch metrics are used as telemetry sources; operational alerts are routed and managed through ODIN / Grafana.
- Dashboard visual states should reflect the GREEN / AMBER / RED boundaries defined on this page so operators have consistent visual alignment between data and incident triage.
- Availability must be reviewed over a rolling 30-day window to match the service SLO and alerting model.

14.3.4 Known Gaps Affecting Alert Coverage

The alerting strategy also identifies current limitations that should be treated as operational caveats:

- Autoscaling speed is not directly alerted; current alert coverage captures downstream throughput and latency effects rather than scaling response time itself.
- Some environment-specific baselines are still best-effort and may require calibration once live data is available.
- Certain service-specific metrics, especially in security and dependency coverage, may need tuning after the first production operating cycle.

14.4 Metrics and Alerts

14.4.1 KPI Summary

The application KPIs are centered on service health, error rate, traffic, and latency. The dashboard provides visibility into those KPIs, and the alert rules trigger when the KPI moves outside the accepted operating range. In practice, this means:

- Availability alerts protect the user-facing success rate over time.
- Error-rate alerts catch sudden degradation before it becomes a wider incident.
- Traffic alerts identify overload, no-traffic, or anomalous load conditions.
- Latency and response-class panels provide supporting evidence for investigation, even where they are not directly alerted.

The alert set is intentionally focused on actionable KPI breaches rather than every metric fluctuation.

14.4.2 KPI Coverage

The dashboard for **Patient Triage API - Service KPI Overview** is the primary KPI view used to interpret service health and alert states.

KPI	Dashboard coverage	Alert coverage	Why this meets the KPI
Availability	Service Availability Over 30 Days	Service Availability 99.9% and Service Availability 99.5%	Keeps the service within the expected availability target and raises an alert when the 30-day outcome drops below the acceptable threshold.
Error rate	5xx Rate, Peak Error Rate, Response Class Split	Peak Error Rate > 0.1%, Peak Error Rate > 0.5%, 5xx Rate	Detects when failure rates rise above the operating band and distinguishes early warning from serious degradation.
Traffic / throughput	Current TPS (5min), Current TPS vs Baseline, Throughput Trend	Current TPS > 9 TPS, Current TPS > 13, Current TPS = 0	Detects unusual traffic movement, overload, or complete traffic loss so the team can respond before the KPI turns into a service issue.
Latency	Latency Signals (Avg/Sum), P50 response time, P95 response time, P99 response time, Triage Latency Avg (Proxy)	None directly; used for triage	Keeps latency visible for operational review and incident diagnosis, even though it is not a paging KPI.

14.4.2.1 Primary operational signals

Dashboard panel	Purpose	How it supports the alerting strategy
Service Availability Over 30 Days	Long-window availability view	Matches the availability alerts and shows whether the service is meeting its SLO target over time.
4xx Rate	Client-error trend	Useful for diagnosis and release validation, but not a paging signal in this strategy.
5xx Rate	Server-error trend	Mirrors the critical error condition behind the 5xx alert.
Current TPS (5min)	Current request volume	Confirms whether traffic is healthy, overloaded, or unexpectedly absent.
Current TPS vs Baseline	Current traffic versus expected baseline	Helps distinguish a real traffic drop from normal variation.
Peak Error Rate	Error burst indicator	Supports the higher-severity error-rate alerts.
Latency Signals (Avg/Sum)	Latency trend context	Provides investigation context for performance issues even though latency is not currently a paging trigger.

Dashboard panel	Purpose	How it supports the alerting strategy
Response Class Split	Response mix over time	Helps explain whether a degradation is dominated by 2xx, 4xx, or 5xx outcomes.

14.4.3 Alert Coverage

The alert rules are built from the same Prometheus/CloudWatch metrics used in the dashboard, which keeps the KPI view and the paging rules aligned.

Alert	Trigger	Severity intent	Strategy fit
Service Availability 99.9%	Rolling 30-day availability drops below 99.9%	Highest importance availability breach	Directly enforces the availability KPI.
Service Availability 99.5%	Rolling 30-day availability drops below 99.5%	Lower availability threshold for extended degradation	Provides the second boundary so the team can distinguish warning-level from critical KPI drift.
Peak Error Rate > 0.1%	Peak 5xx + 4xx error rate exceeds 0.1% on the ref workload slice	Early warning	Detects KPI deterioration before it becomes broad user impact.
Peak Error Rate > 0.5%	Peak 5xx + 4xx error rate exceeds 0.5% on the ref environment	Higher-severity error condition	Escalates when the error KPI is clearly outside the acceptable operating range.
5xx Rate	5xx rate exceeds 0.01% over the last 5 minutes	Immediate server-error signal	Protects the error KPI from server-side failures that users would feel first.
Current TPS > 9 TPS	Request rate rises above 9 TPS	Load anomaly	Flags a traffic KPI shift that may require capacity or dependency checks.
Current TPS > 13	Request rate rises above 13 TPS	Higher load anomaly	Escalates sustained or bursty load above the lower traffic KPI band.
Current TPS = 0	Request rate falls to zero	Availability or intake failure signal	Detects complete traffic loss which may indicate an outage, upstream routing issue, or broken integration path.

14.4.3.1 Alert threshold summary

Metric	Warning	Critical
Availability	Below 99.9%	Below 99.5%
Peak Error Rate	> 0.1%	> 0.5%
5xx Rate	> 0.01% for 5 min	Escalated server-side degradation
Current TPS	> 9 TPS	> 13 TPS or = 0

14.4.4 Intentional Non-Alerted Metrics

Some dashboard metrics are intentionally not used as alerts:

- 4xx rate is visible for troubleshooting, but is usually more useful as a diagnostic trend than as a paging condition.
- P50 / P95 / P99 and latency proxy panels remain in the dashboard for triage, but latency alone is not currently a paging KPI.
- Response class split and throughput trend provide supporting context rather than independent alert conditions.

This keeps the alert set focused and reduces noise while preserving the evidence needed to understand incidents quickly.

14.4.5 Why these metrics meet the strategy

1. Is the availability KPI healthy? The 30-day availability panel and availability alerts answer this.
2. Is the error KPI healthy? The 5xx and peak error panels plus the error-rate alerts answer this.
3. Is the traffic KPI healthy? The TPS panels and traffic alerts answer this.

That combination is effective because it covers both symptom detection and investigation context:

- Availability alerts cover the SLO outcome behind the availability KPI.
- Error-rate alerts catch direct user impact and burst failures that move the error KPI.
- TPS alerts catch traffic anomalies that often explain or precede KPI movement.
- Latency and response-class panels help determine whether the KPI issue is latency, saturation, dependency failure, or a response-mix shift.

14.5 Dashboards

Dashboards should provide a clear operational view of the service and should align directly to the same KPIs and thresholds used in the alert rules. Minimum dashboard coverage should include:

- Service Availability Over 30 Days
- 5xx Rate
- Peak Error Rate
- Current TPS (5min)
- Current TPS vs Baseline
- Latency Signals and P95/P99 response time
- Response Class Split

These views provide both KPI monitoring and the operational evidence needed for user-impact analysis and post-incident review.

14.6 References

- https://docs.aws.amazon.com/dms/latest/userguide/CHAP_Monitoring.html#CHAP_Monitoring.Metrics
- Patient Triage API - Alerting Strategy (see page 25)
- Patient Triage API KPI and alerting strategy (see page 29)

15 Patient Triage Api - Performance Testing Strategy

15.1 Summary

This document outlines the comprehensive performance testing strategy for the Patient Triage API. The strategy is designed to validate that the system meets defined performance objectives, scales appropriately, maintains service reliability under load, and supports the private beta for 111 Online with a full rollout across all providers as future state, with acceptable response times.

The strategy is built on the Non-Functional Requirements (NFRs) for Performance, Scalability, Availability, and Reliability, and includes test objectives, types, execution approach, success criteria, and monitoring requirements.

15.2 Document Control

Item	Details
Author	Performance Engineering Team
Document Version	1.0
Date Created	March 27, 2026
Last Updated	March 27, 2026
Status	Draft
Next Review Date	April 27, 2026

15.3 1. Test Objectives

The performance test strategy aims to:

15.3.1 Primary Objectives

- Validate that the Triage POST endpoint meets the **300 ms P95 latency** SLA under normal and peak load conditions
- Confirm the system scales automatically to handle peak load (6.56 TPS current baseline) without breaching latency targets

- Verify the solution can handle load-tested capacity of **120 TPS** (representing future growth scenarios, Platinum level service with widescale adoption across UEC telephony providers in England and 111 Online.)
- Demonstrate the platform can absorb at least **2x normal traffic volume** without degradation
- Validate overall error rate remains below **0.1%** during peak load operations
- Validate 5xx error rate remains below **0.01%** under normal operating conditions
- Validate system recovery and resilience following failure scenarios

15.3.2 Secondary Objectives

- Identify performance bottlenecks before production deployment
- Establish performance baselines for future comparisons and continuous improvement
- Validate monitoring and alerting mechanisms capture performance metrics accurately
- Test autoscaling response time and effectiveness
- Validate data layer (DynamoDB, ElastiCache) performance under sustained load
- Confirm AWS-managed services (Lambda, API Gateway) function within design parameters

15.4 2. Performance KPIs and Targets

15.4.1 Response Time

Metric	Target	Acceptance Criteria
P95 Response Time (Triage POST)	< 300 ms	Critical - Must be met in all tests
P99 Response Time (Triage POST)	< 500 ms	Test threshold for scalability scenarios
P50 Response Time (Triage POST)	< 150 ms	Baseline expectation
End-to-End Latency	Measured at API Gateway boundary	AWS autoscaling must maintain target latency

15.4.2 Throughput and Capacity

Metric	Current	Target Load Test	2x Traffic Buffer
Peak Demand (TPS)	6.56	120	13.12+
Sessions Per Busiest Hour	2,147	N/A	4,294+

Metric	Current	Target Load Test	2x Traffic Buffer
Average Journey Length	11 transactions	11 transactions	11 transactions

15.4.3 Reliability Under Load

Metric	Target	Measurement
Overall Error Rate (Peak Load)	< 0.1%	Error-rate monitoring during peak load testing
5xx Error Rate (Normal Load)	< 0.01%	5xx response monitoring under standard conditions
Service Availability	99.90%	Continuous measurement across test duration
Message Sequence Resilience	No failures	API calls must not fail due to sequence dependency

15.5 3. Test Scope and Scale

15.5.1 In Scope

- Patient Triage API application (all endpoints, focusing on Triage POST)
- AWS Lambda compute layer and autoscaling behaviour
- API Gateway and load balancer scaling characteristics
- DynamoDB data layer performance and throughput
- ElastiCache (if implemented) performance under load
- End-to-end latency from consumer request through API Gateway boundary
- Error handling and recovery mechanisms
- Monitoring and alerting system accuracy

15.5.2 Out of Scope

- Third-party integrations external to the API platform (ERS/PDS Gateway behaviour reliability)
- Network infrastructure external to AWS (N3/HSCN performance)
- Security vulnerability testing (covered under separate security NFR)
- Load Balancer configuration external to API platform
- Disaster recovery regional failover (covered under Availability NFR testing)

15.6 4. Performance Test Types and Approach

15.6.1 4.1 Baseline Performance Test

Purpose: Establish baseline performance under normal operating conditions

Approach:

- Load: 2 TPS sustained for 30 minutes
- Measurements: P50, P95, P99 response times; error rates; resource utilization
- Acceptance: P95 < 300 ms, error rate < 0.01%
- Tools: JMeter
- Metrics Collection: CloudWatch, AWS X-Ray, ODIN / Grafana dashboards

15.6.2 4.2 Load Test - Current Peak Demand

Purpose: Validate performance at current observed peak load

Approach:

- Load Profile: Sustained at 6.56 TPS (current peak from 2,147 sessions/hour)
- Duration: 60 minutes sustained
- Ramp-up: 10 minutes to reach steady state
- Think Time: Realistic user think time between requests (avg 5-10 seconds)

Success Criteria:

- P95 latency < 300 ms
- Error rate < 0.1%
- System responds to load within 2 minutes
- Resource Monitoring: Lambda concurrency, DynamoDB consumed capacity, API Gateway throttling events

15.6.3 4.3 Stress Test - Future State Capacity

Purpose: Validate the system can handle future growth (production load test target)

Approach:

- Warm up to 6.56 TPS
- Load: Ramp from 6.56 TPS to 120 TPS over 30 minutes
- Sustained: Hold 120 TPS for 60 minutes
- Peak: 120 TPS for 60 minutes

Acceptance Criteria:

- P95 latency remains < 300 ms throughout

- Error rate < 0.1% at sustained 120 TPS
- Autoscaling responds within acceptable timeframe (TBC)
- No throttling or service degradation observed
- Failure Scenarios: Test behaviour when hitting limits
- Measurement: Autoscaling events, Lambda cold starts, cache hits/misses

15.6.4 4.4 Endurance Test (Soak Test)

Purpose: Validate system stability over extended operational periods

Approach:

- Load: 120 TPS
- Duration: 4 hours continuous
- Objective: Identify memory leaks, connection pool exhaustion, gradual degradation
- Metrics: Response time trends over time, resource utilization trending
- Success Criteria: No degradation in P95 latency; consistent error rates

15.6.5 4.5 Spike Test

Purpose: Validate autoscaling response and system behaviour during sudden traffic spikes

Approach:

- Baseline: 2 TPS sustained for 5 minutes
- Spike: Sudden increase to 50 TPS for 10 minutes
- Return: Ramp back down to 2 TPS

Measurements:

- Time to scale up (autoscaling response time)
- Time to scale back down
- P95 latency during spike (should remain < 300 ms)
- Error rate during transition
- Success: System handles spike without SLA breach

15.6.6 4.6 Capacity Test - 2x Normal Traffic

Purpose: Demonstrate resilience to unexpected traffic surges

Approach:

- Load: 2x current peak (13 TPS) sustained for 30 minutes
- Acceptance: System remains responsive; degrades gracefully if necessary
- Minimum Criteria: P95 latency < 500 ms; error rate < 1%

- Documentation: Identify constraint limits and scaling ceiling

15.6.7 4.7 Error Scenario Testing

Purpose: Validate system resilience and error handling under load

Approach:

- Dependency Failure: Simulate downstream service failures (ERS/PDS Gateway timeouts)
- Error Injection: Introduce 5% error responses; measure recovery
- Partial Failure: Simulate availability zone failure; verify multi-AZ failover
- Resource Exhaustion: Approach DynamoDB throughput limits; measure throttling behaviour
- Measurements: Error recovery time, cascade failure prevention, system state consistency

15.6.8 4.8 Data Layer Performance Testing

Purpose: Validate DynamoDB and caching performance

Approach:

- DynamoDB Performance:
  - Write throughput: Test sustained write capacity under 120 TPS load
  - Read throughput: Validate query performance with realistic data volumes
- Latency: Measure P95 latency for high-volume read/write operations
- ElastiCache (if applicable):
- Cache hit ratio: Validate cache effectiveness
- Eviction rate: Monitor cache displacement under memory pressure
- Latency: Compare cached vs. non-cached response times

15.7 5. Test Execution Plan and Timeline

15.7.1 Phase 1: Test Environment Setup and Baseline

- Prepare test environments (DEV, TEST, pre-PROD)
- Configure monitoring and logging (CloudWatch, X-Ray, ODIN / Grafana)
- Develop test scripts and scenarios (JMeter/Gatling)
- Execute baseline performance test
- Establish performance baselines

15.7.2 Phase 2: Functional Load Testing

- Execute current peak demand load test (6.56 TPS)

- Execute 2x traffic capacity test (13.12 TPS)
- Validate error handling and recovery
- Data layer performance validation

15.7.3 Phase 3: Stress and Scalability Testing

- Execute stress test to 120 TPS (future state capacity)
- Execute spike tests with autoscaling validation
- Identify resource bottlenecks and scaling limits
- Document autoscaling response times

15.7.4 Phase 4: Endurance and Soak Testing

- Execute 4-6 hour endurance test
- Monitor for memory leaks and resource degradation
- Validate long-running stability

15.7.5 Phase 5: Reports and Recommendations

- Compile performance test results
- Compare against acceptance criteria
- Document findings and recommendations
- Identify optimization opportunities

15.8 6. Test Environment Requirements

15.8.1 Infrastructure

- Test Environment: Replicate production architecture (multi-AZ deployment)
- Isolation: Dedicated AWS account or isolated VPC to prevent impact on other systems
- Data: Representative production-like data volumes
- Configuration: Match production Lambda concurrency limits, DynamoDB capacity settings

15.8.2 Tools and Technologies

Category	Tool	Purpose
Load Generation	JMeter	Application load generation and scenario execution

Category	Tool	Purpose
Monitoring	CloudWatch	AWS resource metrics and logging
Tracing	AWS X-Ray	End-to-end latency analysis and distributed tracing
Dashboards	ODIN / Grafana	Real-time performance metrics visualization
Alerting	CloudWatch Alarms	Real-time issue detection during tests
Data Collection	CloudWatch Logs	Centralized logging and analysis
Reporting	Custom dashboards / Excel	Test result aggregation and reporting

15.8.3 Test Data Requirements

- User Sessions: Minimum 2,000+ concurrent sessions to simulate realistic load
- Transactions: 6.56 TPS baseline rising to 120 TPS in stress test
- Data Volume: Production-representative schema and record counts
- Cache Population: Pre-populate cache with realistic hit ratio expectations

15.9 7. Success Criteria and Acceptance

15.9.1 Critical Pass Criteria (Must Meet)

- ✓ P95 response time for Triage POST endpoint remains < 300 ms under all load test scenarios up to 120 TPS
- ✓ Overall error rate during peak load (6.56 TPS) remains < 0.1%
- ✓ 5xx error rate under normal load remains < 0.01%
- ✓ Service maintains 99.90% availability during test window (no unplanned downtime)
- ✓ No data loss or corruption observed during any test scenario
- ✓ Autoscaling responds to load changes and maintains SLA during spike tests

15.9.2 Conditional Pass Criteria (Should Meet)

- P99 response time remains < 500 ms under all load test scenarios
- Error rate during 2x traffic (13.12 TPS) remains < 0.5%
- Autoscaling response time is documented and acceptable (TBC specification)
- Cache hit ratio meets expectations (when implemented)

- DynamoDB throughput utilization is optimized (no wasteful over-provisioning)

15.9.3 Fail Criteria (Automatic Failure)

- ✗ P95 response time exceeds 300 ms at current peak load (6.56 TPS)
- ✗ Data corruption or loss detected in any scenario
- ✗ System remains in failed state after error injection test
- ✗ Error rate at peak load exceeds 0.5%
- ✗ Cascading failure observed (single component failure brings down entire system)

15.10 8. Monitoring and Observability During Testing

15.10.1 Metrics to Collect

- Application Metrics:
- P50, P95, P99, P99.9 response times
- Request throughput (RPS)
- Error rates by type (4xx, 5xx)
- Success rate percentage

Infrastructure Metrics:

- Lambda: Concurrent executions, duration, cold start count, throttles
- API Gateway: Request count, latency, throughput throttling
- DynamoDB: Consumed read/write capacity, throttling events, latency
- ElastiCache: Cache hits/misses, evictions, memory utilization

System Metrics:

- CPU utilization
- Memory utilization
- Network throughput (ingress/egress)
- Disk I/O utilization

15.10.2 Alerting Thresholds During Tests

- P95 Latency > 300 ms - Trigger alert (SLA breach)
- Error Rate > 0.5% - Trigger alert (quality degradation)
- Lambda Throttling Events > 0 - Trigger alert (capacity issue)
- DynamoDB Throttling > 0 - Trigger alert (throughput limit reached)
- API Gateway Throttling Events - Trigger alert (upstream limit issue)

15.10.3 Real-Time Dashboards

- ODIN / Grafana Dashboard: Track P50, P95, P99 latency, error rates, and saturation signals
- AWS CloudWatch: Monitor Lambda, API Gateway, DynamoDB performance in real-time
- Custom Dashboard: Aggregate test results with pass/fail indicators

15.11 9. Reporting and Analysis

15.11.1 Test Report Contents

- Executive Summary: Overall pass/fail status against acceptance criteria

Detailed Results:

- Response time percentiles for each test scenario
- Error rate analysis and breakdown
- Throughput and capacity findings
- Autoscaling behaviour observations

Performance Graphs:

- Response time trending over test duration
- Throughput vs. latency relationships
- Resource utilization over time
- Bottleneck Analysis:
- Identified constraints and limits
- Component behaviour analysis
- Scaling effectiveness assessment

Recommendations:

- Configuration optimizations
- Capacity planning insights
- Architectural improvements
- Operational guardrails

15.11.2 Comparative Analysis

- Baseline vs. Peak Load: Performance delta analysis
- Test Iterations: Trend analysis across multiple test runs
- Pre-Production vs. Production: Post-deployment validation

15.12 10. Risk and Mitigation

15.12.1 Test Risks

Risk	Impact	Mitigation
Test data insufficient to represent production volume	Invalid performance conclusions	Pre-populate with representative data; use production snapshots where possible
Monitoring system overload during peak test loads	Loss of critical metrics	Scale monitoring infrastructure; use sampling where needed
External dependency failures during testing	Test results contaminated	Mock external dependencies; test in isolated environment
Cost overruns from extended test environments	Budget impact	Set fixed test windows; use cost-optimized test infrastructure
Test environment configuration drift from production	Results not representative	Automated provisioning; regular sync with production config

15.12.2 Performance Risks

Risk	Impact	Mitigation
Autoscaling insufficient to meet demand	SLA breach at peak load	Pre-test autoscaling configuration; test scaling limits
Cold start impact on latency during spike	Temporary SLA breach	Monitor cold start frequency; optimize function size/provisioning
Data layer unable to scale with application layer	Bottleneck at database	Pre-test DynamoDB capacity; load test data layer independently
Cascade failure from single component	System-wide outage	Test with chaos engineering; validate isolation and fallback mechanisms

15.13 11. Assumptions and Dependencies

15.13.1 Assumptions

- AWS autoscaling policies are correctly configured for automatic scaling
- Production data volume is accurately represented in test data
- External integrations (ERS/PDS Gateway) are stable and perform consistently
- Network baseline latency remains constant during tests (no ISP/HSCN issues)
- Test environment infrastructure is dimensioned identically to production
- Performance NFRs (300 ms P95, 0.1% error rate) remain the target throughout testing

15.13.2 Dependencies

- AWS Lambda provisioning and API Gateway configuration complete
- DynamoDB and ElastiCache (if used) provisioned and optimized
- CloudWatch and X-Ray monitoring fully configured
- ODIN / Grafana dashboards and alerting set up and tested
- Test data preparation completed
- Load generation tools configured and validated

15.14 12. Sign-Off and Approval

15.14.1 Review and Approval

Role	Name	Signature	Date
Performance Engineering Lead	TBC (see page 115)	TBC (see page 115)	TBC (see page 115)
Architecture Lead	TBC (see page 115)	TBC (see page 115)	TBC (see page 115)
Test Manager	TBC (see page 115)	TBC (see page 115)	TBC (see page 115)
Operations Lead	TBC (see page 115)	TBC (see page 115)	TBC (see page 115)
Project Manager	TBC (see page 115)	TBC (see page 115)	TBC (see page 115)

15.15 13. References

- 7.1.5.4 Performance NFR (see page 115)(Performance NFR)
- 7.1.5.5 Scalability NFR (see page 115)(Scalability NFR)
- 7.1.5.6 Availability NFR (see page 115)(Availability NFR)
- 7.1.5.7 Reliability and Resilience NFR (see page 115)(Reliability NFR)
- NFR: Observability, Monitoring, Metrics, Dashboards and Alerting[42](Monitoring NFR)
- Confluence KPI Page (see page 115)(Confluence-KPI-Page.md)
- AWS Well-Architected Framework - Performance Efficiency Pillar
- AWS Well-Architected Framework - Reliability Pillar

15.16 14. Appendices

15.16.1 Appendix A. Test Scenario Reference Guide

- Scenario 1: Baseline (2 TPS, 30 min)
- Scenario 2: Current Peak (6.56 TPS, 60 min)
- Scenario 3: Future State Stress (Ramp to 120 TPS, 60 min sustained)
- Scenario 4: Endurance (4 TPS, 4-6 hours)
- Scenario 5: Spike Test (2 TPS → 50 TPS → 2 TPS)
- Scenario 6: 2x Traffic (13 TPS, 30 min)
- Scenario 7: Error Scenarios (Exception handling, dependency failures)

15.16.2 Appendix B. JMeter Configuration Templates

Will be provided in separate configuration management documents (see page 115)

15.16.3 Appendix C. CloudWatch and X-Ray Configuration Checklist

- ✓ CloudWatch Log Groups created for all components
- ✓ X-Ray tracing enabled for Lambda and API Gateway
- ✓ Log retention policies configured
- ✓ Metric alarms created for critical thresholds
- ✓ Dashboard created for real-time monitoring

42. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1128208069/NFR+Observability+Monitoring+Metrics+Dashboards+and+Alerting

Document Status: DRAFT - Ready for Review
Next Steps: Stakeholder review and sign-off; Test environment provisioning; Load test tool configuration

15.17 How to create performance test TPS graphs

This page describes a process for creating graphs that portray the actual vs intended TPS of a performance test run.

15.17.1 Sign in to AWS console

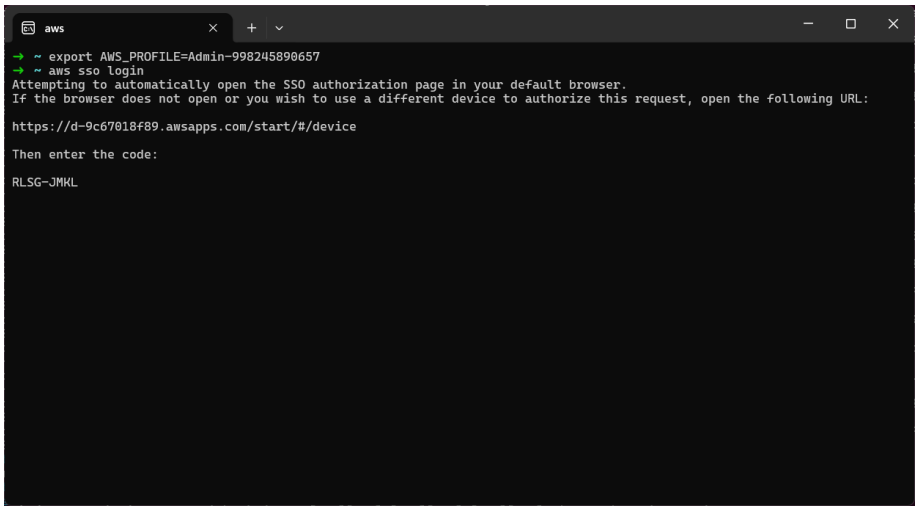
At the time of writing, the performance testing solution is hosted in the **dev** account, but this may move (likely to the **test** one).

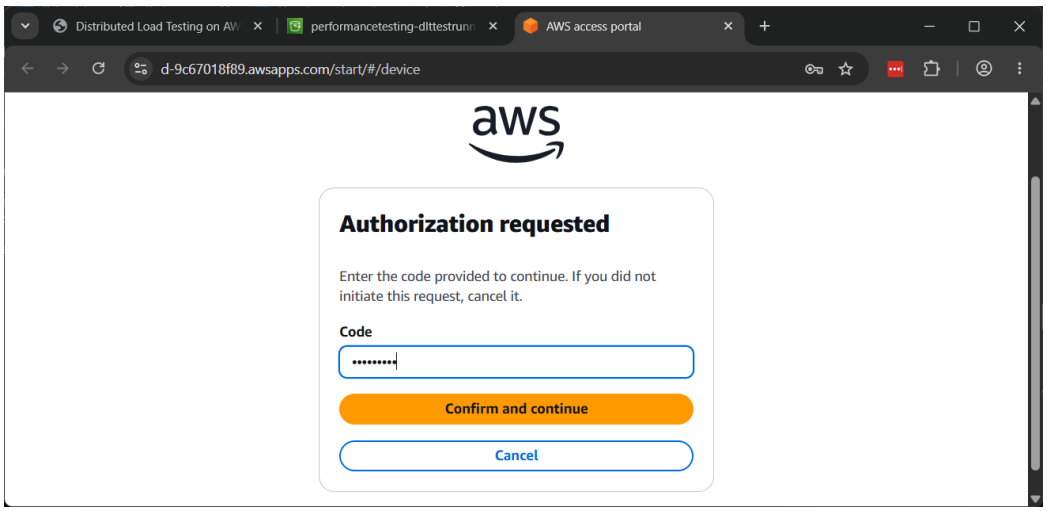
You need **read** access only.

15.17.2 Sign in to AWS CLI

Some of the later steps require the use of the **AWS CLI**, so ensure you are signed in.

```
export AWS_PROFILE=Admin-998245890657 # or whatever profile you are using
aws sso login
```





The terminal will say **Successfully logged into Start URL** unless something goes wrong.

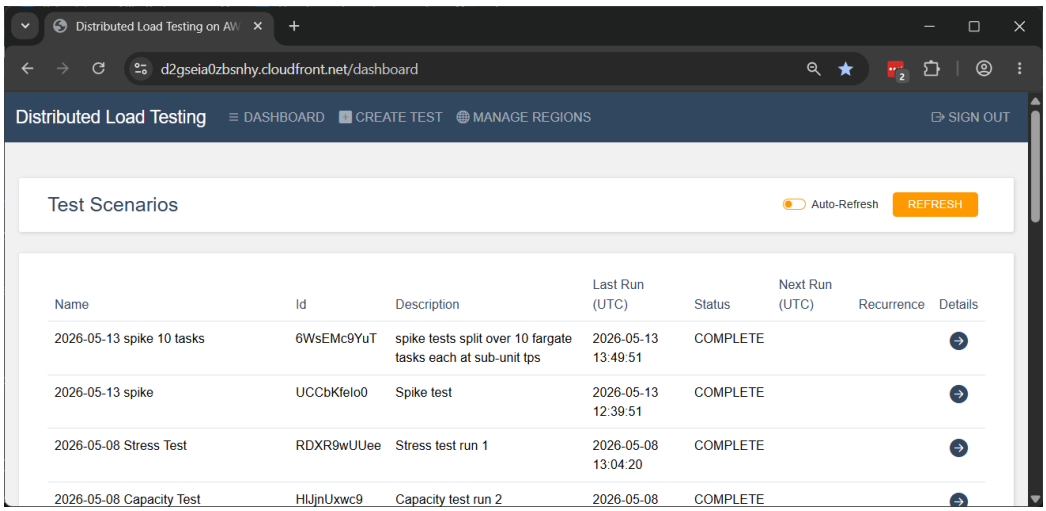
15.17.3 Sign in to the DLT solution

At the time of writing, our performance testing solution is one provided by AWS for Distributed Load Testing, hosted at https://d2gseia0zbsnhy.cloudfront.net/dashboard

If you do not have an account, ask someone with **Admin** access to create you a **Cognito** account in the **performanceTesting-user-pool** user pool.

15.17.4 Go to the run's details

The DLT solution dashboard shows all of the scenarios that have been created with high level info about them, including the status of its most recent run. The link on the very right of the row for the desired run scenario will take you to the details for the scenario.



15.17.5 Open the bucket

A link **Link to results files in S3** is shown at the top of the page.

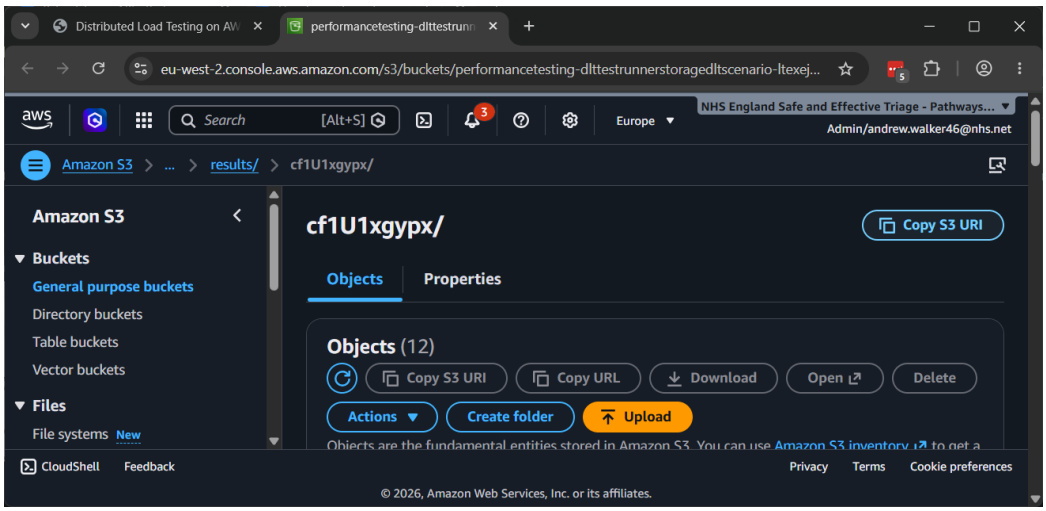
If the most recent run of that scenario is not the one you need, earlier runs are recorded at the bottom of the page in a **Results History** section. Click the correct run's **View Details** link on the right. A link with the same name will be shown in a modal.

Click the link to open the bucket.

15.17.6 Get the S3 URI

Unfortunately, it is not possible to bulk-download the contents of a bucket from the console. Instead, we'll need to get the URI of the bucket and use the AWS CLI to "sync" a local folder with the bucket.

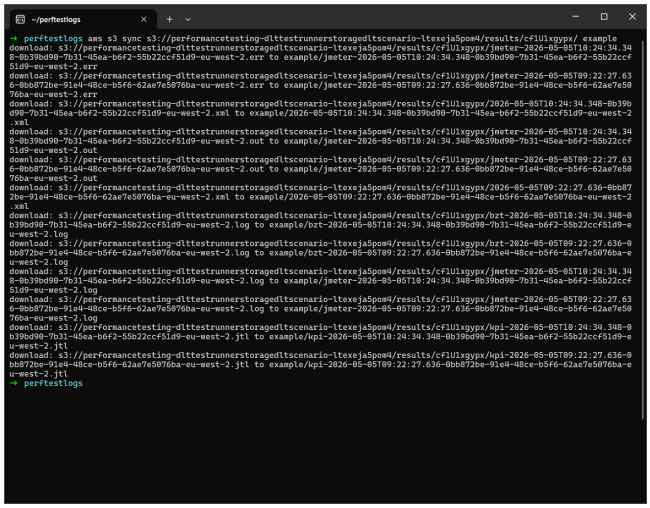
Click the **Copy S3 URI** button in the top right of the S3 page.



15.17.7 Synchronise the bucket

Back in the command line, run

```
aws s3 sync <paste the s3 bucket URI> <target directory>
```



15.17.8 Extract timestamps and TPS

Use command-line tools to extract the timestamps by second and count how many transactions were made in it.

```
cd <target directory>
grep -h 'Latency' jmeter-*.log | egrep -o '[0-9][0-9]:[0-9][0-9]:[0-9][0-9]' | sort |
uniq -c | sed 's/^ *\([^ ][^ ]*\) \(.*\)/\2,\1/' > tps.csv
```

This reads all of the jmeter logs (where transactions are recorded), filters them so only those with the transaction records remain, grabs the timestamps in HH:mm:ss format, sorts them (needed when multiple files have been read), and then groups them with a count of how many time that timestamp was found, and transforms it into csv format.

The file will look like

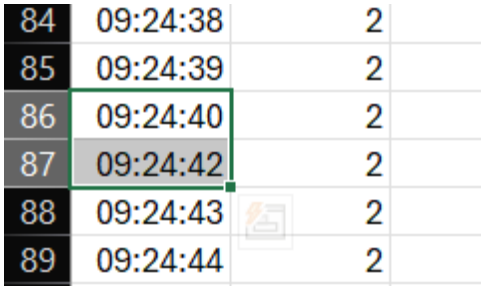
```
09:23:15,2
09:23:16,2
09:23:17,1
09:23:18,3
09:23:19,2
09:23:20,1
09:23:21,2
09:23:22,2
09:23:23,2
09:23:24,2
...
```

15.17.9 Import into excel

Open the tps.csv file in excel. By default, excel might not find *.csv files; if that is the case for you, navigate to the directory with the file and just type in tps.csv, or change the filetype drop-down menu to "All files (*.*)" and select it.

15.17.10 Fill in missing seconds

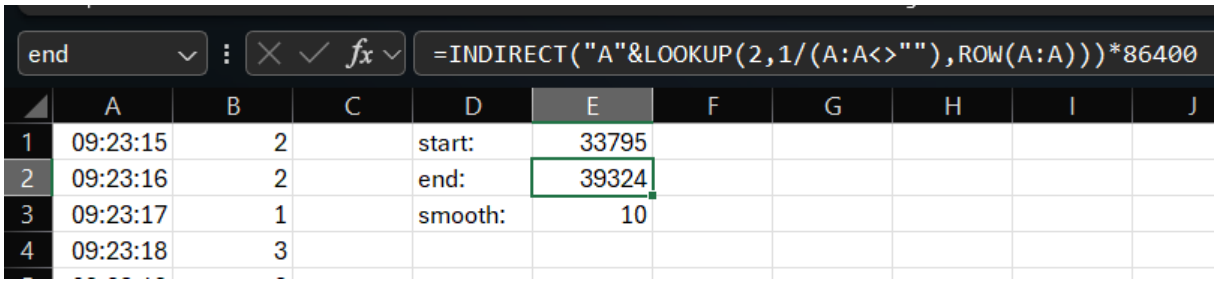
There may some missing seconds, as the previous process will only report on second "buckets" that have at least one entry.



84	09:24:38	2
85	09:24:39	2
86	09:24:40	2
87	09:24:42	2
88	09:24:43	2
89	09:24:44	2

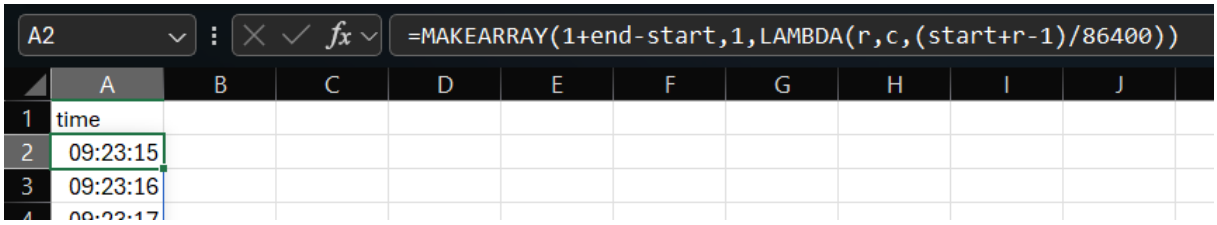
So to make a proper graph, we need to fill in these gaps.

In a blank cell, enter the formula `=A1*86400` , call it "start". In another blank cell, enter the formula `=INDIRECT("A"&LOOKUP(2,1/(A:A<>""),ROW(A:A)))*86400` , call it "end". Format both cells as General. In a third blank cell, enter `10` and call it "smooth". *(Note: by "**call it <whatever>**" I mean giving the cell a name, not the labels in column D - see the top left of the screenshot.)*



Create a new tab. Keep the original tab called "tps" (or rename it to this if it's something else).

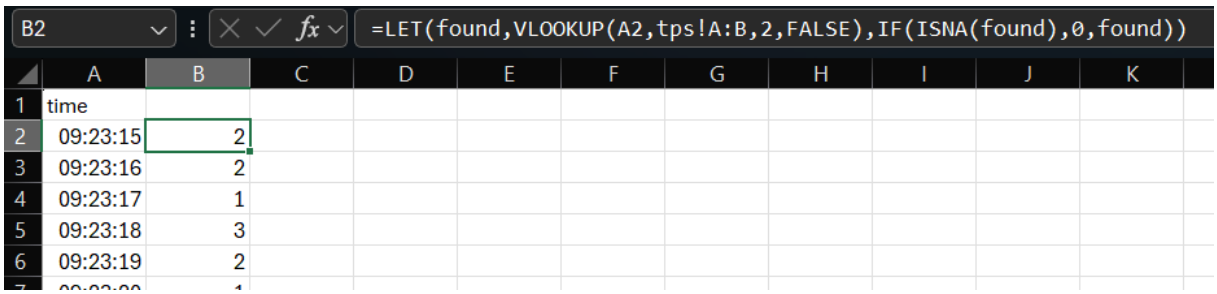
In cell A2 of the new sheet, enter the formula `=MAKEARRAY(1+INT(end-start),1,LAMBDA(r,c,(start+r-1)/86400))` and put the heading "time" above it (A1). If necessary, format the column as "custom", "hh:mm:ss".



This column now includes all of the seconds from the earliest timestamp in the raw data to the latest.

15.17.11 Add the transaction counts

In cell B2 (of the new sheet) enter the formula `=LET(found,VLOOKUP(A2,tps!A:B,2,FALSE),IF(ISNA(found),0,found))` and with that cell selected, double-click on its bottom-right:



Give it a heading of "count".

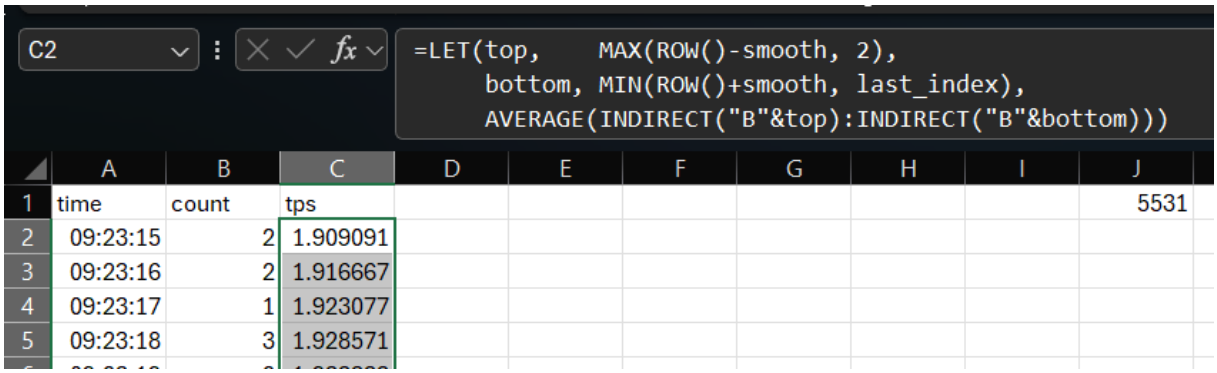
15.17.12 Add smoothed transaction counts

Over to one side, perhaps J1, enter the formula `=LOOKUP(2,1/(A:A<>""),ROW(A:A))` and name the cell "last_index".

In cell C2, enter the formula:

```
=LET(top,    MAX(ROW()-smooth, 2),
     bottom, MIN(ROW()+smooth, last_index),
     AVERAGE(INDIRECT("B"&top):INDIRECT("B"&bottom)))
```

and double-click its bottom right corner to expand it down. Give the column a header of "tps".



15.17.13 Add target tps

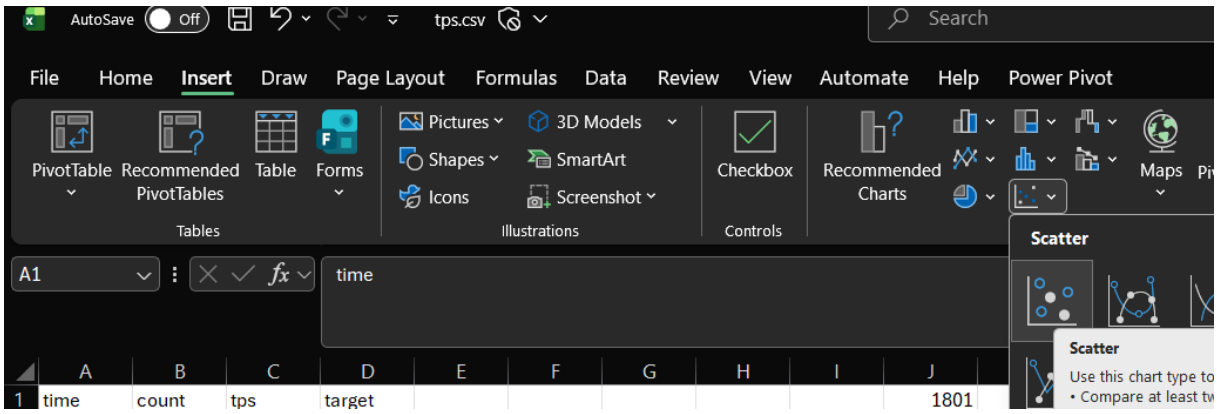
Add a header in cell D1 of "target".

Manually enter the points at which the performance test target tps change. E.g., for a spike test of 2 TPS for 5 min, jumping to 50 TPS for 10 min, before returning to 2 TPS, maybe:

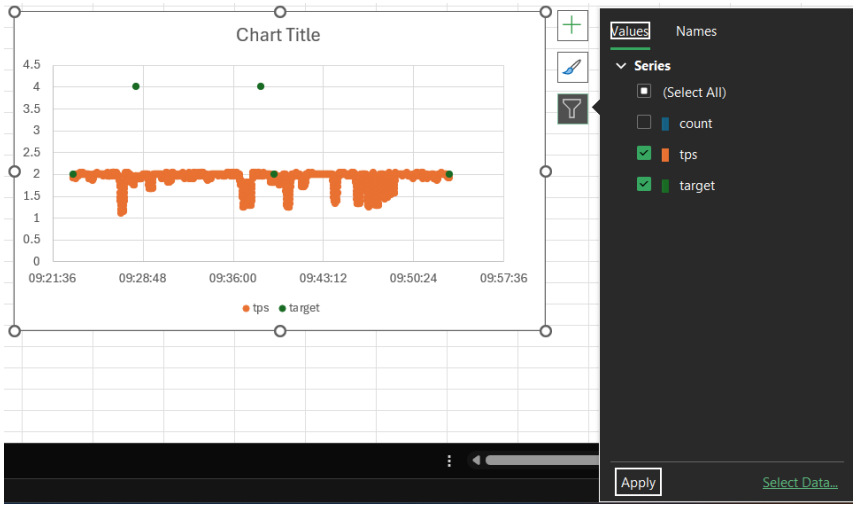
	A	B	C	D
1	time	count	tps	target
2	09:23:15	2	1.909091	2
301	09:28:14	2	1.952381	2
302	09:28:15	1	1.952381	50
902	09:38:15	3	2	50
962	09:39:15	2	1.809524	2
5532				

15.17.14 Create graph

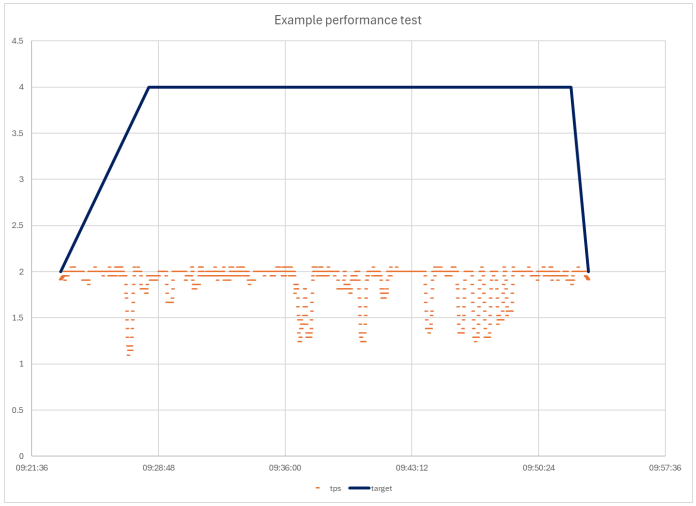
Insert a scatter plot:



By default it will include all the content, but we don't want the original "count". Click the filter icon next to the chart and unselect "count", then click Apply.



You can now format this however you want, though a common style is recommended.



15.18 JMeter Performance Testing Platform — Solution Overview

15.18.1 Purpose

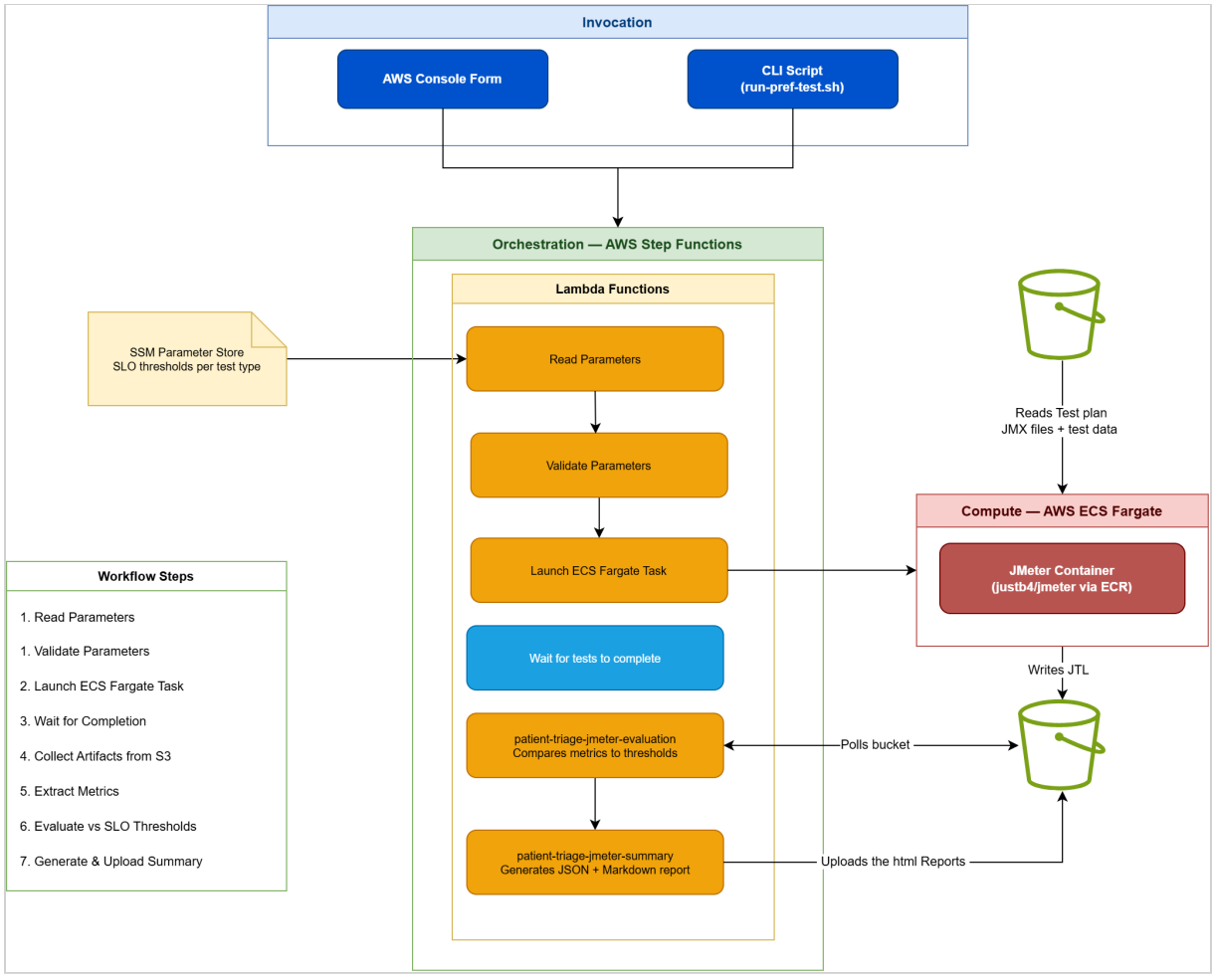
This page documents the complete JMeter performance testing platform for the Patient Triage API. It covers the architecture, infrastructure, test scenarios, how to run tests, and how to interpret results.

15.18.2 Background and Decision

The team identified the need for a repeatable, low-overhead approach to performance testing in AWS. Key pain points driving this platform:

- Test execution was inconsistent and hard to reproduce
- Logs and artifacts had no central storage
- Scaling load generation required manual infrastructure work
- No objective, automated pass/fail evaluation existed

15.18.3 Architecture Overview



Layer	Component	Description
Invocation	AWS Console Form	User fills in a form to trigger a test
Invocation	CLI Script (`run-pref-test.sh`)	User runs a shell command to trigger a test
Orchestration	Step Functions Workflow	Coordinates all steps end-to-end
Orchestration	Validate → Launch → Wait → Collect → Extract → Evaluate → Summarise	Nine automated steps executed in sequence
Compute	ECS Fargate Task	Runs the JMeter container on demand
Compute	JMeter Container (`justb4/jmeter` via ECR)	Executes the test plan and uploads results
Storage	S3: `patient-triage-api-test-plans`	Stores JMX test plan files and test data
Storage	S3: `patient-triage-api-test-artifacts`	Stores JTL results, HTML reports, summaries
Configuration	SSM Parameter Store (`/jmeter/slo-thresholds`)	SLO thresholds loaded at runtime
Configuration	Secrets Manager (`jmeter/pref/api-token`)	API credentials retrieved securely at runtime

Flow: Invocation → Step Functions → ECS Task reads from test-plans S3 → writes results to artifacts S3 → Lambdas extract metrics, evaluate thresholds, generate summary → summary uploaded to S3.

15.18.4 Infrastructure Components

Component	AWS Service	Name	Purpose
Container image	Amazon ECR	`patient-triage-jmeter`	Stores versioned JMeter Docker image

Component	AWS Service	Name	Purpose
Container runtime	ECS Fargate	`jmeter-cluster`	Runs JMeter tasks on demand
Test plans storage	S3	`patient-triage-api-test-plans`	Stores JMX files and test data
Artifacts storage	S3	`patient-triage-api-test-artifacts`	Stores JTL results, HTML reports, summaries
Orchestration	Step Functions	`patient-triage-jmeter-workflow`	Coordinates the end-to-end test execution
Metrics extraction	Lambda	`patient-triage-jmeter-metrics`	Queries CloudWatch and parses JTL
SLO evaluation	Lambda	`patient-triage-jmeter-evaluation`	Compares metrics against thresholds
Summary generation	Lambda	`patient-triage-jmeter-summary`	Generates JSON and Markdown reports
Threshold config	SSM Parameter Store	`/jmeter/slo-thresholds`	Stores SLO thresholds per test type
Secrets	Secrets Manager	`jmeter/pref/api-token`	API credentials for test execution
Logs	CloudWatch Logs	`/aws/states/jmeter-workflow`	Workflow and Lambda execution logs

15.18.5 S3 Folder Structure

15.18.5.1 Test Plans Bucket (`patient-triage-api-test-plans`)

```
baseline/
load/
capacity/
stress/
soak/
spike/
error/
data-layer/
shared/
```

15.18.5.2 Artifacts Bucket (`patient-triage-api-test-artifacts`)

```
baseline/
  run-20260805-140000/
    results.jtl
    report/index.html
    summary.json
    summary.md
load/
capacity/
stress/
soak/
spike/
error/
```

15.18.6 Test Scenarios

Eight test scenarios are defined in the Performance Testing Strategy:

Scenario	Load Profile	Duration	P95 Threshold	Error Threshold
Baseline	2 TPS sustained	30 min	< 300ms	< 0.01%
Load	6.56 TPS sustained	60 min	< 300ms	< 0.1%
Capacity	13 TPS (2× peak)	30 min	< 500ms	< 1.0%

Scenario	Load Profile	Duration	P95 Threshold	Error Threshold
Stress	Ramp 6.56 → 120 TPS over 30 min, hold 60 min	90 min	< 300ms	< 0.1%
Soak	120 TPS continuous	4 hours	< 300ms	< 0.1%
Spike	2 → 50 → 2 TPS rapid ramp	~30 min	< 300ms	< 0.1%
Error Scenarios	Dependency failures, error injection	Varies	N/A	N/A

15.18.7 How the Automated Workflow Runs

When a test is invoked, the Step Functions workflow executes nine steps:

1. **Accept parameters** — test type, release version, environment
2. **Validate parameters** — check test type is known and version format is correct
3. **Launch JMeter task** — start ECS Fargate task with test plan path and S3 output path
4. **Wait for completion** — poll ECS task status; throw error on timeout or failure
5. **Collect artifacts** — verify JTL and HTML report exist in S3; retry up to 3 times
6. **Extract metrics** — query CloudWatch for app metrics; parse JTL for response times
7. **Evaluate results** — compare metrics against SLO thresholds; record pass / fail / watch
8. **Generate summary** — create JSON and Markdown report; upload to S3
9. **Return results** — provide outcome, summary link, and artifact links to caller

15.18.7.1 Error Handling

Scenario	Behaviour
Task failure or timeout	Error thrown — workflow stops immediately
CloudWatch unavailable	Non-fatal — continues with JTL-only metrics
SLO threshold exceeded	Non-fatal — records fail outcome and continues to generate summary

---

15.18.8 How to Run a Test

15.18.8.1 Using the CLI

```
./run-pref-test.sh --test-type baseline --release v1.2.3 --environment pref
```

Optional SLO override:

```
./run-pref-test.sh --test-type capacity --release v1.2.3 --environment pref \
  --slo-override '{"p95_latency_ms": 600}'
```

15.18.8.2 Using the AWS Console

1. Navigate to the AWS Service Catalogue (or Step Functions console)
2. Select **Patient Triage — Run Performance Test**
3. Fill in the form: Test Type, Release Version, Environment
4. Click **Submit**
5. Copy the execution URL shown on confirmation

15.18.8.3 Monitoring a Running Test

- **Step Functions console:** View execution status and step-by-step progress
- **CloudWatch Logs:** `/aws/states/jmeter-workflow` for workflow logs; `/ecs/jmeter` for task logs
- **S3:** Artifacts appear in `patient-triage-api-test-artifacts/<test-type>/` when complete

15.18.9 Understanding Results

After a test completes, a summary is uploaded to S3:

```
# Test Results: Baseline — v1.2.3
Outcome: PASS

- P95 Latency: 285ms  (threshold: 300ms) ✓
- Error Rate:  0.005% (threshold: 0.01%) ✓
```

Outcome	Meaning
PASS	All metrics within thresholds
FAIL	One or more metrics exceeded threshold
WATCH	Results within thresholds but anomaly detected

15.18.10 Access Control

Role	Who	Permissions
`perf-test-operator`	Performance engineers, developers	Invoke tests, read results
`perf-test-admin`	Platform/DevOps team	Invoke tests, update thresholds, manage workflows
CI/CD service account	GitHub Actions	Invoke tests, read results (automated runs)

---

16 Patient Triage Api - Versioning Strategy

16.1 Overview

This versioning strategy defines the standardized approach for version management across projects. It ensures consistent deployments, traceability, quality assurance, and clear communication among engineering, QA, operations, and product stakeholders.

16.2 Semantic Versioning

Projects will follow **semantic versioning** as defined by semver.org[43].

Given a version number **MAJOR.MINOR.PATCH**, increment the:

1. **MAJOR** version when you make incompatible API changes
2. **MINOR** version when you add functionality in a backward compatible manner
3. **PATCH** version when you make backward compatible bug fixes

16.2.1 Additional Labels

Additional labels are available as extensions to the MAJOR.MINOR.PATCH format:

- **Release candidates:** e.g. `1.0.0-rc`
- **Pre-releases:** e.g. `1.0.0-pre.1`
- **QA labels:** e.g. `1.0.0-qa-pass` or `1.0.0-qa-fail`
- **Build metadata:** e.g. `1.0.0+sha-ab12cd3`
  - Added after `+`
  - Purely informational (e.g. commit SHA, build number)
  - Does **not** affect version precedence

16.3 Initial Version

- Development will start at `0.1.0` and incremented following semantic versioning rules
- First formal release intended for production: `1.0.0`

---

43. https://semver.org/

16.4 Automation in GitHub Actions

GitHub Actions automates versioning through the following process:

Stage	Trigger	Version Format	Description
Pre-releases	Auto-generated on `main` merges after build and fast tests	`vMAJOR.MINOR.PATCH-pre.BUILD`	Automatic deployment to dev environment
Release Candidates	Auto-generated on `main` merges after acceptance tests	`vMAJOR.MINOR.PATCH-rc`	Ready for QA testing
Quality Assurance	Triggered manually by QA engineer	`vMAJOR.MINOR.PATCH-qa-pass` or `-qa-fail`	QA validation results
Formal Releases	Triggered manually	`vMAJOR.MINOR.PATCH`	Production-ready release

---

16.5 Version Calculation

Use **conventional commit messages** to automatically determine version bumps:

16.5.1 Commit Message Format

All commit messages must follow this format:

<type>: <JIRA-ID> <description>

The commit contains the following structural elements to communicate intent:

1. **fix:** a commit of type `fix` patches a bug in your codebase (correlates with **PATCH** in Semantic Versioning)
2. **feat:** a commit of type `feat` introduces a new feature to the codebase (correlates with **MINOR** in Semantic Versioning)
3. **BREAKING CHANGE:** a commit that has a footer `BREAKING CHANGE:`, or appends a `!` after the type/scope, introduces a breaking API change (correlates with **MAJOR** in Semantic Versioning)
4. **Other types:** such as `build:`, `chore:`, `ci:`, `docs:`, `style:`, `refactor:`, `perf:`, `test:` **etc...** are allowed but don't trigger version bumps

Exceptions (Auto-Generated Commits)
The following commits are automatically skipped and do NOT require a JIRA ID:
- **Merge commits** - auto-generated by Git
- **Dependabot commits** - automated dependency updates
- **Git revert commits** - auto-generated by `git revert` command (format: `Revert "..."` with commit hash footer)

16.5.2 Examples

```
fix: resolve API timeout issue
→ PATCH bump (1.0.0 → 1.0.1)

feat: add new user authentication endpoint
→ MINOR bump (1.0.0 → 1.1.0)

feat!: migrate to new database schema
BREAKING CHANGE: requires data migration
→ MAJOR bump (1.0.0 → 2.0.0)

improve: NPT-1234 Optimize query performance
→ PATCH bump (custom type with JIRA ID)
```

Reference: Conventional Commits v1.0.0[44]

16.6 Implementation

16.6.1 Formal Releases

- The "Release Build Process" should be followed for the creation of a formal release (i.e. not a tagged release or prerelease build)
- The trigger for physically creating a formal release should be **manual**
- The triggering process will occur in GitHub
- Format: `vMAJOR.MINOR.PATCH` (e.g., `v1.2.3`)

44. https://www.conventionalcommits.org/en/v1.0.0/

16.6.2 Pre-releases

- Pre-releases should be published as part of the **Post-merge Deployment & Pre-release** stage on **pushes** to `develop`
- Pre-releases use the `pre` tag and build versions should be incremented for each new version published
- Format: `vMAJOR.MINOR.PATCH-pre.BUILD-VERSION` (e.g., `v1.2.3-pre.5`)
- There should be a process that can be run to clean up old pre-releases and related artifacts, which can be run manually or on a schedule

16.6.3 Release Candidates

- Release candidates are generated after acceptance tests pass
- Format: `vMAJOR.MINOR.PATCH-rc` (e.g., `v1.2.3-rc`)
- Used for integration and final testing before production deployment

16.6.4 QA Validation

- QA engineers can manually tag releases with validation results
- Format: `vMAJOR.MINOR.PATCH-qa-pass` or `vMAJOR.MINOR.PATCH-qa-fail`
- Provides clear indication of QA approval status

16.7 Environment Promotion Flow

`[Develop] → [Dev] → [Test] → [Integration] → [Reference] → [Production]`

Environment	Source of Truth	Trigger	Deployment Frequency	Purpose
Workspace	`feature/*` or `main` branches	Automatic on push	Per push	Isolated testing of in-progress work
Dev	`develop` branch	Automatic on push	After each merge to `develop`	Early integration testing, regression testing
Test	Tagged release	Manual by QA	Per feature or release	Functional QA, validation, regression, end-to-end tests

Environment	Source of Truth	Trigger	Deployment Frequency	Purpose
Integration	Tagged release	Manual	Pre-release cycle	Integration testing with internal/ external partners
Reference	Tagged release	Manual with approval gates	Pre-release cycle	Pre-live reference, performance testing
Production	Tagged release	Manual with approval gates	On release	Live usage in clinical systems

16.8 Version Lifecycle

16.8.1 Development Phase

```
0.1.0 → 0.2.0-pre.1 → 0.2.0-pre.2 → 0.2.0-rc → 0.2.0-qa-pass → 0.2.0
```

16.8.2 Production Phase

```
1.0.0 → 1.1.0-pre.1 → 1.1.0-rc → 1.1.0-qa-pass → 1.1.0
```

16.9 Best Practices

1. **Always use conventional commits** to ensure automatic version calculation works correctly
2. **Tag releases immediately** after deployment to maintain clear version history
3. **Clean up old pre-releases** regularly to avoid artifact bloat
4. **Document breaking changes** thoroughly in commit messages and release notes
5. **Never manually edit version numbers** - let automation handle it
6. **Use build metadata** for traceability (commit SHA, build number, timestamps)

16.10 Rollback Procedures

16.10.1 Rollback a Deployment

If a released version has issues:

1. Identify the previous stable version from Git tags

2. Go to GitHub Actions → **Deploy Release Pipeline**
3. Click **Run workflow**
4. Enter previous release tag (e.g., `v1.1.0`)
5. Select target environment: `test`, `int`, `ref`, or `prod`
6. Click **Run workflow**
7. Review Terraform plan
8. Manual approval is required before applying changes
9. Wait for completion
10. Verify deployment success

16.11 Clinical Versioning

16.11.1 Specifying Clinical Logic

Clinical logic versions are specified via the **version parameter** in the Accept header:

```
Accept: application/fhir+json; version=49
```

Key Principles:

- **Applies to major logic versions only** - Minor updates are backward-compatible and do not require explicit versioning
- **Frequent updates expected** - Logic updates may occur daily or weekly
- **Clinical safety sign-off required** - All updates must receive clinical safety approval
- **Regression testing mandatory** - Comprehensive regression testing must be performed before deployment

16.11.2 Version Consistency and State Management

API Response Behavior:

- The API response will carry the **version of the Pathways Release** the logic relates to
- The consuming system **must check this version for consistency** as part of local state management
- If the versions do not align, the consuming system **must stop** processing

16.11.3 Parallel Versioning

Version Support Strategy:

- Multiple major versions can be supported in parallel to facilitate gradual migration
- API Management (APIM) tracks version usage and volumes per supplier

- Enables controlled rollout and supplier-specific timelines

16.11.4 Version Retirement Policy

Deprecation Roadmap:

- A formal version retirement policy must be defined and agreed upon during **onboarding and conformance**
- **Recommended deprecation timeline:** 6 months per major version
- Advanced notice will be provided before version retirement

Monitoring and Communication:

1. **velVersion Usage Tracking**
    - Monitor active version usage through APIM
    - Track request volumes per version and supplier
    - Identify migration readiness
2. **Deprecation Notices**
    - 6 months notice before retirement (or as per agreed policy)
    - Regular communications to affected suppliers
    - Migration support and guidance provided
3. **Retirement Process**
    - Final cutoff date announced
    - Grace period for urgent exceptions (case-by-case basis)
    - Version disabled after retirement date

Version Lifecycle Example:

```
Version 48: Active
Version 49: Active (Current)
Version 50: Beta (Limited release)
Version 45: Deprecated (Retire in 3 months)
Version 44: Retired
```

16.11.5 Clinical Versioning Best Practices

1. **Always specify version** in Accept headers for clinical logic requests
2. **Implement version consistency checks** in consuming systems
3. **Subscribe to deprecation notices** to stay informed
4. **Test against beta versions** before they become current
5. **Monitor API responses** for version alignment
6. **Plan migrations** well ahead of retirement dates
7. **Maintain backward compatibility** for minor version updates

16.11.6 Safety Considerations

- ⚠️ **Clinical Safety Critical:** Version mismatches could result in incorrect clinical logic being applied
- ✓ **Always validate** the pathways release version in responses
- ✓ **Implement failsafe mechanisms** to halt processing on version conflicts
- ✓ **Log all version-related issues** for audit and safety review

16.12 References

- Semantic Versioning Specification[45]
- Conventional Commits Specification[46]
- Angular Commit Guidelines[47]

Document Version: 1.0
Last Updated: February 25, 2026

45. https://semver.org/#semantic-versioning-specification-semver
46. https://www.conventionalcommits.org/en/v1.0.0/
47. https://github.com/angular/angular/blob/22b96b9/CONTRIBUTING.md#-commit-message-guidelines